

Benchmarking a New Paradigm: An Experimental Analysis of a Real Processing-in-Memory Architecture

Juan Gómez-Luna¹ Izzat El Hajj² Ivan Fernandez^{1,3} Christina Giannoula^{1,4}
Geraldo F. Oliveira¹ Onur Mutlu¹

¹ETH Zürich ²American University of Beirut ³University of Malaga ⁴National Technical University of Athens

ABSTRACT

Many modern workloads, such as neural networks, databases, and graph processing, are fundamentally memory-bound. For such workloads, the data movement between main memory and CPU cores imposes a significant overhead in terms of both latency and energy. A major reason is that this communication happens through a narrow bus with high latency and limited bandwidth, and the low data reuse in memory-bound workloads is insufficient to amortize the cost of main memory access. Fundamentally addressing this *data movement bottleneck* requires a paradigm where the memory system assumes an active role in computing by integrating processing capabilities. This paradigm is known as *processing-in-memory* (PIM).

Recent research explores different forms of PIM architectures, motivated by the emergence of new 3D-stacked memory technologies that integrate memory with a logic layer where processing elements can be easily placed. Past works evaluate these architectures in simulation or, at best, with simplified hardware prototypes. In contrast, the UPMEM company has designed and manufactured the first publicly-available real-world PIM architecture. The UPMEM PIM architecture combines traditional DRAM memory arrays with general-purpose in-order cores, called *DRAM Processing Units* (DPUs), integrated in the same chip.

This paper provides the first comprehensive analysis of the first publicly-available real-world PIM architecture. We make two key contributions. First, we conduct an experimental characterization of the UPMEM-based PIM system using microbenchmarks to assess various architecture limits such as compute throughput and memory bandwidth, yielding new insights. Second, we present *PrIM* (*Processing-In-Memory benchmarks*), a benchmark suite of 16 workloads from different application domains (e.g., dense/sparse linear algebra, databases, data analytics, graph processing, neural networks, bioinformatics, image processing), which we identify as memory-bound. We evaluate the performance and scaling characteristics of *PrIM* benchmarks on the UPMEM PIM architecture, and compare their performance and energy consumption to their state-of-the-art CPU and GPU counterparts. Our extensive evaluation conducted on two real UPMEM-based PIM systems with 640 and 2,556 DPUs provides new insights about suitability of different workloads to the PIM system, programming recommendations for software designers, and suggestions and hints for hardware and architecture designers of future PIM systems.

KEYWORDS

processing-in-memory, near-data processing, memory systems, data movement bottleneck, DRAM, benchmarking, real-system characterization, workload characterization

1 INTRODUCTION

In modern computing systems, a large fraction of the execution time and energy consumption of modern data-intensive workloads is spent moving data between memory and processor cores. This *data movement bottleneck* [1–7] stems from the fact that, for decades, the performance of processor cores has been increasing at a faster rate than the memory performance. The gap between an arithmetic operation and a memory access in terms of latency and energy keeps widening and the memory access is becoming increasingly more expensive. As a result, recent experimental studies report that data movement accounts for 62% [8] (reported in 2018), 40% [9] (reported in 2014), and 35% [10] (reported in 2013) of the total system energy in various consumer, scientific, and mobile applications, respectively.

One promising way to alleviate the data movement bottleneck is *processing-in-memory* (PIM), which equips memory chips with processing capabilities [2]. This paradigm has been explored for more than 50 years [1–4, 8, 11–131], but limitations in memory technology prevented commercial hardware from successfully materializing. More recently, difficulties in DRAM scaling (i.e., challenges in increasing density and performance while maintaining reliability, latency and energy consumption) [132–166] have motivated innovations such as 3D-stacked memory [63, 167–172] and nonvolatile memory [146, 173–183] which present new opportunities to redesign the memory subsystem while integrating processing capabilities. 3D-stacked memory integrates DRAM layers with a logic layer, which can embed processing elements. Several works explore this approach, called *processing-near-memory* (PNM), to implement different types of processing components in the logic layer, such as general-purpose cores [8, 56, 63–67, 128, 184], application-specific accelerators [57–61, 69, 79, 81, 82, 88, 89, 101, 104, 105, 107, 108, 124–127, 129–131, 185, 186], simple functional units [62, 84, 120, 122, 123, 187], GPU cores [78, 80, 85, 87], or reconfigurable logic [68, 73, 75, 121]. However, 3D-stacked memory suffers from high cost and limited capacity, and the logic layer has hardware area and thermal dissipation constraints, which limit the capabilities of the embedded processing components. On the other hand, *processing-using-memory* (PUM) takes advantage of the analog operational properties of memory cells in SRAM [21–24], DRAM [25–40, 99, 100, 119, 188–191], or nonvolatile memory [41–55, 70, 103, 114, 118, 192–196] to perform specific types of operations efficiently. However, processing-using-memory is either limited to simple bitwise operations (e.g., majority, AND, OR) [21, 25, 26, 188, 191], requires high area overheads to perform more complex operations [35, 36, 99], or requires significant changes to data organization, manipulation, and handling mechanisms to enable bit-serial computation, while still having limitations

on certain operations [29, 37, 40].¹ Moreover, processing-using-memory approaches are usually efficient mainly for regular computations, since they naturally operate on a large number of memory cells (e.g., entire rows across many subarrays [25–28, 37, 188–191, 197]) simultaneously. For these reasons, complete PIM systems based on 3D-stacked memory or processing-using-memory have not yet been commercialized in real hardware.

The UPMEM PIM architecture [198, 199] is the first PIM system to be commercialized in real hardware. To avoid the aforementioned limitations, it uses conventional 2D DRAM arrays and combines them with general-purpose processing cores, called *DRAM Processing Units (DPUs)*, on the same chip. Combining memory and processing components on the same chip imposes serious design challenges. For example, DRAM designs use only three metal layers [200, 201], while conventional processor designs typically use more than ten [199, 202–204]. While these challenges prevent the fabrication of fast logic transistors, UPMEM overcomes these challenges via DPU cores that are relatively deeply pipelined and fine-grained multithreaded [205–209] to run at several hundred megahertz. The UPMEM PIM architecture provides several key advantages with respect to other PIM proposals. First, it relies on mature 2D DRAM design and fabrication process, avoiding the drawbacks of emerging 3D-stacked memory technology. Second, the general-purpose DPUs support a wide variety of computations and data types, similar to simple modern general-purpose processors. Third, the architecture is suitable for irregular computations because the threads in a DPU can execute independently of each other (i.e., they are not bound by lockstep execution as in SIMD²). Fourth, UPMEM provides a complete software stack that enables DPU programs to be written in the commonly-used C language [213].

Rigorously understanding the UPMEM PIM architecture, the first publicly-available PIM architecture, and its suitability to various workloads can provide valuable insights to programmers, users and architects of this architecture as well as of future PIM systems. To this end, our work provides the first comprehensive experimental characterization and analysis of the first publicly-available real-world PIM architecture. To enable our experimental studies and analyses, we develop new microbenchmarks and a new benchmark suite, which we openly and freely make available [214].

We develop a set of microbenchmarks to evaluate, characterize, and understand the limits of the UPMEM-based PIM system, yielding new insights. First, we obtain the compute throughput of a DPU for different arithmetic operations and data types. Second, we measure the bandwidth of two different memory spaces that a DPU can directly access using load/store instructions: (1) a DRAM bank called *Main RAM (MRAM)*, and (2) an SRAM-based scratchpad memory called *Working RAM (WRAM)*. We employ streaming (i.e., unit-stride), strided, and random memory access patterns to measure the sustained bandwidth of both types of memories. Third, we measure the sustained bandwidth between the standard main

memory and the MRAM banks for different types and sizes of transfers, which is important for the communication of the DPU with the host CPU and other DPUs.

We present *PrIM (Processing-In-Memory benchmarks)*, the first benchmark suite for a real PIM architecture. PrIM includes 16 workloads from different application domains (e.g., dense/sparse linear algebra, databases, data analytics, graph processing, neural networks, bioinformatics, image processing), which we identify as memory-bound using the roofline model for a conventional CPU [215]. We perform strong scaling³ and weak scaling⁴ experiments with the 16 benchmarks on a system with 2,556 DPUs, and compare their performance and energy consumption to their modern CPU and GPU counterparts. Our extensive evaluation provides new insights about suitability of different workloads to the PIM system, programming recommendations for software designers, and suggestions and hints for hardware and architecture designers of future PIM systems. All our microbenchmarks and PrIM benchmarks are publicly and freely available [214] to serve as programming samples for real PIM architectures, evaluate and compare current and future PIM systems, and help further advance PIM architecture, programming, and software research.⁵

The main contributions of this work are as follows:

- We perform the first comprehensive characterization and analysis of the first publicly-available real-world PIM architecture. We analyze the new architecture’s potential, limitations and bottlenecks. We analyze (1) memory bandwidth at different levels of the DPU memory hierarchy for different memory access patterns, (2) DPU compute throughput of different arithmetic operations for different data types, and (3) strong and weak scaling characteristics for different computation patterns. We find that (1) the UPMEM PIM architecture is fundamentally compute bound, since workloads with more complex operations than integer addition fully utilize the instruction pipeline before they can potentially saturate the memory bandwidth, and (2) workloads that require inter-DPU communication do *not* scale well, since there is no direct communication channel among DPUs, and therefore, all inter-DPU communication takes place via the host CPU, i.e., through the narrow memory bus.
- We present and open-source PrIM, the first benchmark suite for a real PIM architecture, composed of 16 real-world workloads that are memory-bound on conventional processor-centric systems. The workloads have different characteristics, exhibiting heterogeneity in their memory access patterns, operations and data types, and communication patterns. The PrIM benchmark suite provides a common set of workloads to evaluate the UPMEM PIM architecture with and can be useful for programming, architecture and systems researchers all alike to improve multiple aspects of future PIM hardware and software.⁵
- We compare the performance and energy consumption of PrIM benchmarks on two UPMEM-based PIM systems with 2,556 DPUs and 640 DPUs to modern conventional processor-centric systems,

¹PUM approaches performing bit-serial computation [29, 37, 40] need to layout data elements vertically (i.e., all bits of an element in the same bitline), which (1) does *not* allow certain data manipulation operations (e.g., shuffling of data elements in an array) and (2) requires paying the overhead of bit transposition, when the format of data needs to change [37], i.e., prior to performing bit-serial computation.

²Single Instruction Multiple Data (SIMD) [210–212] refers to an execution paradigm where multiple processing elements execute the *same* operation on *multiple* data elements simultaneously.

³Strong scaling refers to how the execution time of a program solving a particular problem varies with the number of processors for a fixed problem size [216, 217].

⁴Weak scaling refers to how the execution time of a program solving a particular problem varies with the number of processors for a fixed problem size per processor [216, 218].

⁵We refer the reader to a recent overview paper [2] on the state-of-the-art challenges in PIM research.

i.e., CPUs and GPUs. Our analysis reveals several new and interesting findings. We highlight three major findings. First, both UPMEM-based PIM systems outperform a modern CPU (by 93.0× and 27.9×, on average, respectively) for 13 of the PrIM benchmarks, which do *not* require intensive inter-DPU synchronization or floating point operations.⁶ Section 5.2 provides a detailed analysis of our comparison of PIM systems to modern CPU and GPU. Second, the 2,556-DPU PIM system is faster than a modern GPU (by 2.54×, on average) for 10 PrIM benchmarks with (1) streaming memory accesses, (2) little or no inter-DPU synchronization, and (3) little or no use of complex arithmetic operations (i.e., integer multiplication/division, floating point operations).⁷ Third, energy consumption comparison of the PIM, CPU, and GPU systems follows the same trends as the performance comparison: the PIM system yields large energy savings over the CPU and the GPU, for workloads where it largely outperforms the CPU and the GPU. We are comparing the first ever commercial PIM system to CPU and GPU systems that have been heavily optimized for decades in terms of architecture, software, and manufacturing. Even then, we see significant advantages of PIM over CPU and GPU in most PrIM benchmarks (Section 5.2). We believe the architecture, software, and manufacturing of PIM systems will continue to improve (e.g., we suggest optimizations and areas for future improvement in Section 6). As such, more fair comparisons to CPU and GPU systems would be possible and can reveal higher benefits for PIM systems in the future.

2 UPMEM PIM ARCHITECTURE

We describe the organization of a UPMEM PIM-enabled system (Section 2.1), the architecture of a DPU core (Section 2.2), and important aspects of programming DPUs (Section 2.3).

2.1 System Organization

Figure 1 (left) depicts a UPMEM-based PIM system with (1) a *host* CPU (e.g., an x86 [219], ARM64 [220], or 64-bit RISC-V [221] multi-core system), (2) standard main memory (DRAM memory modules [222–225]), and (3) PIM-enabled memory (UPMEM modules) [198, 199]. PIM-enabled memory can reside on one or more memory channels. A UPMEM module is a standard DDR4-2400 DIMM (module) [226] with several PIM chips. Figure 2 shows two UPMEM modules. All DPUs in the UPMEM modules operate together as a parallel coprocessor to the host CPU.

Inside each UPMEM PIM chip (Figure 1 (right)), there are 8 DPUs. Each DPU has exclusive access to (1) a 64-MB DRAM bank, called *Main RAM (MRAM)* ❶, (2) a 24-KB instruction memory, called *Instruction RAM (IRAM)* ❷, and (3) a 64-KB scratchpad memory, called *Working RAM (WRAM)* ❸. MRAM is accessible by the host CPU (Figure 1 (left)) for *copying* input data (from main memory to

MRAM) ❹ and *retrieving* results (from MRAM to main memory) ❺. These CPU-DPU and DPU-CPU data transfers can be performed in parallel (i.e., concurrently across multiple MRAM banks), if the buffers transferred from/to all MRAM banks are of the same size. Otherwise, the data transfers happen serially (i.e., a transfer from/to another MRAM bank starts after the transfer from/to an MRAM bank completes). There is no support for direct communication between DPUs. All inter-DPU communication takes place through the host CPU by *retrieving* results from the DPU to the CPU and *copying* data from the CPU to the DPU. In current UPMEM-based PIM systems, concurrent host CPU and DPU accesses to the same MRAM bank are *not* possible.

The programming interface for serial transfers [213] provides functions for copying a buffer to (`dpu_copy_to`) and from (`dpu_copy_from`) a specific MRAM bank. The programming interface for parallel transfers [213] provides functions for assigning buffers to specific MRAM banks (`dpu_prepare_xfer`) and then initiating the actual CPU-DPU or DPU-CPU transfers to execute in parallel (`dpu_push_xfer`). Parallel transfers require that the transfer sizes to/from all MRAM banks be the same. If the buffer to copy to all MRAM banks is the same, we can execute a broadcast CPU-DPU memory transfer (`dpu_broadcast_to`).

Main memory and PIM-enabled memory require different data layouts. While main memory uses the conventional horizontal DRAM mapping [199, 228], which maps consecutive 8-bit words onto consecutive DRAM chips, PIM-enabled memory needs entire 64-bit words mapped onto the same MRAM bank (in one PIM chip) [199]. The reason for this special data layout in PIM-enabled memory is that each DPU has access to only a single MRAM bank, but it can operate on data types of up to 64 bits. The UPMEM SDK includes a transposition library [199] to perform the necessary data shuffling when transferring data between main memory and MRAM banks. These data layout transformations are transparent to programmers. The UPMEM SDK-provided functions for serial/parallel/broadcast CPU-DPU and serial/parallel DPU-CPU transfers call the transposition library internally, and the library ultimately performs data layout conversion, as needed.

The host CPU can allocate the desired number of DPUs, i.e., a *DPU set*, to execute a DPU function or *kernel*. Then, the host CPU launches the DPU kernel *synchronously* or *asynchronously* [213]. Synchronous execution suspends the host CPU thread until the DPU set completes the kernel execution. Asynchronous execution returns immediately the control to the host CPU thread, which can later check the completion status [213].⁸

In current UPMEM-based PIM system configurations [227], the maximum number of UPMEM DIMMs is 20. A UPMEM-based PIM system with 20 UPMEM modules can contain up to 2,560 DPUs which amounts to 160 GB of PIM-capable memory.

Table 1 presents the two real UPMEM-based PIM systems that we use in this work.

We use a real UPMEM-based PIM system that contains 2,556 DPUs, and a total of 159.75 GB MRAM. The DPUs are organized into

⁶Two of the other three PrIM benchmarks, Breadth-first Search (BFS) and Needleman-Wunsch (NW), pay the huge overhead of inter-DPU synchronization via the host CPU. The third one, Sparse Matrix-Vector Multiply (SpMV), makes intensive use of floating point multiplication and addition.

⁷We also evaluate the 640-DPU PIM system and find that it is *slower* than the GPU for most PrIM benchmarks, but the performance gap between the two systems (640-DPU PIM and GPU) is significantly smaller for the 10 PrIM benchmarks that do *not* need (1) heavy inter-DPU communication or (2) intensive use of multiplication operations. The 640-DPU PIM system is faster than the GPU for two benchmarks, which are not well-suited for the GPU. Section 5.2 provides a detailed analysis of our comparison.

⁸In this work, we only use synchronous execution, since our benchmarks (Section 4)) do *not* require the host CPU to compute while a DPU kernel is running. We believe exploring the use of asynchronous execution is a promising topic for future work.

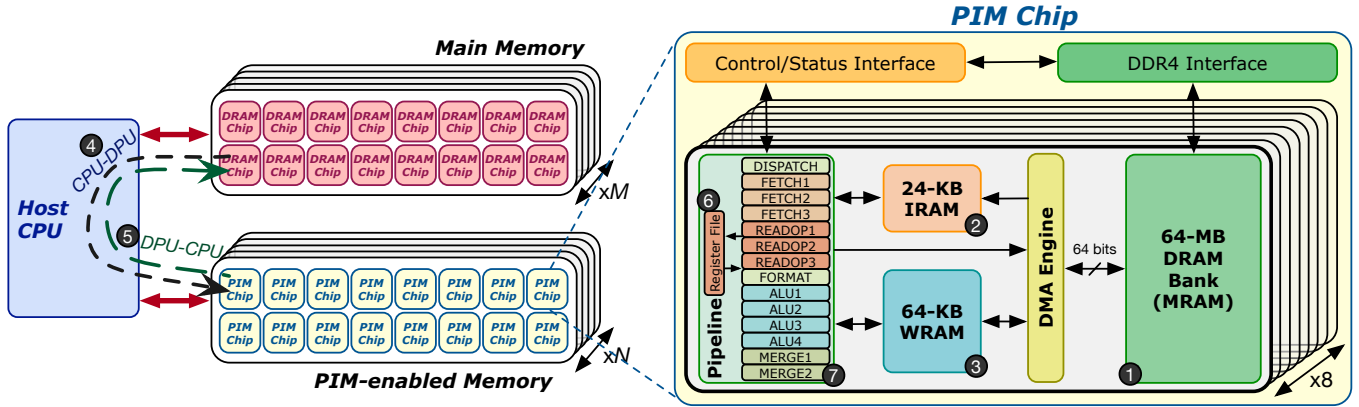


Figure 1: UPMEM-based PIM system with a host CPU, standard main memory, and PIM-enabled memory (left), and internal components of a UPMEM PIM chip (right) [198, 199].

Table 1: UPMEM-based PIM Systems.
(a) Memory Parameters.

System	DIMM Codename	Number of DIMMs	PIM-enabled Memory			DPU Frequency	Total Memory	DRAM Memory	
			Ranks/ DIMM	DPUs/ DIMM	Total DPUs			Number of DIMMs	Total Memory
2,556-DPU System	P21	20	2	128	2,556 ⁹	350 MHz	159.75 GB	4	256 GB
640-DPU System	E19	10	1	64	640	267 MHz	40 GB	2	64 GB

(b) CPU Parameters.

System	CPU				
	CPU Processor	CPU Frequency	CPU Sockets	Mem. Controllers/ Socket	Channels/ Mem. Controller
2,556-DPU System	Intel Xeon Silver 4215 [229]	2.50 GHz	2	2	3
640-DPU System	Intel Xeon Silver 4110 [230]	2.10 GHz	1	2	3

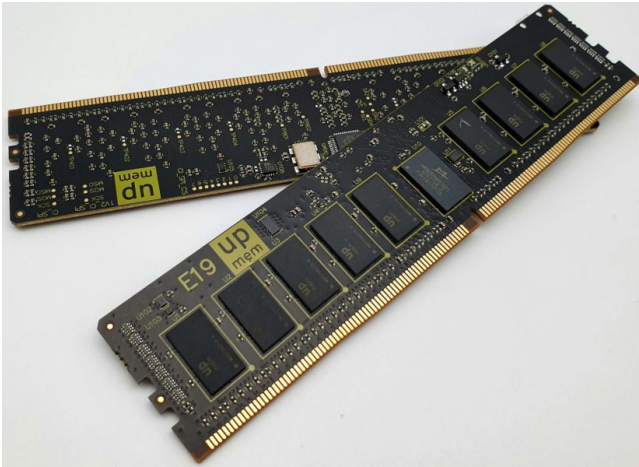


Figure 2: UPMEM-based PIM modules (downloaded from [227]).

20 double-rank DIMMs, with 128 DPUs per DIMM.⁹ Each DPU runs at 350 MHz. The 20 UPMEM DIMMs are in a dual x86 socket with 2 memory controllers per socket. Each memory controller has 3 memory channels [229]. In each socket, two DIMMs of conventional

⁹There are four faulty DPUs in the system where we run our experiments. They cannot be used and do not affect system functionality or the correctness of our results, but take away from the system's full computational power of 2,560 DPUs.

DRAM (employed as main memory of the host CPU) are on one channel of one of the memory controllers. Figure 3 shows a UPMEM-based PIM system with 20 UPMEM DIMMs.

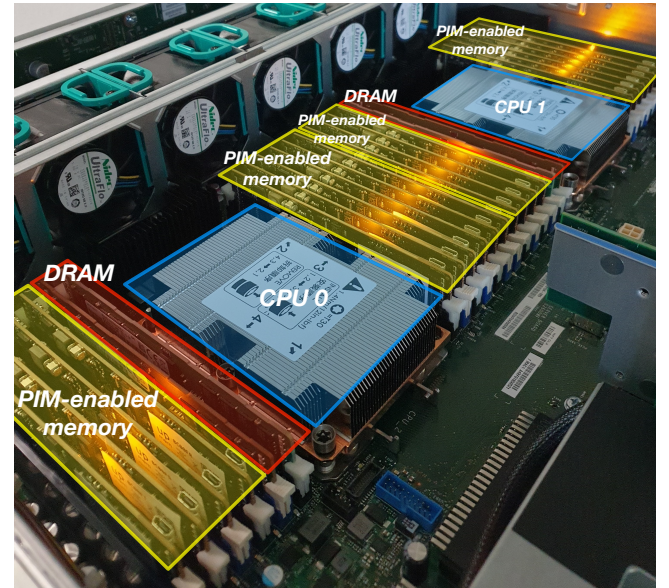


Figure 3: UPMEM-based PIM system with 2,560 DPUs. See Table 1 for the specifications.

We also use an older real system with 640 DPUs. The DPUs are organized into 10 single-rank DIMMs, with 64 DPUs per DIMM. The total amount of MRAM is thus 40 GB. Each DPU in this system runs at 267 MHz. The 10 UPMEM DIMMs are in an x86 socket with 2 memory controllers. Each memory controller has 3 memory channels [230]. Two DIMMs of conventional DRAM are on one channel of one of the memory controllers.

2.2 DRAM Processing Unit (DPU) Architecture

A DPU (Figure 1 (right)) is a multithreaded in-order 32-bit RISC core with a specific Instruction Set Architecture (ISA) [213]. The DPU has 24 hardware threads, each with 24 32-bit general-purpose registers (⑥ in Figure 1 (right)). These hardware threads share an instruction memory (IRAM) ② and a scratchpad memory (WRAM) ③ to store operands. The DPU has a pipeline depth of 14 stages ⑦, however, only the last three stages of the pipeline (i.e., ALU4, MERGE1, and MERGE2 in Figure 1 (right)) can execute in parallel with the DISPATCH and FETCH stages of the next instruction in the same thread. Therefore, instructions from the same thread must be dispatched 11 cycles apart, requiring at least 11 threads to fully utilize the pipeline [231].

The 24 KB IRAM can hold up to 4,096 48-bit encoded instructions. The WRAM has a capacity of 64 KB. The DPU can access the WRAM through 8-, 16-, 32-, and 64-bit load/store instructions. The ISA provides DMA instructions [213] to move instructions from the MRAM bank to the IRAM, and data between the MRAM bank and the WRAM.

The frequency of a DPU can potentially reach more than 400 MHz [227]. At 400 MHz, the maximum possible MRAM-WRAM bandwidth per DPU can achieve around 800 MB/s. Thus, the maximum aggregated MRAM bandwidth for a configuration with 2,560 DPUs can potentially be 2 TB/s. However, the DPUs run at 350 MHz in our 2,556-DPU setup and at 267 MHz in the 640-DPU system. For this reason, the maximum possible MRAM-WRAM bandwidth per DPU in our setup is 700 MB/s (534 MB/s in the 640-DPU setup), and the maximum aggregated bandwidth for the 2,556 DPUs is 1.7 TB/s (333.75 GB/s in the 640-DPU system).

2.3 DPU Programming

UPMEM-based PIM systems use the *Single Program Multiple Data (SPMD)* [232] programming model, where software threads, called *tasklets*, (1) execute the same code but operate on different pieces of data, and (2) can execute different control-flow paths at runtime.

Up to 24 tasklets can run on a DPU, since the number of hardware threads is 24. Programmers determine the number of tasklets per DPU at compile time, and tasklets are statically assigned to each DPU.

Tasklets inside the same DPU can share data among each other in MRAM and in WRAM, and can synchronize via *mutexes*, *barriers*, *handshakes*, and *semaphores* [233].

Tasklets in different DPUs do *not* share memory or any direct communication channel. As a result, they cannot directly communicate or synchronize. As mentioned in Section 2.1, the host CPU handles communication of intermediate data between DPUs, and merges partial results into final ones.

2.3.1 Programming Language and Runtime Library. DPU programs are written in the C language with some library calls [198, 213].¹⁰ The UPMEM SDK [234] supports common data types supported in the C language and the LLVM compilation framework [235]. For the complete list of supported instructions, we refer the reader to the UPMEM user manual [213].

The UPMEM runtime library [213] provides library calls to move (1) instructions from the MRAM bank to the IRAM, and (2) data between the MRAM bank and the WRAM (namely, `mram_read()` for MRAM-WRAM transfers, and `mram_write()` for WRAM-MRAM transfers).

The UPMEM runtime library also provides functions to (1) lock and unlock mutexes (`mutex_lock()`, `mutex_unlock()`), which create critical sections, (2) access barriers (`barrier_wait()`), which suspend tasklet execution until all tasklets in the DPU reach the same point in the program, (3) wait for and notify a handshake (`handshake_wait_for()`, `handshake_notify()`), which enables one-to-one tasklet synchronization, and (4) increment and decrement semaphore counters (`sem_give()`, `sem_take()`).

Even though using the C language to program the DPUs ensures a low learning curve, programmers need to deal with several challenges. First, programming thousands of DPUs running up to 24 tasklets requires careful workload partitioning and orchestration. Each tasklet has a tasklet ID that programmers can use for that purpose. Second, programmers have to explicitly move data between the standard main memory and the MRAM banks, and ensuring data coherence between the CPU and DPUs (i.e., ensuring that CPU and DPUs use up-to-date and correct copies of data) is their responsibility. Third, DPUs do *not* employ cache memories. The data movement between the MRAM banks and the WRAM is explicitly managed by the programmer.

2.3.2 General Programming Recommendations. General programming recommendations of the UPMEM-based PIM system that we find in the UPMEM programming guide [213], presentations [199], and white papers [198] are as follows.

The first recommendation is to **execute on the DPUs portions of parallel code that are as long as possible**, avoiding frequent interactions with the host CPU. This recommendation minimizes CPU-DPU and DPU-CPU transfers, which happen through the narrow memory bus (Section 2.1), and thus cause a data movement bottleneck [2–4, 7], which the PIM paradigm promises to alleviate.

The second recommendation is to **split the workload into independent data blocks**, which the DPUs operate on independently (and concurrently). This recommendation maximizes parallelism and minimizes the need for inter-DPU communication and synchronization, which incurs high overhead, as it happens via the host CPU using CPU-DPU and DPU-CPU transfers.

The third recommendation is to **use as many working DPUs in the system as possible**, as long as the workload is sufficiently large to keep the DPUs busy performing actual work. This recommendation maximizes parallelism and increases utilization of the compute resources.

The fourth recommendation is to **launch at least 11 tasklets in each DPU**, in order to fully utilize the fine-grained multithreaded pipeline, as mentioned in Section 2.2.

¹⁰In this work, we use UPMEM SDK 2021.1.1 [234].

GENERAL PROGRAMMING RECOMMENDATIONS

1. Execute on the *DRAM Processing Units (DPUs)* portions of parallel code that are as long as possible.
2. Split the workload into independent data blocks, which the DPUs operate on independently.
3. Use as many working DPUs in the system as possible.
4. Launch at least 11 *tasklets* (i.e., software threads) per DPU.

In this work, we perform the first comprehensive characterization and analysis of the UPMEM PIM architecture, which allows us to (1) validate these programming recommendations and identify for which workload characteristics they hold, as well as (2) propose additional programming recommendations and suggestions for future PIM software designs, and (3) propose suggestions and hints for future PIM hardware designs, which can enable easier programming as well as broad applicability of the hardware to more workloads.

3 PERFORMANCE CHARACTERIZATION OF A UPMEM DPU

This section presents the first performance characterization of a UPMEM DPU using microbenchmarks to assess various architectural limits and bottlenecks. Section 3.1 evaluates the throughput of arithmetic operations and WRAM bandwidth of a DPU using a streaming microbenchmark. Section 3.2 evaluates the sustained bandwidth between MRAM and WRAM. Section 3.3 evaluates the impact of the operational intensity of a workload on the arithmetic throughput of the DPU. Finally, Section 3.4 evaluates the bandwidth between the main memory of the host and the MRAM banks. Unless otherwise stated, we report experimental results on the larger, 2,556-DPU system presented in Section 2.1. All observations and trends identified in this section also apply to the older 640-DPU system (we verified this experimentally). All microbenchmarks used in this section are publicly and freely available [214].

3.1 Arithmetic Throughput and WRAM Bandwidth

The DPU pipeline is capable of performing one integer addition/subtraction operation every cycle and up to one 8-byte WRAM load/store every cycle when the pipeline is full [199]. Therefore, at 350 MHz, the theoretical peak arithmetic throughput is 350 Millions of Operations per Second (MOPS), assuming only integer addition operations are issued into the pipeline, and the theoretical peak WRAM bandwidth is 2,800 MB/s. In this section, we evaluate the arithmetic throughput and sustained WRAM bandwidth that can be achieved by a streaming microbenchmark (i.e., a benchmark with unit-stride access to memory locations) and how the arithmetic throughput and WRAM bandwidth vary with the number of tasklets deployed.

3.1.1 Microbenchmark Description. To evaluate arithmetic throughput and WRAM bandwidth in streaming workloads, we implement a set of microbenchmarks [214] where every tasklet loops

over elements of an array in WRAM and performs read-modify-write operations. We measure the time it takes to perform WRAM loads, arithmetic operations, WRAM stores, and loop control. We do *not* measure the time it takes to perform MRAM-WRAM DMA transfers (we will study them separately in Section 3.2).

Arithmetic Throughput. For arithmetic throughput, we examine the addition, subtraction, multiplication, and division operations for 32-bit integers, 64-bit integers, floats, and doubles. Note that the throughput for unsigned integers is the same as that for signed integers. As we indicate at the beginning of Section 3.1, the DPU pipeline is capable of performing one integer addition/subtraction operation every cycle, assuming that the pipeline is full [199]. However, real-world workloads do not execute *only* integer addition/subtraction operations. Thus, the theoretical peak arithmetic throughput of 350 MOPS is not realistic for full execution of real workloads. Since the DPUs store operands in WRAM (Section 2.2), a realistic evaluation of arithmetic throughput should consider the accesses to WRAM to read source operands and write destination operands. One access to WRAM involves one WRAM address calculation and one load/store operation.

Listing 1 shows an example microbenchmark for the throughput evaluation of 32-bit integer addition. Listing 1a shows our microbenchmark written in C. The operands are stored in `bufferA`, which we allocate in WRAM using `mem_alloc` [213] (line 2). The `for` loop in line 3 goes through each element of `bufferA` and adds a scalar value `scalar` to each element. In each iteration of the loop, we load one element of `bufferA` into a temporal variable `temp` (line 4), add `scalar` to it (line 5), and store the result back into the same position of `bufferA` (line 6). Listing 1b shows the compiled code, which we can inspect using UPMEM’s Compiler Explorer [236]. The loop contains 6 instructions: WRAM address calculation (`lsl_addr`, line 3), WRAM load (`lw`, line 4), addition (`add`, line 5), WRAM store (`sw`, line 6), loop index update (`add`, line 7), and conditional branch (`jneq`, line 8). For a 32-bit integer subtraction (`sub`), the number of instructions in the streaming loop is also 6, but for other operations and data types the number of instructions can be different (as we show below).

Given the instructions in the loop of the streaming microbenchmark (Listing 1b), we can obtain the expected throughput of arithmetic operations. Only one out of the six instructions is an arithmetic operation (`add` in line 5 in Listing 1b). Assuming that the pipeline is full, the DPU issues (and retires) one instruction every cycle [199]. As a result, we need as many cycles as instructions in the streaming loop to perform one arithmetic operation. If the number of instructions in the loop is n and the DPU frequency is f , we calculate the arithmetic throughput in operations per second (OPS) as expressed in Equation 1.

$$\text{Arithmetic Throughput (in OPS)} = \frac{f}{n} \quad (1)$$

For a 32-bit integer addition (Listing 1), the expected arithmetic throughput on a DPU running at 350 MHz is 58.33 millions of operations per second (MOPS). We verify this on real hardware in Section 3.1.2.

WRAM Bandwidth. To evaluate sustained WRAM bandwidth, we examine the four versions of the STREAM benchmark [237],

```

1 #define SIZE 256
2 int* bufferA = mem_alloc(SIZE*sizeof(int));
3 for(int i = 0; i < SIZE; i++){
4     int temp = bufferA[i];
5     temp += scalar;
6     bufferA[i] = temp;
7 }
8

```

(a) C-based code.

```

1 move r2, 0
2 .LBB0_1: // Loop header
3 lsl_add r3, r0, r2, 2
4 lw r4, r3, 0
5 add r4, r4, r1
6 sw r3, 0, r4
7 add r2, r2, 1
8 jneq r2, 256, .LBB0_1
9

```

(b) Compiled code in UPMEM DPU ISA.

Listing 1: Microbenchmark for throughput evaluation of 32-bit integer addition [214].

which are COPY, ADD, SCALE, and TRIAD, for 64-bit integers. These microbenchmarks access two (COPY, SCALE) or three (ADD, TRIAD) arrays in a streaming manner (i.e., with unit-stride or sequentially). The operations performed by ADD, SCALE, and TRIAD are addition, multiplication, and addition+multiplication, respectively.

In our experiments, we measure the *sustained bandwidth* of WRAM, which is the average bandwidth that we measure over a relatively long period of time (i.e., while streaming through an entire array in WRAM).

We can obtain the maximum theoretical WRAM bandwidth of our STREAM microbenchmarks, which depends on the number of instructions needed to execute the operations in each version of STREAM. Assuming that the DPU pipeline is full, we calculate the maximum theoretical WRAM bandwidth in bytes per second (B/s) with Equation 2, where b is the total number of bytes read and written, n is the number of instructions in a version of STREAM to read, modify, and write the b bytes, and f is the DPU frequency.

$$\text{WRAM Bandwidth (in B/s)} = \frac{b \times f}{n} \quad (2)$$

For example, COPY executes one WRAM load (ld) and one WRAM store (sd) per 64-bit element. These two instructions require 22 cycles to execute for a single tasklet. When the pipeline is full (i.e., with 11 tasklets or more), $11 \times 16 = 176$ bytes are read and written in 22 cycles. As a result, $b = 176$ and $n = 22$, and thus, the maximum theoretical WRAM bandwidth for COPY, at $f=350$ MHz, is 2,800 MB/s. We verify this on real hardware in Section 3.1.3.

3.1.2 Arithmetic Throughput. Figure 4 shows how the measured arithmetic throughput on one DPU (in MOPS) varies with the number of tasklets. We use 1 to 24 tasklets, which is the maximum number of hardware threads.

We make four key observations from Figure 4.

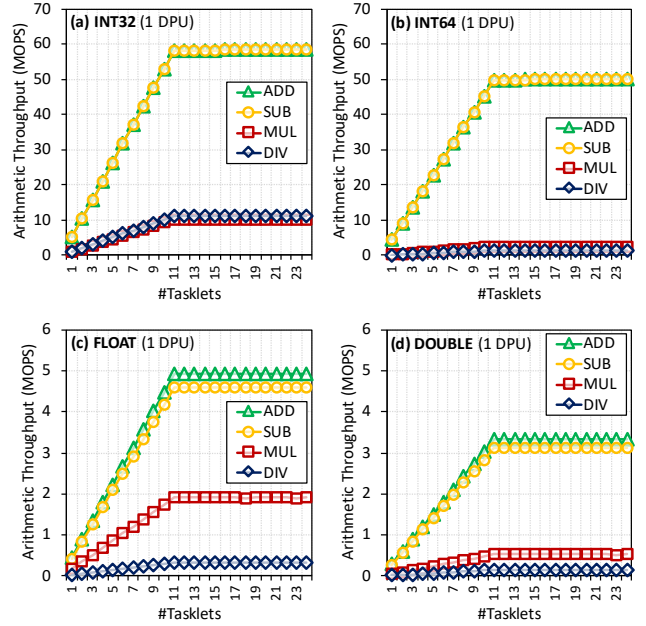


Figure 4: Throughput of arithmetic operations (ADD, SUB, MUL, DIV) on one DPU for four different data types: (a) INT32, (b) INT64, (c) FLOAT, (d) DOUBLE.

First, the throughput of all arithmetic operations and data types saturates after 11 tasklets. This observation is consistent with the description of the pipeline in Section 2.2. Recall that the DPU uses fine-grained multithreading across tasklets to fully utilize its pipeline. Since instructions in the same tasklet are dispatched 11 cycles apart, 11 tasklets is the minimum number of tasklets needed to fully utilize the pipeline.

KEY OBSERVATION 1

The arithmetic throughput of a DRAM Processing Unit saturates at 11 or more tasklets. This observation is consistent for different data types (INT32, INT64, UINT32, UINT64, FLOAT, DOUBLE) and operations (ADD, SUB, MUL, DIV).

Second, the throughput of addition/subtraction is 58.56 MOPS for 32-bit integer values (Figure 4a), and 50.16 MOPS for 64-bit integer values (Figure 4b). The number of instructions inside the streaming loop for 32-bit integer additions/subtractions is 6 (Listing 1). Hence, the expected throughput at 350 MHz is 58.33 MOPS (obtained with Equation 1), which is close to what we measure (58.56 MOPS). A loop with 64-bit integer additions/subtractions contains 7 instructions: the same 6 instructions as the 32-bit version plus an addition/subtraction with carry-in bit (addc/subc) for the upper 32 bits of the 64-bit operands. Hence, the expected throughput at 350 MHz is 50 MOPS which is also close to what we measure (50.16 MOPS).

Third, the throughput of integer multiplication and division is significantly lower than that of integer addition and subtraction (note the large difference in y-axis scale between Figure 4a,b and

Figure 4c,d). A major reason is that the DPU pipeline does not include a complete 32×32 -bit multiplier due to hardware cost concerns and limited number of available metal layers [199]. Multiplications and divisions of 32-bit operands are implemented using two instructions (`mul_step`, `div_step`) [213], which are based on bit shifting and addition. With these instructions, multiplication and division can take up to 32 cycles (32 `mul_step` or `div_step` instructions) to perform, depending on the values of the operands. In case multiplication and division take 32 cycles, the expected throughput (Equation 1) is 10.94 MOPS, which is similar to what we measure (10.27 MOPS for 32-bit multiplication and 11.27 MOPS for 32-bit division, as shown in Figure 4a). For multiplication and division of 64-bit integer operands, programs call two UPMEM runtime library functions (`__muldi3`, `__divdi3`) [213, 238] with 123 and 191 instructions, respectively. The expected throughput for these 64-bit operations is significantly lower than for 32-bit operands, as our measurements confirm (2.56 MOPS for 64-bit multiplication and 1.40 MOPS for 64-bit division, as shown in Figure 4b).

Fourth, the throughput of floating point operations (as shown in Figures 4c and 4d) is more than an order of magnitude lower than that of integer operations. A major reason is that the DPU pipeline does *not* feature native floating point ALUs. The UPMEM runtime library emulates these operations in software [213, 238]. As a result, for each 32-bit or 64-bit floating point operation, the number of instructions executed in the pipeline is between several tens (32-bit floating point addition) and more than 2000 (64-bit floating point division). This explains the low throughput. We measure 4.91/4.59/1.91/0.34 MOPS for FLOAT add/sub/multiply/divide (Figure 4c) and 3.32/3.11/0.53/0.16 MOPS for DOUBLE add/sub/multiply/divide (Figure 4d).

KEY OBSERVATION 2

- **DRAM Processing Units (DPUs) provide native hardware support for 32- and 64-bit integer addition and subtraction**, leading to high throughput for these operations.
- **DPUs do *not* natively support 32- and 64-bit multiplication and division, and floating point operations.** These operations are emulated by the UPMEM runtime library, leading to much lower throughput.

3.1.3 Sustained WRAM Bandwidth. Figure 5 shows how the sustained WRAM bandwidth varies with the number of tasklets (from 1 to 16 tasklets). In these experiments, we unroll the loop of the STREAM microbenchmarks, in order to exclude loop control instructions, and achieve the highest possible sustained WRAM bandwidth. We make three major observations.

First, similar to arithmetic throughput, we observe that WRAM bandwidth saturates after 11 tasklets which is the number of tasklets needed to fully utilize the DPU pipeline.

Second, the maximum measured sustained WRAM bandwidth depends on the number of instructions needed to execute the operation. For COPY, we measure 2,818.98 MB/s, which is similar to the maximum theoretical WRAM bandwidth of 2,800 MB/s, which we obtain with Equation 2 (see Section 3.1.1). ADD executes 5 instructions per 64-bit element: two WRAM loads (1d), one addition (add),

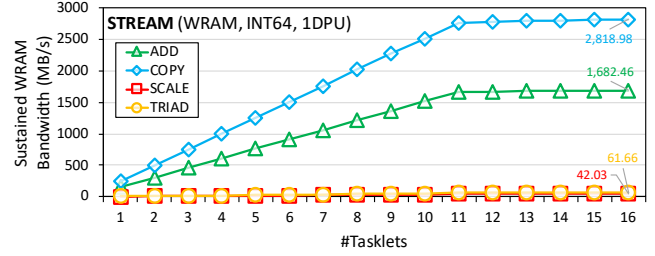


Figure 5: Sustained WRAM bandwidth for streaming access patterns.

one addition with carry-in bit (`addc`), and one WRAM store (`sd`). In this case, $11 \times 24 = 264$ bytes are accessed in 55 cycles when the pipeline is full. Therefore, the maximum theoretical WRAM bandwidth for ADD is 1,680 MB/s, which is similar to what we measure (1,682.46 MB/s). The maximum sustained WRAM bandwidth for SCALE and TRIAD is significantly smaller (42.03 and 61.66 MB/s, respectively), since these microbenchmarks use the costly multiplication operation, which is a library function with 123 instructions (Section 3.1.2).

Third, and importantly (but not shown in Figure 5), **WRAM bandwidth is independent of the access pattern** (streaming, strided, random),¹¹ since all 8-byte WRAM loads and stores take one cycle when the DPU pipeline is full, same as any other native instruction executed in the pipeline [199].

KEY OBSERVATION 3

The sustained bandwidth provided by the DRAM Processing Unit's internal Working memory (WRAM) is independent of the memory access pattern (either streaming, strided, or random access pattern).

All 8-byte WRAM loads and stores take one cycle, when the DRAM Processing Unit's pipeline is full (i.e., with 11 or more tasklets).

3.2 MRAM Bandwidth and Latency

Recall that a DPU, so as to be able to access data from WRAM via load/store instructions, should first transfer the data from its associated MRAM bank to its WRAM via a DMA engine. This section evaluates the bandwidth that can be sustained from MRAM, including read and write bandwidth (Section 3.2.1), streaming access bandwidth (Section 3.2.2), and strided/random access bandwidth (Section 3.2.3).

3.2.1 Read and Write Latency and Bandwidth. In this experiment, we measure the latency of a single DMA transfer of

¹¹We have verified this observation using a microbenchmark (which we also provide as part of our open source release [214]), but do not show the detailed results here for brevity. This microbenchmark uses three arrays in WRAM, *a*, *b*, and *c*. Array *a* is a list of addresses to copy from *b* to *c* (i.e., $c[a[i]] = b[a[i]]$). This list of addresses can be (1) unit-stride (i.e., $a[i] = a[i-1] + 1$), (2) strided (i.e., $a[i] = a[i-1] + \text{stride}$), or (3) random (i.e., $a[i] = \text{rand}()$). For a given number of tasklets and size of the arrays, we measure the *same* execution time for *any* access pattern (i.e., unit-stride, strided, or random), which verifies that WRAM bandwidth is independent of the access pattern.

different sizes for a single tasklet, and compute the corresponding MRAM bandwidth. These DMA transfers are performed via the `mram_read(mram_source, wram_destination, SIZE)` and `mram_write(wram_source, mram_destination, SIZE)` functions, where `SIZE` is the transfer size in bytes and must be a multiple of 8 between 8 and 2,048 according to UPMEM SDK 2021.1.1 [213].

Analytical Modeling. We can analytically model the MRAM access latency (in cycles) using the linear expression in Equation 3, where α is the fixed cost of a DMA transfer, β represents the variable cost (i.e., cost per byte), and $size$ is the transfer size in bytes.

$$\text{MRAM Latency (in cycles)} = \alpha + \beta \times size \quad (3)$$

After modeling the MRAM access latency using Equation 3, we can analytically model the MRAM bandwidth (in B/s) using Equation 3.2.1, where f is the DPU frequency.

$$\begin{aligned} \text{MRAM Bandwidth (in B/s)} &= \frac{size \times f}{\text{MRAM Latency}} = \\ &= \frac{size \times f}{\alpha + \beta \times size} \quad (4) \end{aligned}$$

Measurements. Figure 6 shows how the measured MRAM read and write latency and bandwidth vary with transfer size and how well the measured latency follows the analytical model we develop above.

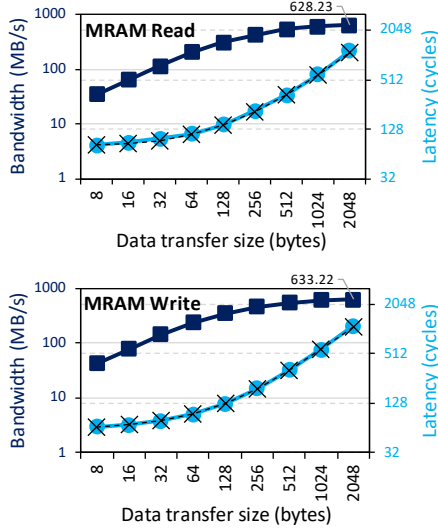


Figure 6: MRAM read and write latency (log scale) and bandwidth (log scale) for data transfer sizes between 8 and 2,048 bytes. The black dashed line represents latency estimates with a linear model (Equation 3).

In our measurements, we find that α is ~ 77 cycles for `mram_read` and ~ 61 cycles for `mram_write`. For both types of transfers, the value β is 0.5 cycles/B. The inverse of β is the maximum theoretical MRAM bandwidth (assuming the fixed cost $\alpha = 0$), which results in 2 B/cycle. The latency values estimated with our analytical model in Equation 3.2.1 (as shown by the black dashed lines in Figure 6) accurately match the latency measurements (light blue lines in Figure 6).

KEY OBSERVATION 4

- The DRAM Processing Unit's Main memory (MRAM) bank access latency increases linearly with the transfer size.
- The maximum theoretical MRAM bandwidth is 2 bytes per cycle.

We make four observations from Figure 6.

First, we observe that read and write accesses to MRAM are symmetric. The latency and bandwidth of read and write transfers are very similar for a given data transfer size.

Second, we observe that the sustained MRAM bandwidth (both read and write) increases with data transfer size. The maximum sustained MRAM bandwidth we measure is 628.23 MB/s for read and 633.22 MB/s for write transfers (both for 2,048-byte transfers). Based on this observation, a general recommendation to maximize MRAM bandwidth utilization is to **use large DMA transfer sizes when all the accessed data is going to be used**. According to Equation 3.2.1, the theoretical maximum MRAM bandwidth is 700 MB/s at a DPU frequency of 350 MHz (assuming no fixed transfer cost, i.e., $\alpha = 0$). Our measurements are within 12% of this theoretical maximum.

PROGRAMMING RECOMMENDATION 1

For data movement between the DRAM Processing Unit's Main memory (MRAM) bank and the internal Working memory (WRAM), **use large DMA transfer sizes when all the accessed data is going to be used**.

Third, we observe that MRAM latency changes slowly between 8-byte and 128-byte transfers. According to Equation 3, the read latency for 128 bytes is 141 cycles and the read latency for 8 bytes is 81 cycles. In other words, latency increases by only 74% while transfer size increases by 16 $\times$. The reason is that, for small data transfer sizes, the fixed cost (α) of the transfer latency dominates the variable cost ($\beta \times size$). For large data transfer sizes, the fixed cost (α) does not dominate the variable cost ($\beta \times size$), and in fact the opposite starts becoming true. We observe that, for read transfers, α (77 cycles) represents 95% of the latency for 8-byte reads and 55% of the latency for 128-byte reads. Based on this observation, one recommendation for programmers is to **fetch more bytes than necessary within a 128-byte limit when using small data transfer sizes**. Doing so increases the probability of finding data in WRAM for later accesses, eliminating future MRAM accesses. **The program can simply check if the desired data has been fetched in a previous MRAM-WRAM transfer, before issuing a new small data transfer.**

PROGRAMMING RECOMMENDATION 2

For small transfers between the DRAM Processing Unit's Main memory (MRAM) bank and the internal Working memory (WRAM), **fetch more bytes than necessary within a 128-byte limit**. Doing so increases the likelihood of finding data in WRAM for later accesses (i.e., the program can check whether the desired data is in WRAM before issuing a new MRAM access).

Fourth, MRAM bandwidth scales almost linearly between 8 and 128 bytes due to the slow MRAM latency increase. After 128 bytes,

MRAM bandwidth begins to saturate. The reason the MRAM bandwidth saturates at large data transfer sizes is related to the inverse relationship of bandwidth and latency (Equation 3.2.1). The fixed cost (α) of the transfer latency becomes negligible with respect to the variable cost ($\beta \times \text{size}$) as the data transfer size increases. For example, α for read transfers (77 cycles) represents only 23%, 13%, and 7% of the MRAM latency for 512-, 1,024-, and 2,048-byte read transfers, respectively. As a result, the MRAM read bandwidth increases by only 13% and 17% for 1,024- and 2,048-byte transfers over 512-byte transfers. Based on this observation, **the recommended data transfer size, when all the accessed data is going to be used, depends on a program's WRAM usage**, since WRAM has a limited size (only 64 KB). For example, if each tasklet of a DPU program needs to allocate 3 temporary WRAM buffers for data from 3 different arrays stored in MRAM, using 2,048-byte data transfers requires that the size of each WRAM buffer is 2,048 bytes. This limits the number of tasklets to 10, which is less than the recommended minimum of 11 tasklets (Sections 2.3.2 and 3.1.2), since $\frac{64KB}{3 \times 2,048} < 11$. In such a case, using 1,024-byte data transfers is preferred, since the bandwidth of 2,048-byte transfers is only 4% higher than that of 1,024-byte transfers, according to our measurements (shown in Figure 6).

PROGRAMMING RECOMMENDATION 3

Choose the data transfer size between the DRAM Processing Unit's Main memory (MRAM) bank and the internal Working memory (WRAM) based on the program's WRAM usage, as it imposes a tradeoff between the sustained MRAM bandwidth and the number of tasklets that can run in the DRAM Processing Unit (which is dictated by the limited WRAM capacity).

3.2.2 Sustained Streaming Access Bandwidth. In this experiment, we use the same four versions of the STREAM benchmark [237] described in Section 3.1.1, but include the MRAM-WRAM DMA transfer time in our measurements. We also add another version of the copy benchmark, COPY-DMA, which copies data from MRAM to WRAM and back without performing any WRAM loads/stores in the DPU core. We use 1024-byte DMA transfers. We scale the number of tasklets from 1 to 16. The tasklets collectively stream 2M 8-byte elements (total of 16 MB), which are divided evenly across the tasklets.

Figure 7 shows how the MRAM streaming access bandwidth varies with the number of tasklets.

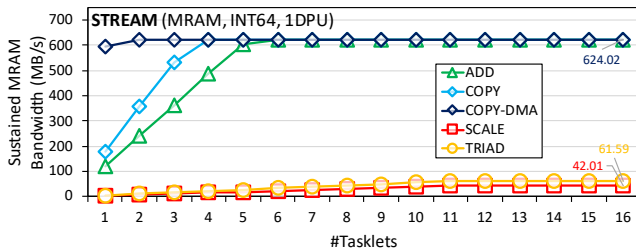


Figure 7: Sustained MRAM bandwidth for streaming access patterns.

We make four key observations.

First, the sustained MRAM bandwidth of COPY-DMA is 624.02 MB/s, which is close to the theoretical maximum bandwidth (700

MB/s derived in Section 2.2). The measured aggregate sustained bandwidth for 2,556 DPUs is 1.6 TB/s. In the 640-DPU system, we measure the sustained MRAM bandwidth to be 470.50 MB/s per DPU (theoretical maximum = 534 MB/s), resulting in aggregate sustained MRAM bandwidth of 301 GB/s for 640 DPUs.

Second, the MRAM bandwidth of COPY-DMA saturates with two tasklets. Even though the DMA engine can perform only one data transfer at a time [231], using two or more tasklets in COPY-DMA guarantees that there is always a DMA request enqueued to keep the DMA engine busy when a previous DMA request completes, thereby achieving the highest MRAM bandwidth.

Third, the MRAM bandwidth for COPY and ADD saturates at 4 and 6 tasklets, respectively, i.e., earlier than the 11 tasklets needed to fully utilize the pipeline. This observation indicates that these microbenchmarks are limited by access to MRAM (and not the instruction pipeline). When the COPY benchmark uses fewer than 4 tasklets, the latency of pipeline instructions (i.e., WRAM loads/stores) is longer than the latency of MRAM accesses (i.e., MRAM-WRAM and WRAM-MRAM DMA transfers). After 4 tasklets, this trend flips, and the latency of MRAM accesses becomes longer. The reason is that the MRAM accesses are serialized, such that the MRAM access latency increases linearly with the number of tasklets. Thus, after 4 tasklets, the overall latency is dominated by the MRAM access latency, which hides the pipeline latency. As a result, the sustained MRAM bandwidth of COPY saturates with 4 tasklets at the highest MRAM bandwidth, same as COPY-DMA. Similar observations apply to the ADD benchmark with 6 tasklets.

Fourth, the sustained MRAM bandwidth of SCALE and TRIAD is approximately one order of magnitude smaller than that of COPY-DMA, COPY, and ADD. In addition, SCALE and TRIAD's MRAM bandwidth saturates at 11 tasklets, i.e., the number of tasklets needed to fully utilize the pipeline. This observation indicates that SCALE and TRIAD performance is limited by pipeline throughput, not MRAM access. Recall that SCALE and TRIAD use costly multiplications, which are based on the mul_step instruction, as explained in Section 3.1.2. As a result, instruction execution in the pipeline has much higher latency than MRAM access. Hence, it makes sense that SCALE and TRIAD are bound by pipeline throughput, and thus the maximum sustained WRAM bandwidth of SCALE and TRIAD (Figure 5) is the same as the maximum sustained MRAM bandwidth (Figure 7).

KEY OBSERVATION 5

- When the access latency to a DRAM Processing Unit's Main memory (MRAM) bank for a streaming benchmark (COPY-DMA, COPY, ADD) is larger than the pipeline latency (i.e., execution latency of arithmetic operations and WRAM accesses), the performance of the DRAM Processing Unit (DPU) saturates at a number of tasklets (i.e., software threads) smaller than 11. This is a memory-bound workload.
- When the pipeline latency for a streaming benchmark (SCALE, TRIAD) is larger than the MRAM access latency, the performance of a DPU saturates at 11 tasklets. This is a compute-bound workload.

3.2.3 Sustained Strided and Random Access Bandwidth. We evaluate the sustained MRAM bandwidth of strided and random access patterns.

To evaluate strided access bandwidth in MRAM, we devise an experiment in which we write a new microbenchmark that accesses MRAM in a strided manner. The microbenchmark accesses an array at a constant stride (i.e., constant distance between consecutive memory accesses), copying elements from the array into another array using the same stride. We implement two versions of the microbenchmark, *coarse-grained DMA* and *fine-grained DMA*, to test both coarse-grained and fine-grained MRAM access. In coarse-grained DMA, the microbenchmark accesses via DMA a large contiguous segment (1024 B) of the array in MRAM, and the strided access happens in WRAM. The coarse-grained DMA approach resembles what modern CPU hardware does (i.e., reads large cache lines from main memory and strides through them in the cache). In fine-grained DMA, the microbenchmark transfers via DMA only the data that will be used by the microbenchmark from MRAM. The fine-grained DMA approach results in more DMA requests, but less total amount of data transferred between MRAM and WRAM.

To evaluate random access bandwidth in MRAM, we implement the GUPS benchmark [239], which performs read-modify-write operations on random positions of an array. We use only fine-grained DMA for random access, since random memory accesses in GUPS do not benefit from fetching large chunks of data, because they are *not* spatially correlated.

In our experiments, we scale the number of tasklets from 1 to 16. The tasklets collectively access arrays in MRAM with (1) coarse-grained strided access, (2) fine-grained strided access, or (3) fine-grained random access. Each array contains 2M 8-byte elements (total of 16MB), which are divided evenly across the tasklets.

Figure 8 shows how the sustained MRAM bandwidth varies with access pattern (strided and random access) as well as with the number of tasklets.

We make four key observations.

First, we measure maximum sustained MRAM bandwidth to be 622.36 MB/s for coarse-grained DMA (with 16 tasklets and a stride of 1, Figure 8a), and 72.58 MB/s for fine-grained DMA (with 16 tasklets, Figure 8b). This difference in the sustained MRAM bandwidth values of coarse-grained DMA and fine-grained DMA is related to the difference in MRAM bandwidth for different transfer sizes (as we analyze in Section 3.2.1). While coarse-grained DMA uses 1,024-byte transfers, fine-grained DMA uses 8-byte transfers.

Second, we observe that the sustained MRAM bandwidth of coarse-grained DMA (Figure 8a) decreases as the stride becomes larger. This is due to the effective utilization of the transferred data, which decreases for larger strides (e.g., a stride of 4 means that only one fourth of the transferred data is effectively used).

Third, the coarse-grained DMA approach has higher sustained MRAM bandwidth for smaller strides while the fine-grained DMA approach has higher sustained MRAM bandwidth for larger strides. The larger the stride in coarse-grained DMA, the larger the amount of fetched data that remains unused, causing fine-grained DMA to become more efficient with larger strides. In these experiments, the coarse-grained DMA approach achieves higher sustained MRAM bandwidth than the fine-grained DMA approach for strides between 1 and 8. For a stride of 16 or larger, the fine-grained DMA approach

achieves higher sustained MRAM bandwidth. This is because with larger strides, the fraction of transferred data that is actually used by the microbenchmark becomes smaller (i.e., effectively-used MRAM bandwidth becomes smaller). With a stride of 16 and coarse-grained DMA, the microbenchmark uses only one sixteenth of the fetched data. As a result, we measure the sustained MRAM bandwidth to be 38.95 MB/s for coarse-grained DMA, which is only one sixteenth of the maximum sustained MRAM bandwidth of 622.36 MB/s, and is lower than the sustained MRAM bandwidth of fine-grained DMA (72.58 MB/s).

Fourth, the maximum sustained MRAM bandwidth for random access is 72.58 MB/s (with 16 tasklets, as shown in Figure 8b). This bandwidth value is very similar to the maximum MRAM bandwidth of the fine-grained DMA approach for strided access (e.g., 72.58 MB/s with 16 tasklets and stride 4,096, as shown in Figure 8b), since our microbenchmark uses fine-grained DMA for random access.

Based on these observations, we recommend that programmers **use the coarse-grained DMA approach for workloads with small strides and the fine-grained DMA approach for workload with large strides or random access patterns.**

PROGRAMMING RECOMMENDATION 4

- For strided access patterns with a **stride smaller than 16 8-byte elements**, **fetch a large contiguous chunk** (e.g., 1,024 bytes) from a DRAM Processing Unit’s Main memory (MRAM) bank.
- For strided access patterns with **larger strides and random access patterns**, **fetch only the data elements that are needed** from an MRAM bank.

3.3 Arithmetic Throughput versus Operational Intensity

Due to its fine-grained multithreaded architecture [205–209], a DPU overlaps instruction execution latency in the pipeline and MRAM access latency [199, 213]. As a result, the overall DPU performance is determined by the dominant latency (either instruction execution latency or MRAM access latency). We observe this behavior in our experimental results in Section 3.2.2, where the dominant latency (pipeline latency or MRAM access latency) determines the sustained MRAM bandwidth for different versions of the STREAM benchmark [237].

To further understand the DPU architecture, we design a new microbenchmark where we vary the number of pipeline instructions with respect to the number of MRAM accesses, and measure performance in terms of arithmetic throughput (in MOPS, as defined in Section 3.1.1). By varying the number of pipeline instructions per MRAM access, we move from microbenchmark configurations where the MRAM access latency dominates (i.e., *memory-bound regions*) to microbenchmark configurations where the pipeline latency dominates (i.e., *compute-bound regions*).

Our microbenchmark includes MRAM-WRAM DMA transfers, WRAM load/store accesses, and a variable number of arithmetic operations. The number of MRAM-WRAM DMA transfers in the microbenchmark is constant, and thus the total MRAM latency is

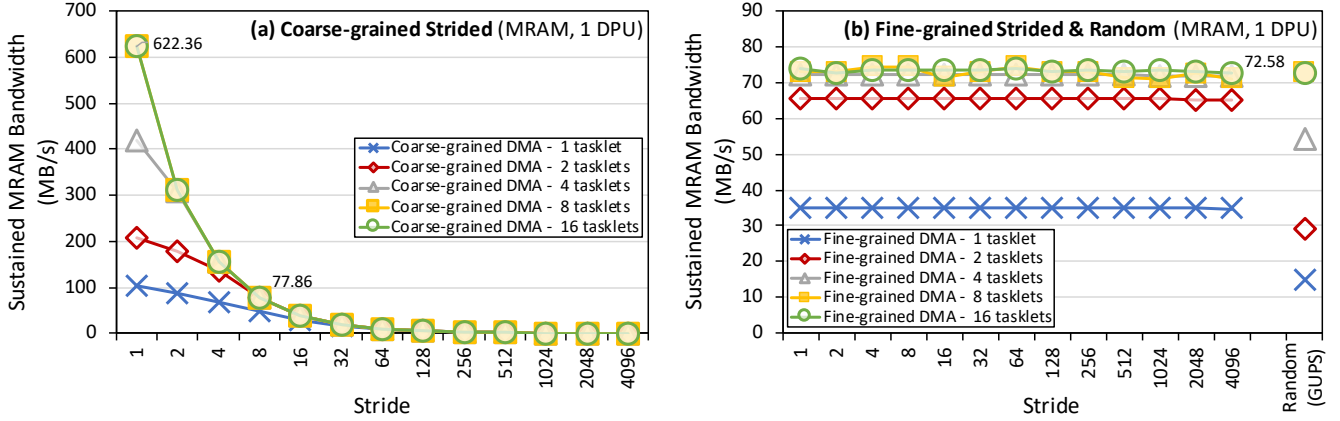


Figure 8: Sustained MRAM bandwidth for (a) coarse-grained strided and (b) fine-grained strided and random access patterns.

also constant. However, the latency of instructions executed in the pipeline varies with the variable number of arithmetic operations.

Our experiments aim to show how arithmetic throughput varies with operational intensity. We define *operational intensity* as the number of arithmetic operations performed per byte accessed from MRAM (OP/B). As explained in Section 3.1.2, an arithmetic operation in the UPMEM PIM architecture takes multiple instructions to execute. The experiment is inspired by the roofline model [215], a performance analysis methodology that shows the performance of a program (arithmetic instructions executed per second) as a function of the *arithmetic intensity* (arithmetic instructions executed per byte accessed from memory) of the program, as compared to the peak performance of the machine (determined by the compute throughput and the L3 and DRAM memory bandwidth).

Figure 9 shows results of arithmetic throughput versus operational intensity for representative data types and operations: (a) 32-bit integer addition, (b) 32-bit integer multiplication, (c) 32-bit floating point addition, and (d) 32-bit floating point multiplication. Results for other data types (64-bit integer and 64-bit floating point) and arithmetic operations (subtraction and division) follow similar trends. We change the operational intensity from very low values ($\frac{1}{2048}$ operations/byte, i.e., one operation per every 512 32-bit elements fetched) to high values (8 operations/byte, i.e., 32 operations per every 32-bit element fetched), and measure the resulting throughput for different numbers of tasklets (from 1 to 16).

We make four key observations from Figure 9.

First, the four plots in Figure 9 show (1) the memory-bound region (where arithmetic throughput increases with operational intensity) and (2) the compute-bound region (where arithmetic throughput is flat at its maximum value) for each number of tasklets. For a given number of tasklets, the transition between the memory-bound region and the compute-bound region occurs when the latency of instruction execution in the pipeline surpasses the MRAM latency. We refer to the operational intensity value where the transition between the memory-bound region and the compute-bound region happens as the *throughput saturation point*.

Second, arithmetic throughput saturates at low (e.g., $\frac{1}{4}$ OP/B for integer addition, i.e., 1 integer addition per every 32-bit element fetched) or very low (e.g., $\frac{1}{128}$ OP/B for floating point multiplication,

i.e., 1 multiplication per every 32 32-bit elements fetched) operational intensity. This result demonstrates that **the DPU is fundamentally a compute-bound processor** designed for workloads with low data reuse.

KEY OBSERVATION 6

The arithmetic throughput of a DRAM Processing Unit (DPU) saturates at low or very low operational intensity (e.g., 1 integer addition per 32-bit element). Thus, **the DPU is fundamentally a compute-bound processor**.

We expect **most real-world workloads be compute-bound in the UPMEM PIM architecture**.

Third, the throughput saturation point is lower for data types and operations that require more instructions per operation. For example, the throughput for 32-bit multiplication (Figure 9b), which requires up to 32 `mul_step` instructions (Section 3.1.2), saturates at $\frac{1}{32}$ OP/B, while the throughput for 32-bit addition (Figure 9a), which is natively supported (it requires a single add instruction), saturates at $\frac{1}{4}$ OP/B. Floating point operations saturate earlier than integer operations, since they require from several tens to hundreds of instructions: 32-bit floating point addition (Figure 9c) and multiplication (Figure 9d) saturate at $\frac{1}{64}$ and $\frac{1}{128}$ OP/B, respectively.

Fourth, we observe that in the compute-bound regions (i.e., after the *saturation points*), arithmetic throughput saturates with 11 tasklets, which is the number of tasklets needed to fully utilize the pipeline. On the other hand, in the memory-bound region, throughput saturates with fewer tasklets because the memory bandwidth limit is reached before the pipeline is fully utilized. For example, at very low operational intensity values ($\leq \frac{1}{64}$ OP/B), throughput of 32-bit integer addition saturates with just two tasklets which is consistent with the observation in Section 3.2.2 where COPY-DMA bandwidth saturates with two tasklets. However, an operational intensity of $\frac{1}{64}$ OP/B is extremely low, as it entails only one addition for every 64 B accessed (16 32-bit integers). We expect higher operational intensity (e.g., greater than $\frac{1}{4}$ OP/B) in most real-world workloads [184, 215] and, thus, arithmetic throughput to saturate with 11 tasklets in real-world workloads.

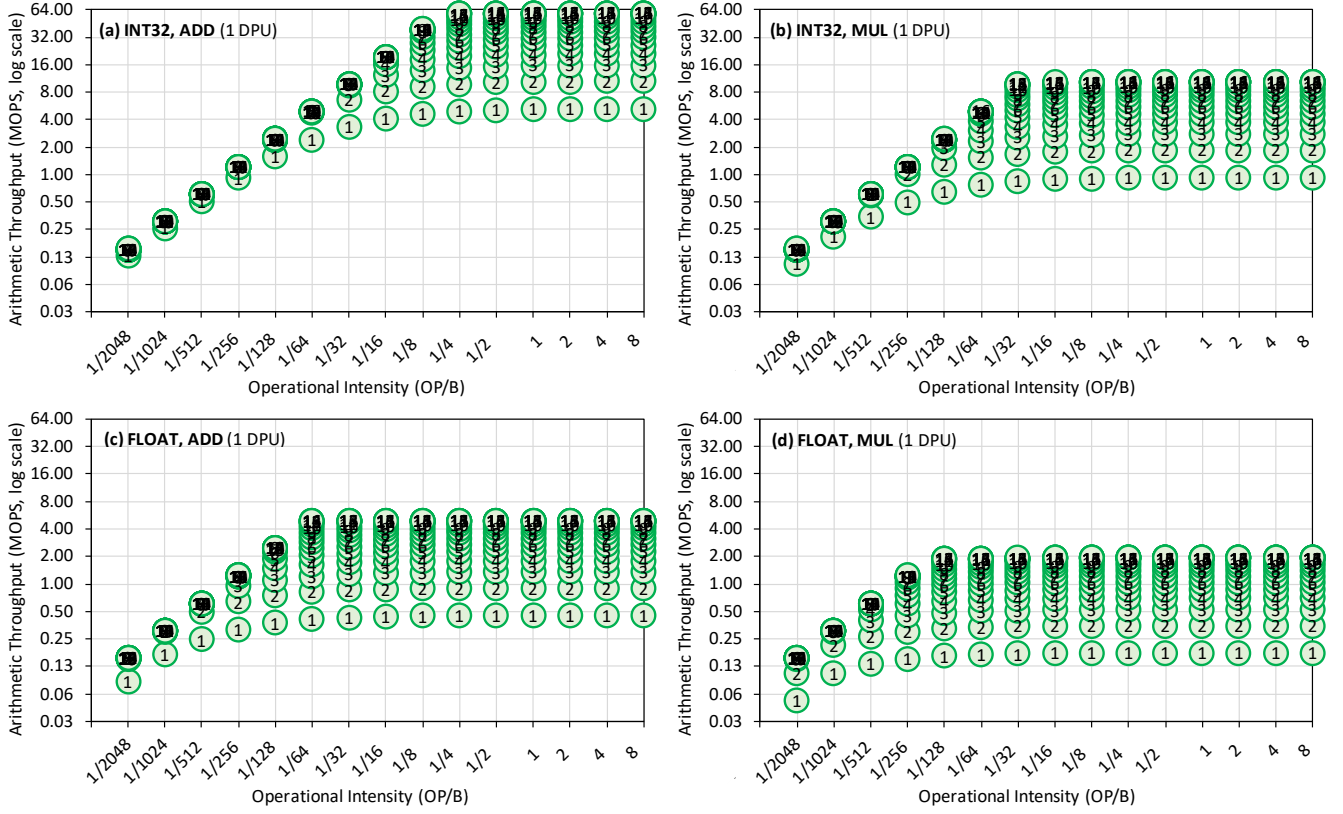


Figure 9: Arithmetic throughput versus operational intensity for (a) 32-bit integer addition, (b) 32-bit integer multiplication, (c) 32-bit floating point addition, and (d) 32-bit floating point multiplication. The number inside each dot indicates the number of tasklets. Both x- and y-axes are log scale.

In the Appendix (Section 9.1), we present a different view of these results, where we show how arithmetic throughput varies with the number of tasklets at different operational intensities.

3.4 CPU-DPU Communication

The host CPU and the DPUs in PIM-enabled memory communicate via the memory bus. The host CPU can access MRAM banks to (1) transfer input data from main memory to MRAM (i.e., CPU-DPU), and (2) transfer results back from MRAM to main memory (i.e., DPU-CPU), as Figure 1 shows. We call these data transfers CPU-DPU and DPU-CPU transfers, respectively. As explained in Section 2.1, these data transfers can be *serial* (i.e., performed sequentially across multiple MRAM banks) or *parallel* (i.e., performed concurrently across multiple MRAM banks). The UPMEM SDK [213] provides functions for serial and parallel transfers. For serial transfers, `dpu_copy_to` copies a buffer from the host main memory to a specific MRAM bank (i.e., CPU-DPU), and `dpu_copy_from` copies a buffer from one MRAM bank to the host main memory (i.e., DPU-CPU). For parallel transfers, a program needs to use two functions. First, `dpu_prepare_xfer` prepares the parallel transfer by assigning different buffers to specific MRAM banks. Second, `dpu_push_xfer` launches the actual transfers to execute in parallel. One argument of `dpu_push_xfer` defines whether the parallel data transfer happens from the host main memory to the MRAM

banks (i.e., CPU-DPU) or from the MRAM banks to the host main memory (i.e., DPU-CPU). Parallel transfers have the limitation (in UPMEM SDK 2021.1.1 [213]) that the transfer sizes to all MRAM banks involved in the same parallel transfer need to be the same. A special case of parallel CPU-DPU transfer (`dpu_broadcast_to`) broadcasts the same buffer from main memory to all MRAM banks.

In this section, we measure the sustained bandwidth of all types of CPU-DPU and DPU-CPU transfers between the host main memory and MRAM banks. We perform two different experiments. The first experiment transfers a buffer of varying size to/from a single MRAM bank. Thus, we obtain the sustained bandwidth of CPU-DPU and DPU-CPU transfers of different sizes for one MRAM bank. In this experiment, we use `dpu_copy_to` and `dpu_copy_from` and vary the transfer size from 8 bytes to 32 MB. The second experiment transfers buffers of size 32 MB per MRAM bank from/to a set of 1 to 64 MRAM banks within the same rank. We experiment with both serial and parallel transfers (`dpu_push_xfer`), including broadcast CPU-DPU transfers (`dpu_broadcast_to`). Thus, we obtain the sustained bandwidth of **serial/parallel/broadcast** CPU-DPU transfers and serial/parallel DPU-CPU transfers for a number of MRAM banks in the same rank between 1 and 64.

Figure 10 presents the sustained bandwidth results of both experiments.

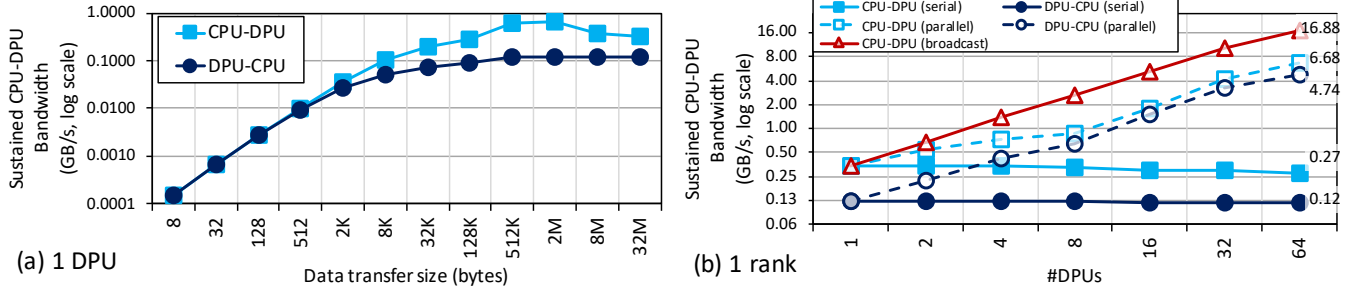


Figure 10: Sustained bandwidth (log scale x- and y-axes) of (a) CPU-DPU (host main memory to one MRAM bank) and DPU-CPU (one MRAM bank to host main memory) transfers of different sizes for one DPU, and (b) serial/parallel/broadcast CPU-DPU (host main memory to several MRAM banks) and serial/parallel DPU-CPU (several MRAM banks to host main memory) transfers of 32 MB for a set of 1-64 DPUs within one rank.

We make seven key observations.¹²

First, sustained bandwidths of CPU-DPU and DPU-CPU transfers for a single DPU (Figure 10a) are similar for transfer sizes between 8 and 512 bytes. For transfer sizes greater than 512 bytes, sustained bandwidth of CPU-DPU transfers is higher than that of DPU-CPU transfers. For the largest transfer size we evaluate (32 MB), CPU-DPU and DPU-CPU bandwidths are 0.33 GB/s and 0.12 GB/s, respectively.

Second, the sustained bandwidths of CPU-DPU and DPU-CPU transfers for a single DPU (Figure 10a) increase linearly between 8 bytes and 2 KB, and tend to saturate for larger transfer sizes.

KEY OBSERVATION 7

Larger CPU-DPU and DPU-CPU transfers between the host main memory and the DRAM Processing Unit's Main memory (MRAM) banks result in higher sustained bandwidth.

Third, for one rank (Figure 10b) the sustained bandwidths of serial CPU-DPU and DPU-CPU transfers remain flat for different numbers of DPUs. Since these transfers are executed serially, latency increases proportionally with the number of DPUs (hence, the total amount of data transferred). As a result, the sustained bandwidth does not increase.

Fourth, the sustained bandwidth of the parallel transfers increases with the number of DPUs, reaching the highest sustained bandwidth values at 64 DPUs. The maximum sustained CPU-DPU bandwidth that we measure is 6.68 GB/s, while the maximum sustained DPU-CPU bandwidth is 4.74 GB/s. However, we observe that the increase in sustained bandwidth with DPU count is sub-linear. The sustained CPU-DPU bandwidth for 64 DPUs is 20.13×

higher than that for one DPU. For DPU-CPU transfers, the sustained bandwidth increase of 64 DPUs to one DPU is 38.76×

KEY OBSERVATION 8

The sustained bandwidth of parallel CPU-DPU and DPU-CPU transfers between the host main memory and the DRAM Processing Unit's Main memory (MRAM) banks increases with the number of DRAM Processing Units inside a rank.

Fifth, we observe large differences between sustained bandwidths of CPU-DPU and DPU-CPU transfers for both serial and parallel transfers. These differences are due to different implementations of CPU-DPU and DPU-CPU transfers in UPMEM SDK 2021.1.1 [231]. While CPU-DPU transfers use x86 AVX write instructions [240], which are asynchronous, DPU-CPU transfers use AVX read instructions [240], which are synchronous. As a result, DPU-CPU transfers cannot sustain as many memory accesses as CPU-DPU transfers, which results in lower sustained bandwidths of both serial and parallel DPU-CPU transfers than the CPU-DPU transfer counterparts.

Sixth, sustained bandwidth of broadcast CPU-DPU transfers reaches up to 16.88 GB/s. One reason why this maximum sustained bandwidth is significantly higher than that of parallel CPU-DPU transfers is better locality in the cache hierarchy of the host CPU [231]. While a broadcast transfer copies the *same* buffer to *all* MRAM banks, which increases temporal locality in the CPU cache hierarchy, a parallel CPU-DPU transfer copies *different* buffers to *different* MRAM banks. These buffers are more likely to miss in the CPU cache hierarchy and need to be fetched from main memory into CPU caches before being copied to MRAM banks.

Seventh, in all our experiments across an entire rank, the sustained bandwidth is lower than the theoretical maximum bandwidth of DDR4-2400 DIMMs (19.2 GB/s) [226]. We attribute this bandwidth loss to the transposition library [199] that the UPMEM SDK uses to map entire 64-bit words onto the same MRAM bank (Section 2.1).

¹²Note that our measurements of and observations about CPU-DPU and DPU-CPU transfers are both platform-dependent (i.e., measurements and observations may change for a different host CPU) and UPMEM SDK-dependent (i.e., the implementation of CPU-DPU/DPU-CPU transfers may change in future releases of the UPMEM SDK). For example, our bandwidth measurements on the 640-DPU system (not shown) differ from those on the 2,556-DPU system (but we find the trends we observe to be similar on both systems).

KEY OBSERVATION 9

The sustained bandwidth of parallel CPU-DPU transfers between the host main memory and the DRAM Processing Unit’s Main memory (MRAM) banks is higher than the sustained bandwidth of parallel DPU-CPU transfers between the MRAM banks and the host main memory due to different implementations of CPU-DPU and DPU-CPU transfers in the UPMEM runtime library.

The sustained bandwidth of broadcast CPU-DPU transfers (i.e., the same buffer is copied to multiple MRAM banks) is higher than that of parallel CPU-DPU transfers (i.e., different buffers are copied to different MRAM banks) due to higher temporal locality in the CPU cache hierarchy.

4 PRIM BENCHMARKS

We present the benchmarks included in our open-source *PrIM* (*Processing-In-Memory*) benchmark suite, the first benchmark suite for a real PIM architecture. PrIM benchmarks are publicly and freely available [214].

For each benchmark, we include in this section a description of its implementation on a UPMEM-based PIM system with multiple DPUs. Table 2 shows a summary of the benchmarks. We group benchmarks by the application domain they belong to. Within each application domain, we sort benchmarks by (1) incremental complexity of the PIM implementation (e.g., we explain VA before GEMV) and (2) alphabetical order. We use the order of the benchmarks in Table 2 consistently throughout the rest of the paper. For each benchmark, the table includes (1) the benchmark’s short name, which we use in the remainder of the paper, (2) memory access patterns of the benchmark (sequential, strided, random), (3) computation pattern (operations and data types), and (4) communication/synchronization type of the PIM implementation (intra-DPU, inter-DPU). For intra-DPU communication, the table specifies the synchronization primitives, such as barriers, handshakes, and mutexes, that the benchmark uses (Section 2.3.1).

All implementations of PrIM benchmarks follow the general programming recommendations presented in Section 2.3.2. Note that our goal is not to provide extremely optimized implementations, but implementations that follow the general programming recommendations and make good use of the resources in PIM-enabled memory with reasonable programmer effort. For several benchmarks, where we can design more than one implementation that is suitable to the UPMEM-based PIM system, we develop all alternative implementations and compare them. As a result, we provide two versions of two of the benchmarks, Image histogram (HST) and Prefix sum (SCAN). In the Appendix (Section 9.2), we compare these versions and find the cases (i.e., dataset characteristics) where each version of each of these benchmarks results in higher performance. We also design and develop three versions of Reduction (RED). However, we do not provide them as separate benchmarks, since one of the three versions always provides higher performance than (or at least equal to) the other two (see Appendix, Section 9.2).¹³

¹³We provide the three versions of RED as part of the same benchmark. Users can select the version they want to test via compiler flags.

Our benchmark selection is based on several criteria: (1) suitability for PIM, (2) domain diversity, and (3) diversity of memory access, computation, and communication/synchronization patterns, as shown in Table 2. We identify the suitability of these workloads for PIM by studying their memory boundedness. We employ the roofline model [215], as described in Section 3.3, to quantify the memory boundedness of the CPU versions of the workloads. Figure 11 shows the roofline model on an Intel Xeon E3-1225 v6 CPU [241] with Intel Advisor [242]. In these experiments, we use the first dataset for each workload in Table 3 (see Section 5).

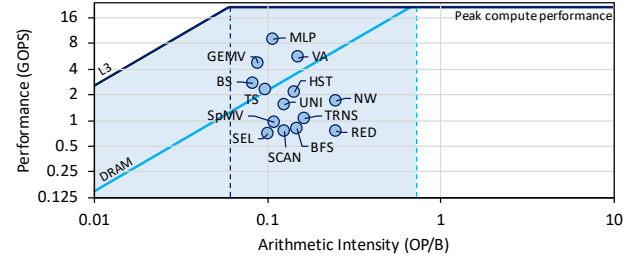


Figure 11: Roofline model for the CPU versions of the 14 PrIM workloads on an Intel Xeon E3-1225 v6 CPU.

We observe from Figure 11 that all of the CPU versions of the PrIM workloads are in the memory-bounded area of the roofline model (i.e., the shaded region on the left side of the intersection between the DRAM bandwidth line and the peak compute performance line). Hence, these workloads are all limited by memory. We conclude that all 14 CPU versions of PrIM workloads are potentially suitable for PIM [184]. We briefly describe each PrIM benchmark and its PIM implementation next.

4.1 Vector Addition

Vector Addition (VA) [243] takes two vectors a and b as inputs and performs their element-wise addition.

Our PIM implementation divides the input vectors a and b into as many equally-sized chunks as the number of DPUs in the system, and makes a linear assignment (i.e., chunk i assigned to DPU i). The host CPU loads one chunk of both vectors a and b to the MRAM bank of each DPU. Inside each DPU, we assign blocks of elements from a and b to tasklets in a cyclic manner (i.e., block j assigned to tasklet $j\%T$ for a total number T of tasklets per DPU). Each tasklet (1) moves the blocks of elements from a and b to the WRAM, (2) performs the element-wise addition, and (3) moves the results to the MRAM bank. Tasklets iterate as many times as needed until the whole chunk assigned to a DPU is processed. At the end of the execution on the DPUs, the CPU retrieves the output vector chunks from the MRAM banks to the host main memory and constructs the complete output vector.

4.2 Matrix-Vector Multiply

Matrix-Vector multiply (GEMV) [243] is a dense linear algebra routine that takes a matrix of size $m \times n$ and a vector of size $n \times 1$ as inputs and performs the multiplication between them, producing a new $m \times 1$ vector as a result.

Table 2: PRIM benchmarks.

Domain	Benchmark	Short name	Memory access pattern			Computation pattern		Communication/synchronization	
			Sequential	Strided	Random	Operations	Datatype	Intra-DPU	Inter-DPU
Dense linear algebra	Vector Addition	VA	Yes			add	int32_t		
	Matrix-Vector Multiply	GEMV	Yes			add, mul	uint32_t		
Sparse linear algebra	Sparse Matrix-Vector Multiply	SpMV	Yes		Yes	add, mul	float		
Databases	Select	SEL	Yes			add, compare	int64_t	handshake, barrier	Yes
	Unique	UNI	Yes			add, compare	int64_t	handshake, barrier	Yes
Data analytics	Binary Search	BS	Yes		Yes	compare	int64_t		
	Time Series Analysis	TS	Yes			add, sub, mul, div	int32_t		
Graph processing	Breadth-First Search	BFS	Yes		Yes	bitwise logic	uint64_t	barrier, mutex	Yes
Neural networks	Multilayer Perceptron	MLP	Yes			add, mul, compare	int32_t		
Bioinformatics	Needleman-Wunsch	NW	Yes	Yes		add, sub, compare	int32_t	barrier	Yes
Image processing	Image histogram (short)	HST-S	Yes		Yes	add	uint32_t	barrier	Yes
	Image histogram (long)	HST-L	Yes		Yes	add	uint32_t	barrier, mutex	Yes
Parallel primitives	Reduction	RED	Yes	Yes		add	int64_t	barrier	Yes
	Prefix sum (scan-scan-add)	SCAN-SSA	Yes			add	int64_t	handshake, barrier	Yes
	Prefix sum (reduce-scan-scan)	SCAN-RSS	Yes			add	int64_t	handshake, barrier	Yes
	Matrix transposition	TRNS	Yes		Yes	add, sub, mul	int64_t	mutex	

Our PIM implementation of GEMV partitions the matrix across the DPUs available in the system, assigning a fixed number of consecutive rows to each DPU, while the input vector is replicated across all DPUs. The host CPU assigns each set of consecutive rows to a DPU using linear assignment (i.e., set of rows i assigned to DPU i). Inside each DPU, tasklets are in charge of computing on the set of the rows assigned to that DPU. We assign a subset of consecutive rows from the set of rows assigned to a DPU to each tasklet (i.e., subset of rows j assigned to tasklet j). First, each tasklet reads a block of elements, both from one row of the input matrix and from the vector, and places these elements in the WRAM. Second, each tasklet performs multiply and accumulation of those elements, and it jumps to the first step until it reaches the end of the row. Third, each tasklet stores the sums of multiplications in MRAM. Fourth, each tasklet repeats these three steps as many times as there are rows in its subset. Fifth, each DPU produces a **contiguous** chunk of elements of the output vector. The CPU retrieves the output vector chunks and builds the complete output vector.

4.3 Sparse Matrix-Vector Multiply

Sparse Matrix-Vector multiply (SpMV) [244] is a sparse linear algebra routine where a sparse matrix is multiplied by a dense vector.

Our PIM implementation of SpMV uses the Compressed Sparse Row (CSR) storage format [245–247] to represent the matrix. First, the host CPU distributes the rows of the matrix evenly across DPUs, using linear assignment (i.e., set of rows i assigned to DPU i) as in GEMV (Section 4.2). Within each DPU, the rows of the matrix are distributed evenly across tasklets (i.e., subset of rows j assigned to tasklet j , same as in GEMV). The input vector is replicated across DPUs. Each tasklet multiplies its subset of rows with the input vector and produces a contiguous chunk of the output vector. At the end of the execution on the DPUs, the CPU copies back the output vector chunks from the MRAM banks to the host main memory, in order to construct the entire output vector.

4.4 Select

Select (SEL) [248] is a database operator that, given an input array, filters the array elements according to a given input predicate. Our version of SEL removes the elements that satisfy the predicate, and keeps the elements that do not.

Our PIM implementation of SEL partitions the array across the DPUs available in the system. The tasklets inside a DPU coordinate using handshakes (Section 2.3.1). First, each tasklet moves a block of elements to WRAM. Second, each tasklet filters the elements and counts the number of filtered elements. Third, each tasklet passes its number of filtered elements to the next tasklet using handshake-based communication, which inherently performs a prefix-sum operation [249–251] to determine where in MRAM to store the filtered elements. The tasklet then moves its filtered elements to MRAM. Fourth, the host CPU performs the final merge of the filtered arrays returned by each DPU via serial DPU-CPU transfers, since parallel DPU-CPU transfers are not feasible because each DPU may return a different number of filtered elements.

4.5 Unique

Unique (UNI) [248] is a database operator that, for each group of consecutive array elements with the same value, removes all but the first of these elements.

Our PIM implementation of UNI follows a similar approach to SEL. The main difference lies in the more complex handshake-based communication that UNI needs. Besides the number of unique elements, each tasklet has to pass the value of its last unique element to the next tasklet. This way, the next tasklet can check whether its first element is unique or not in the context of the entire array.

4.6 Binary Search

Binary Search (BS) [252] takes a sorted array as input and finds the position of some query values within the sorted array.

Our PIM implementation of binary search distributes the sorted array across the DPUs. Inside each DPU, tasklets are in charge of a subpartition of the assigned query values. First, each tasklet checks the assigned set of query values to find, moving them from the MRAM bank to WRAM and iterating over them using a for loop. Second, each tasklet performs the binary search algorithm, moving from left to right or vice-versa, depending on the current value to find. Third, the tasklet stops the algorithm when it finds one query value. Fourth, at the end of the execution on the DPUs, the host CPU retrieves the positions of the found query values.

4.7 Time Series Analysis

Time Series analysis (TS) [253] aims to find anomalies and similarities between subsequences of a given time series. Our version of time series analysis is based on Matrix Profile [254], an algorithm that works in a streaming-like manner, where subsequences (or query sequences) coming from a source of data are compared to a well-known time series that has the expected behavior.

Our PIM implementation of time series analysis divides the time series across the DPUs, adding the necessary overlapping between them, and replicating the query sequence across the tasklets to compare to the time series. Different slices of the time series are assigned to different tasklets. First, each tasklet performs the dot product of its slice of the time series and the query sequence. Second, each tasklet calculates the similarity between the slice of the time series and the query sequence by computing the z-normalized Euclidean distance [254]. Third, each tasklet compares the calculated similarity to the minimum similarity (or maximum, depending on the application) found so far, and updates it if the calculated similarity is a new minimum (or maximum). Fourth, at the end of the execution on the DPUs, the host CPU retrieves the minimum (or maximum) similarity values and their positions from all DPUs, and finds the overall minimum (or maximum) and its position.

4.8 Breadth-First Search

Breadth-First Search (BFS) [255] is a graph algorithm that labels each node in the graph with its distance from a given source node. In our version, all edges have the same weight, therefore the distance represents the number of edges.

Our PIM implementation of BFS uses a Compressed Sparse Row (CSR) [245–247] representation of the *adjacency matrix*, which represents the graph. Each element (i, j) of the adjacency matrix indicates whether vertices i and j are connected by an edge. Vertices are distributed evenly across DPUs, with each DPU receiving the *neighbor lists* for the vertices that it owns. The neighbor list of vertex i contains the vertex IDs of the vertices that are connected to vertex i by an edge. Each DPU maintains its own local copy of the list of visited vertices in the graph, which is represented as a bit-vector. At the end of each iteration of the BFS algorithm, the host CPU merges all local per-DPU copies of the list of visited vertices. The whole list of visited vertices is called the *frontier*.

At the beginning of each iteration, the host CPU broadcasts the complete current frontier to all the DPUs. Each DPU uses the current frontier to update its local copy of the visited list. The DPU keeps the vertices of the current frontier that correspond to the vertices that it owns and discards the rest. The tasklets in the DPU (1) go through these vertices concurrently, (2) visit their neighbors, and (3) add the neighbors to the next frontier if they have not previously been visited. This approach to BFS is called *top-down* approach [256, 257]. Tasklets use critical sections (implemented via mutexes) to update the next frontier concurrently without data conflicts. At the end of each iteration, the CPU retrieves the next frontier produced by each DPU, and computes their union to construct the complete next frontier. The iterations continue until the next frontier is empty at the end of an iteration.

4.9 Multilayer Perceptron

Multilayer perceptron (MLP) [258] is a class of feedforward artificial neural network with at least three layers: input, hidden and output.

Our PIM implementation of MLP performs MLP inference. In each layer, the weights are arranged as a matrix and the input is a vector. The computation in each layer is a matrix-vector multiplication. The implementation of each layer is based on our implementation of GEMV (Section 4.2). Thus, in each layer of MLP, the distribution of the workload among DPUs and tasklets is the same as in GEMV. ReLU is the activation function at the end of each layer. When a layer terminates, the host CPU (1) retrieves the output vector chunks from the MRAM banks, (2) constructs the complete vector, (3) feeds this vector to the DPUs as the input of the next layer, and (4) transfers the weights matrix of the next layer to the DPUs. At the end of the output layer, the host CPU retrieves the output vector chunks, and constructs the final output vector.

4.10 Needleman-Wunsch

Needleman-Wunsch (NW) [259] is a bioinformatics algorithm that performs global sequence alignment, i.e., it compares two biological sequences over their *entire length* to find out the optimal alignment of these sequences. NW is a dynamic programming algorithm that consists of three steps: (i) initialize a 2D score matrix $m \times n$, where m, n are the lengths of the sequences (i.e., the number of *base pairs*, *bps*, in each sequence); (ii) fill the score matrix by calculating the score for each cell in the matrix, which is the maximum of the scores of the neighboring cells (left, top, or top-left cells) plus a penalty in case of a mismatch; and (iii) trace back the optimal alignment by marking a path from the cell on the bottom right back to the cell on the top left of the score matrix. Note that there may be more than one possible optimal alignments between two sequences.

Our PIM implementation first fills the upper triangle (top-left part) of the 2D score matrix, and then the lower triangle (bottom-right part) of it. The matrix is partitioned into large 2D blocks, and the algorithm iterates over the diagonals at a large block granularity (from the top-left diagonal to the bottom-right diagonal). In each iteration, all large blocks that belong to the same diagonal of the 2D score matrix are calculated in parallel by evenly distributing them across DPUs. Inside the DPU, each large 2D block is further partitioned into small 2D sub-blocks. The tasklets of each DPU work on the diagonals at a small sub-block granularity, i.e., in each iteration the tasklets of a DPU concurrently calculate different small sub-blocks that belong to the same large block of one diagonal.

For each diagonal of the 2D score matrix, the host CPU retrieves the large blocks produced by all DPUs. Then, it uses the filled cells of the last row and the last column of each large block as input to the next iteration (i.e., the next diagonal), since only these cells are neighboring cells with the next diagonal blocks. The iterations continue until all diagonal large blocks of the whole 2D score matrix are calculated. The host CPU finally uses the resulting 2D score matrix to trace back the optimal alignment.

In the Appendix (Section 9.2.1), we show additional experimental results for NW to demonstrate that the computation of the complete problem and the computation of the longest diagonal scale differently across one rank of DPUs.

4.11 Image Histogram

Image histogram (HST) [260] calculates the histogram of an image, i.e., it counts the number of occurrences of each possible pixel value in an input image and stores the aggregated counts of occurrences into a set of bins.

We develop two PIM implementations of image histogram: short (HST-S) and long (HST-L).

HST-S distributes chunks of the input image across tasklets running on a DPU. Each tasklet creates a local histogram in WRAM. When the local histograms are created, the tasklets synchronize with a barrier, and the local histograms are merged in a parallel manner. Since each tasklet features a local histogram in WRAM, the maximum histogram size is relatively small (e.g., 256 32-bit bins, when running 16 tasklets).¹⁴

HST-L can generate larger histograms, the size of which is limited only by the total amount of WRAM, since only one local histogram per DPU is stored in WRAM. Same as HST-S, HST-L distributes chunks of the input image across tasklets, which update the histogram in WRAM by using a mutex, in order to ensure that only a single tasklet updates the histogram at a time.

Both HST-S and HST-L merge all per-DPU histograms into a single final histogram in the host CPU.

We compare HST-S and HST-L for different histogram sizes in the Appendix (Section 9.2.2), in order to find out which HST version is preferred on the UPMEM-based PIM system for each histogram size.

4.12 Reduction

Reduction (RED) [261] computes the sum of the elements in an input array.

Our PIM implementation of reduction has two steps. In the first step, each tasklet inside a DPU is assigned a chunk of the array. The tasklet accumulates all values of the chunk and produces a local reduction result. In the second step, after a barrier, a single tasklet reduces the partial results of all tasklets from the first step. At the end of the second step, the host CPU retrieves the reduction result.

Alternatively, we can implement the second step as a parallel tree reduction [262, 263]. We implement two versions of this parallel tree reduction, which use different intra-DPU synchronization primitives. One of the versions uses handshakes for communication between tasklets from one level of the tree to the next one. The other version uses barriers between levels of the tree. In the Appendix (Section 9.2.3), we compare the single-tasklet implementation to the two versions of parallel tree reduction.

4.13 Prefix Sum (Scan)

Prefix sum or scan (SCAN) [249] is a parallel primitive that computes the prefix sum of the values in an array. We implement an exclusive scan: the i -th element of the output contains the sum of all elements of the input array from the first element to the $(i-1)$ -th element.

We implement two PIM versions of scan: Scan-Scan-Add (SCAN-SSA) [251, 264, 265] and Reduce-Scan-Scan (SCAN-RSS) [251, 264,

266]. Both versions assign a large chunk of the input array to each DPU.

SCAN-SSA has three steps. First, it computes the scan operation locally inside each DPU. Second, it copies the last element of the local scan to the host CPU, and places it in a vector in the position corresponding to the DPU order. The host CPU scans this vector and moves each result value to the corresponding DPU. Third, it adds the value computed in the host CPU to all elements of the local scan output in each DPU. Fourth, the host CPU retrieves the complete scanned array from the MRAM banks.

SCAN-RSS also has three steps. First, it computes the reduction operation in each DPU. Second, it copies the reduction results to the host CPU, where the host CPU scans them. Third, it moves the result values of the scan operation in the host CPU to the corresponding DPUs, where the tasklets perform a local scan (including the value coming from the host CPU). Fourth, the host CPU retrieves the complete scanned array from the MRAM banks.

The advantage of SCAN-RSS over SCAN-SSA is that SCAN-RSS requires fewer accesses to MRAM. For an array of size N , SCAN-RSS needs $3 \times N + 1$ accesses: N reads and 1 write for Reduce, and N reads and N writes for Scan. SCAN-SSA needs $4 \times N$ accesses: N reads and N writes for Scan, and N reads and N writes for Add. The advantage of SCAN-SSA over SCAN-RSS is that it requires less synchronization. The reduction operation in SCAN-RSS requires a barrier, but the addition operation in SCAN-SSA does not require any synchronization. We expect SCAN-RSS to perform better for large arrays where access to MRAM dominates the execution time, and SCAN-SSA to perform better for smaller arrays where the reduction that requires synchronization constitutes a larger fraction of the entire computation. We compare both implementations of SCAN for arrays of different sizes in Appendix Section 9.2.4.

4.14 Matrix Transposition

Matrix transposition (TRNS) [267] converts an $M \times N$ array into an $N \times M$ array. We focus on in-place transposition, where the transposed array occupies the same physical storage locations as the original array. In-place transposition is a permutation, which can be factored into disjoint cycles [268]. A straightforward parallel implementation can assign entire cycles to threads. However, in such a straightforward implementation, (1) the length of cycles is *not* uniform in rectangular matrices, causing load imbalance, and (2) the memory accesses are random as operations are done on single matrix elements (without exploiting spatial locality). Thus, efficient parallelization is challenging.

Our PIM implementation follows an efficient 3-step tiled approach [269, 270] that (1) exploits spatial locality by operating on tiles of matrix elements, as opposed to single elements, and (2) balances the workload by partitioning the cycles across tasklets. To perform the three steps, we first factorize the dimensions of the $M \times N$ array as an $M' \times m \times N' \times n$ array, where $M = M' \times m$ and $N = N' \times n$.

The first step operates on tiles of size n . This step performs the transposition of an $M \times N'$ array, where each element is a tile of size n . The resulting array has dimensions $N' \times M \times n = N' \times M' \times m \times n$. In the UPMEM-based PIM system, we perform this step using n -sized CPU-DPU transfers that copy the input array from the main memory of the host CPU to the corresponding MRAM banks.

¹⁴ 256 32-bit bins is the maximum histogram size for 16 tasklets (1) assuming power-of-two size of the histogram and (2) taking into account that each tasklet allocates a WRAM buffer for its chunk of the input image.

The second step performs $N' \times M'$ transpositions of $m \times n$ tiles. In each DPU, one tasklet transposes an $m \times n$ tile in WRAM. The resulting array has dimensions $N' \times M' \times n \times m$.

The third step operates on tiles of size m . This step performs transpositions of N' arrays of dimensions $M' \times n$, where each element is a tile of size m . The resulting array has dimensions $N' \times n \times M' \times m$. In each DPU, the available tasklets collaborate on the transposition of an $M' \times n$ array (with m -sized elements) using the algorithm presented in [271]. Differently from the algorithm in [271], which uses atomic instructions for synchronization [272], our PIM implementation uses mutexes for synchronization of tasklets via an array of flags that keeps track of the moved tiles (we choose this implementation because the UPMEM ISA [213] does *not* include atomic instructions).

After the three steps, the host CPU retrieves the transposed matrix from the MRAM banks.

5 EVALUATION OF PRIM BENCHMARKS

In this section, we evaluate the 16 PrIM benchmarks on the 2,556-DPU system (Section 2.1), unless otherwise stated. Our evaluation uses the datasets presented in Table 3, which are publicly and freely available [214]. Since these datasets are large and do not fit in WRAM, we need to use MRAM-WRAM DMA transfers repeatedly. The results we present are for the best performing transfer sizes, which we include in Table 3 to facilitate the reproducibility of our results. We provide the command lines we use to execute each benchmark along with all parameters in [214].

First, we present performance and scaling results. We evaluate strong scaling³ for the 16 PrIM benchmarks (Section 5.1.1) on the 2,556-DPU system by running the experiments on (1) 1 DPU, (2) 1 rank (from 1 to 64 DPUs), and (3) 32 ranks (from 256 to 2,048 DPUs). The goal of these experiments is to evaluate how the performance of the UPMEM-based PIM system scales with the number of DPUs for a fixed problem size. The ideal strong scaling is linear scaling, i.e., the ideal speedup for strong scaling with N DPUs over the execution on a single DPU should be N .

We also evaluate weak scaling⁴ for the 16 PrIM benchmarks (Section 5.1.2) on 1 rank (from 1 to 64 DPUs). In this experiment, we evaluate how the performance of the UPMEM-based PIM system scales with the number of DPUs for a fixed problem size per DPU. In an ideal weak scaling scenario, the execution time remains constant for any number of DPUs.

Second, we compare the performance and energy consumption of two full-blown UPMEM-based PIM systems (Table 1) with 2,556 DPUs (newer system) and with 640 DPUs (older system) to those of a modern Intel Xeon E3-1240 CPU [241] and a modern NVIDIA Titan V GPU [277] (Section 5.2).

In Section 6, we provide new insights about suitability of different workloads to the PIM system, programming recommendations for software designers, and suggestions and hints for hardware and architecture designers of future PIM systems.

5.1 Performance and Scaling Results

We evaluate the performance of all the benchmarks with strong and weak scaling experiments using the datasets in Table 3. Section 5.1.1 presents strong scaling results for a single DPU, a single rank (from 1 to 64 DPUs), and for sets of 4 to 32 ranks (from 256 to 2,048 DPUs). We also evaluate the cost of inter-DPU synchronization.

In Section 5.1.2, we analyze weak scaling on an entire rank for 1 to 64 DPUs. We include in the analysis the cost of inter-DPU synchronization via the host CPU, as well as CPU-DPU and DPU-CPU latencies.

5.1.1 Strong Scaling Results. We evaluate strong scaling with three different configurations: (1) 1-16 tasklets inside one DPU, (2) 1-64 DPUs inside one rank, and (3) 4-32 ranks. For the experiments inside one rank and multiple ranks, we use the best-performing number of tasklets from the experiment on one DPU.

One DPU. Figure 12 presents execution time and speedup scaling (versus tasklet count) results for 16 benchmarks on a single DPU. The speedup results (right y-axis of each plot) correspond to only the execution time portion spent on the DPU (i.e., "DPU" portion of each bar in Figure 12). In these experiments, we set the number of tasklets to 1, 2, 4, 8, and 16. The benchmarks distribute computation among the available tasklets in a data-parallel manner. The datasets and their sizes are in Table 3. The times shown in Figure 12 are broken down into the execution time on the DPU ("DPU"), the time for inter-DPU communication via the host CPU ("Inter-DPU"), the time for CPU to DPU transfer of input data ("CPU-DPU"), and the time for DPU to CPU transfer of final results ("DPU-CPU").

We make the following five observations from Figure 12.

First, in VA, GEMV, SpMV, SEL, UNI, TS, MLP, NW, HST-S, RED, SCAN-SSA (Scan kernel), SCAN-RSS (both kernels), and TRNS (Step 2 kernel), the best performing number of tasklets is 16. This is in line with our observations in Section 3.1.2: a number of tasklets greater than 11 is usually a good choice to achieve the best performance from the pipeline. These benchmarks show good scaling from 1 to 8 tasklets with speedups between 1.5 $\times$ and 2.0 $\times$ as we double the number of tasklets until 8. From 8 to 16 tasklets, the speedups are between 1.2 $\times$ and 1.5 $\times$ due to the pipeline throughput saturating at 11 tasklets. For BS and BFS, 16 tasklets provide the highest performance too. However, scaling in BS and BFS is more limited than in the kernels listed in the beginning of this paragraph, as we discuss later in this section.

KEY OBSERVATION 10

A number of tasklets greater than 11 is a good choice for most real-world workloads we tested (16 kernels out of 19 kernels from 16 benchmarks), as it fully utilizes the DRAM Processing Unit's pipeline.

Second, some of these benchmarks (VA, GEMV, SpMV, BS, TS, MLP, HST-S, TRNS (Step 2)) do *not* use synchronization primitives, while in others (SEL, UNI, NW, RED, SCAN-SSA (Scan kernel), SCAN-RSS (both kernels)), synchronization across tasklets (via handshakes and/or barriers) is lightweight.

Third, BFS, HST-L, and TRNS (Step 3) show limited scaling when increasing the number of tasklets because they use mutexes, which cause contention when accessing shared data structures (i.e., output frontier in BFS, local per-DPU histogram in HST-L, array of flags in TRNS (Step 3)). While in BFS using 16 tasklets provides the highest performance since it can compensate for the large synchronization cost, in HST-L and TRNS (Step 3) the best performing number of

Table 3: Evaluated Datasets.

Benchmark	Strong Scaling Dataset	Weak Scaling Dataset	MRAM-WRAM Transfer Sizes
VA	1 DPU-1 rank: 2.5M elem. (10 MB) 32 ranks: 160M elem. (640 MB)	2.5M elem./DPU (10 MB)	1024 bytes
GEMV	1 DPU-1 rank: 8192 × 1024 elem. (32 MB) 32 ranks: 163840 × 4096 elem. (2.56 GB)	1024 × 2048 elem./DPU (8 MB)	1024 bytes
SpMV	<i>bcsstk30</i> [273] (12 MB)	<i>bcsstk30</i> [273]	64 bytes
SEL	1 DPU-1 rank: 3.8M elem. (30 MB) 32 ranks: 240M elem. (1.9 GB)	3.8M elem./DPU (30 MB)	1024 bytes
UNI	1 DPU-1 rank: 3.8M elem. (30 MB) 32 ranks: 240M elem. (1.9 GB)	3.8M elem./DPU (30 MB)	1024 bytes
BS	2M elem. (16 MB). 1 DPU-1 rank: 256K queries. (2 MB) 32 ranks: 16M queries. (128 MB)	2M elem. (16 MB). 256K queries./DPU (2 MB).	8 bytes
TS	256 elem. query. 1 DPU-1 rank: 512K elem. (2 MB) 32 ranks: 32M elem. (128 MB)	512K elem./DPU (2 MB)	256 bytes
BFS	<i>loc-gowalla</i> [274] (22 MB)	<i>rMat</i> [275] ($\approx 100K$ vertices and 1.2M edges per DPU)	8 bytes
MLP	3 fully-connected layers. 1 DPU-1 rank: 2K neurons (32 MB) 32 ranks: $\approx 160K$ neur. (2.56 GB)	3 fully-connected layers. 1K neur./DPU (4 MB)	1024 bytes
NW	1 DPU-1 rank: 2560 bps (50 MB), large/small sub-block = $\frac{2560}{\#DPUs}/2$ 32 ranks: 64K bps (32 GB), L/s. = 32/2	512 bps/DPU (2MB), L/s. = 512/2	8, 16, 32, 40 bytes
HST-S	1 DPU-1 rank: 1536 × 1024 input image [276] (6 MB) 32 ranks: 64 × input image	1536 × 1024 input image [276]/DPU (6 MB)	1024 bytes
HST-L	1 DPU-1 rank: 1536 × 1024 input image [276] (6 MB) 32 ranks: 64 × input image	1536 × 1024 input image [276]/DPU (6 MB)	1024 bytes
RED	1 DPU-1 rank: 6.3M elem. (50 MB) 32 ranks: 400M elem. (3.1 GB)	6.3M elem./DPU (50 MB)	1024 bytes
SCAN-SSA	1 DPU-1 rank: 3.8M elem. (30 MB) 32 ranks: 240M elem. (1.9 GB)	3.8M elem./DPU (30 MB)	1024 bytes
SCAN-RSS	1 DPU-1 rank: 3.8M elem. (30 MB) 32 ranks: 240M elem. (1.9 GB)	3.8M elem./DPU (30 MB)	1024 bytes
TRNS	1 DPU-1 rank: 12288 × 16 × 64 × 8 (768 MB) 32 ranks: 12288 × 16 × 2048 × 8 (24 GB)	12288 × 16 × 1 × 8/DPU (12 MB)	128, 1024 bytes

tasklets is 8 due to the high synchronization overheads beyond 8 tasklets.

KEY OBSERVATION 11

Intensive use of **intra-DPU synchronization across tasklets (e.g., mutexes, barriers, handshakes)** may **limit scalability**, sometimes causing the best performing number of tasklets to be lower than 11.

Fourth, SCAN-SSA (Add kernel) experiences speedups between 1.5× and 2.0× when we double the number of tasklets until 8. However, there is no speedup from 8 to 16 tasklets, even though this step of the SCAN-SSA benchmark does *not* use any synchronization primitives. We observe the same behavior for the STREAM ADD microbenchmark in Figure 7, i.e., performance saturation happens before the 11 tasklets required to fully utilize the pipeline. As explained in Section 3.2.2, the reason is that both STREAM ADD and SCAN-SSA (Add kernel) are *not* compute-intensive kernels, since they perform only one integer addition per input element accessed from MRAM. As a result, the overall latency is dominated by the MRAM access latency, which hides the pipeline latency (and thus performance saturates at fewer than 11 tasklets required to fully utilize the pipeline). The same reason explains that BS obtains almost no speedup (only 3%) from 8 to 16 tasklets, since BS performs only one comparison per input element.

KEY OBSERVATION 12

Most real-world workloads are in the compute-bound region of the DRAM Processing Unit (all kernels except SCAN-SSA (Add kernel)), i.e., the pipeline latency dominates the MRAM access latency.

Fifth, while the amount of time spent on CPU-DPU transfers and DPU-CPU transfers is relatively low compared to the time spent on DPU execution for most benchmarks, we observe that CPU-DPU transfer time is very high in TRNS. The CPU-DPU transfer of TRNS performs step 1 of the matrix transposition algorithm [269, 270]

by issuing $M' \times m$ data transfers of n elements, as explained in Section 4.14. Since we use a small n value in the experiment ($n = 8$, as indicated in Table 3), the sustained CPU-DPU bandwidth is far from the maximum CPU-DPU bandwidth (see sustained CPU-DPU bandwidth for different transfer sizes in Figure 10a).

KEY OBSERVATION 13

Transferring large data chunks from/to the host CPU is preferred for input data and output results due to higher sustained CPU-DPU/DPU-CPU bandwidths.

One Rank (1-64 DPUs). We evaluate strong scaling with 1 to 64 DPUs. The size of the input is the dataset size we can fit in a single DPU (see Table 3). We especially examine CPU-DPU transfer and DPU-CPU transfer times, in order to assess how they change as we increase the number of DPUs. Figure 13 shows execution time and speedup scaling (versus DPU count) results for 1, 4, 16, and 64 DPUs. The speedup results (right y-axis of each plot) correspond to only the execution time portion spent on the DPU (i.e., the "DPU" portion of each bar in Figure 13). The breakdown of execution time is the same as that done in Figure 12 for the single-DPU results.

We make the following seven observations from Figure 13.

First, we observe that DPU performance scaling is linear with DPU count (i.e., the execution times on the DPU reduce linearly as we increase the number of DPUs) for VA, GEMV, SpMV, SEL, UNI, BS, TS, MLP, HST-S, HST-L, RED, SCAN-SSA (both kernels), SCAN-RSS (both kernels), and TRNS (both kernels) (speedups between 3.1× and 4.0× when increasing the number of DPUs by 4). As a result, increasing the DPU count from 1 to 64 for these benchmarks produces speedups between 37× (SpMV) and 64× (TS, TRNS).

Second, scaling of DPU performance is sublinear for two benchmarks (BFS, NW). Increasing the DPU count from 1 to 64 for these two benchmarks produces speedups between 8.3× (BFS) and 17.2× (NW). For BFS, the speedups are sublinear (1.7 – 2.7× when increasing the number of DPUs by 4) due to load imbalance across DPUs produced by the irregular topology of the *loc-gowalla* graph [274]. In NW, the speedups are between 2.2× and 3.3× when multiplying

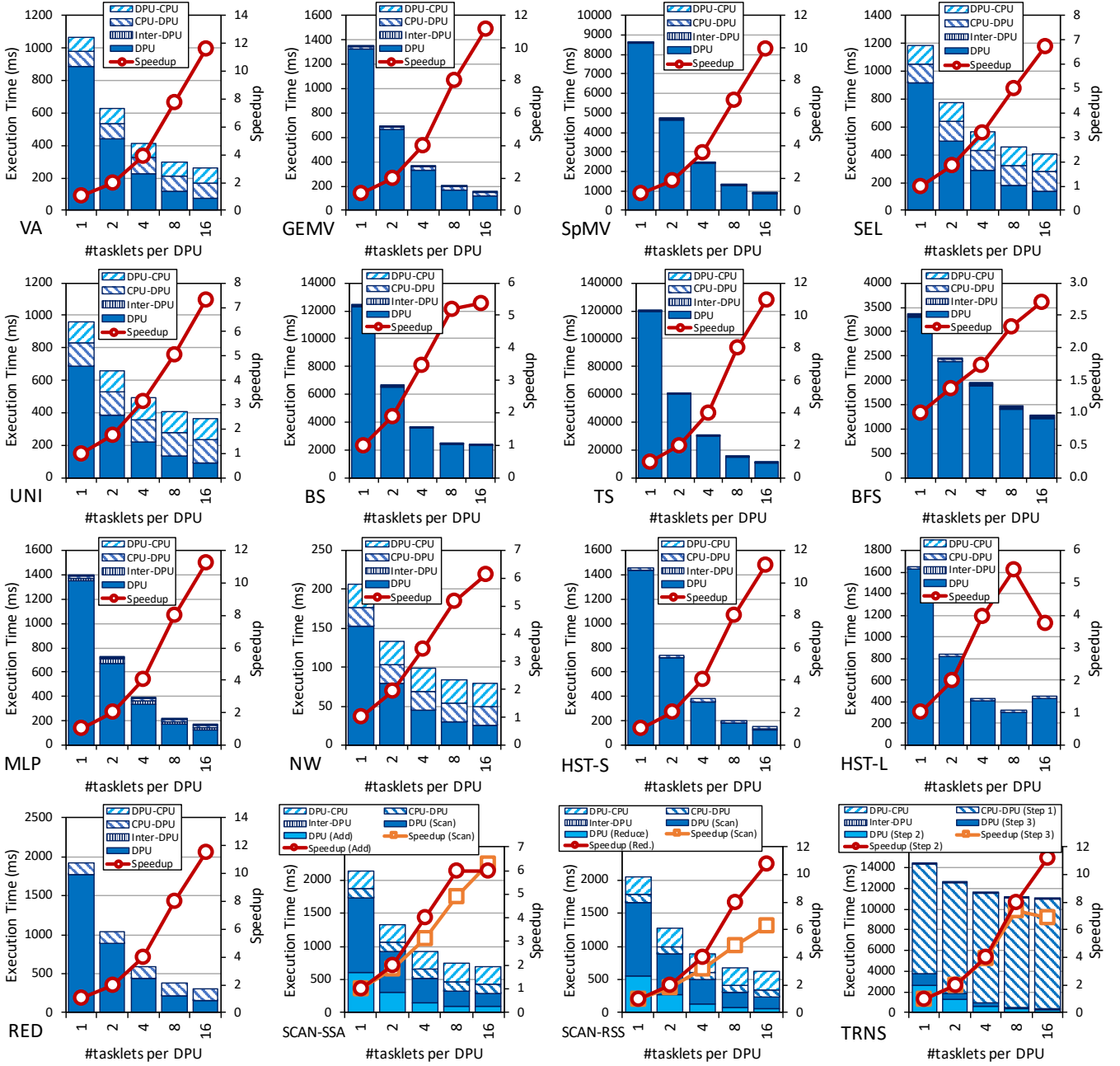


Figure 12: Execution time (ms) of 16 benchmarks on 1, 2, 4, 8, and 16 tasklets in one DPU (left y-axis), and speedup (considering only the portion of execution time spent on the DPU) provided by more tasklets normalized to the performance of 1 tasklet (right y-axis).

the DPU count by 4. In this benchmark, the parallelization factor in each iteration (i.e., number of active DPUs) depends on the size of the diagonal of the 2D score matrix that is processed, and the number of large 2D blocks in the diagonal. When we increase the number of DPUs by 4, the parallelization factor in smaller diagonals is low (i.e., equal to only the number of blocks in these diagonals), and only increases up to 4 $\times$ in the larger diagonals (i.e., when

there are enough blocks to use all available DPUs). As a result, the scalability of NW is sublinear.

Third, the overhead (if any) of inter-DPU synchronization (as depicted by the "Inter-DPU" portion of each bar in Figure 13) is low in 14 of the benchmarks (VA, GEMV, SpMV, SEL, UNI, BS, TS, HST-S, HST-L, RED, SCAN-SSA, SCAN-RSS, TRNS). As a result, these benchmarks achieve higher performance when we increase the number of DPUs (even including the inter-DPU synchronization

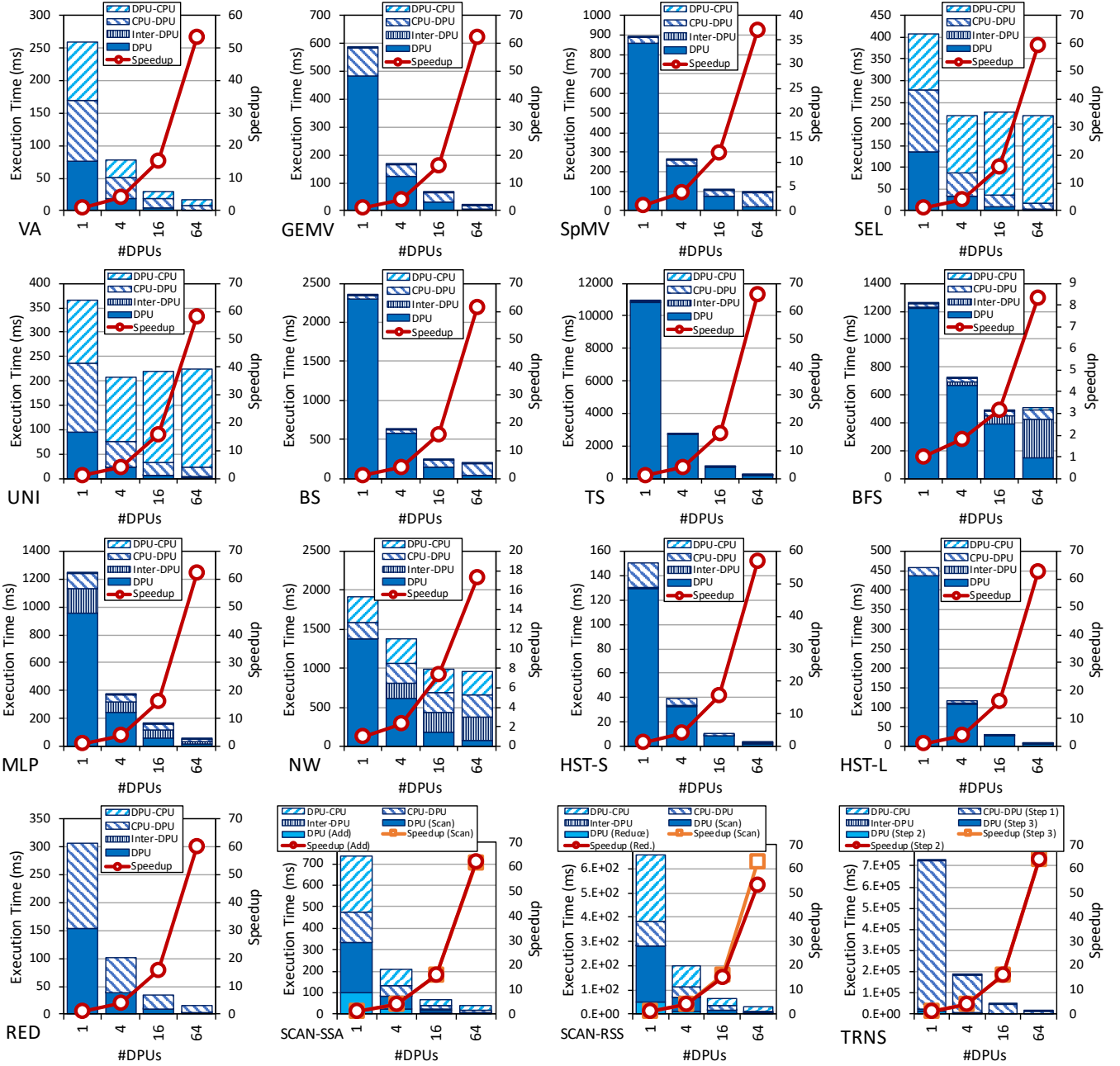


Figure 13: Execution time (ms) of 16 benchmarks on one rank (1, 4, 16, and 64 DPUs, using strong scaling³) (left y-axis), and speedup (considering only the portion of execution time spent on the DPU) provided by more DPUs normalized to the performance of 1 DPU (right y-axis). Inside a DPU, we use the best-performing number of tasklets from Figure 12.

time). There is no inter-DPU synchronization in VA, GEMV, SpMV, BS, TS, and TRNS. There is inter-DPU synchronization in HST-S and HST-L (for final histogram reduction), but its overhead is negligible. The inter-DPU synchronization time is noticeable in SEL, UNI, and RED (for final result merging) and in SCAN-SSA and SCAN-RSS (for intermediate scan step in the host). For 64 DPUs, the inter-DPU synchronization times of SEL, UNI, RED, SCAN-SSA, and

SCAN-RSS account for 53%, 91%, 48%, 42%, and 17% the execution times on the DPUs (not visible in Figure 13), respectively. Despite that, we still obtain the best performance (including portions of the execution time spent on the DPUs, i.e., "DPU", and inter-DPU synchronization, i.e., "Inter-DPU") with 64 DPUs for SEL, UNI, RED, SCAN-SSA, and SCAN-RSS.

Fourth, we observe significantly higher overhead of inter-DPU synchronization for BFS, MLP, and NW. In MLP, the inter-DPU synchronization overhead (due to the distribution of weights matrix and input vector to each layer) reduces as the number of DPUs increases. The reason is that the distribution of the weights matrix (i.e., copying assigned matrix rows to the corresponding DPUs) takes advantage of parallel CPU-DPU transfers, while the overhead of transferring the input vector is negligible. However, the trend is different for BFS and NW. The overall performance (including portions of the execution time on the DPUs, i.e., "DPU", and inter-DPU synchronization, i.e., "Inter-DPU") of 64 DPUs is only 5% and 17% higher than that of 16 DPUs for BFS and NW, respectively. The reason in BFS is that, after each iteration, the CPU has to compute the union of the next frontier from all DPUs sequentially and redistribute it across the DPUs. Thus, the inter-DPU synchronization cost increases linearly with the number of DPUs. In NW, the inter-DPU synchronization overhead is substantial due to a similar reason. For each diagonal of the 2D score matrix, the host CPU (1) retrieves the results of the sub-blocks produced by all DPUs, and (2) sends the cells of the last row and the last column of each sub-block as input to the next diagonal (processed in the next iteration).

Fifth, we observe the CPU-DPU transfer and DPU-CPU transfer times decrease for VA, GEMV, TS, MLP, HST-S, HST-L, RED, SCAN-SSA, SCAN-RSS, and TRNS, when we increase the number of DPUs in the strong scaling experiment for 1 rank. These benchmarks use parallel CPU-DPU and DPU-CPU transfers between the main memory of the host CPU and the MRAM banks.

Sixth, the CPU-DPU and DPU-CPU transfer times do not decrease for BS and NW with increasing number of DPUs, even though BS and NW use parallel transfers. BS distributes the values to search in a sorted array across the available DPUs, but the sorted array is replicated in each DPU. As a result, the total CPU-DPU time increases with the number of DPUs. NW processes a diagonal in each iteration. For shorter diagonals, the algorithm does not need to use all available DPUs. Thus, more available DPUs does not always mean more parallelism in CPU-DPU and DPU-CPU transfers.

Seventh, the remaining benchmarks (SpMV, SEL, UNI, BFS) cannot use parallel transfers to copy input data and/or retrieve results. In SEL and UNI, DPU-CPU transfer times increase when we increase the number of DPUs because we cannot use parallel transfers for retrieving results. In these two benchmarks, the size of the output in each DPU may differ as it depends on the element values of the input array. SpMV and BFS cannot use parallel CPU-DPU and DPU-CPU transfers because the size of the inputs assigned to each DPU may be different (e.g., different number of nonzero elements of different sparse rows in SpMV, different numbers of edges for different vertices in BFS). As a result, we observe that CPU-DPU and DPU-CPU transfer times do not reduce in SpMV and BFS when increasing the number of DPUs.

PROGRAMMING RECOMMENDATION 5

Parallel CPU-DPU/DPU-CPU transfers inside a rank of UPMEM DRAM Processing Units are recommended for real-world workloads when all transferred buffers are of the same size.

32 Ranks (256-2,048 DPUs). We evaluate strong scaling with 4, 8, 16, and 32 ranks. The size of the input is the maximum dataset size we can fit in four ranks (i.e., 256 DPUs), as shown in Table 3. We do not include CPU-DPU and DPU-CPU transfer times in our performance analysis, because these transfers are *not* simultaneous across ranks, as we mentioned in Section 3.4. Figure 14 shows execution time and speedup scaling (versus DPU count) results for 256, 512, 1,024, and 2,048 DPUs, corresponding to 4, 8, 16, and 32 ranks. The speedup results (right y-axis of each plot) correspond to only the execution time portion spent on the DPU (i.e., the "DPU" portion of each bar in Figure 14).

We make the following observations from Figure 14.

First, as in the experiments on one rank, we observe that the execution times on the DPU (i.e., the "DPU" portion of each bar in Figure 14) reduce linearly with the number of DPUs (i.e., $\sim 2\times$ when we double the number of DPUs, and $\sim 8\times$ from 256 to 2,048 DPUs) for VA, GEMV, SEL, UNI, BS, TS, MLP, HST-S, HST-L, RED, SCAN-SSA (both kernels), SCAN-RSS (both kernels), and TRNS (both kernels). For SCAN-SSA (Scan) and SCAN-RSS (Scan), we observe more than $8\times$ speedup when we scale from 256 to 2,048 DPUs. The reason is that the amount of synchronization across tasklets (i.e., handshakes in Scan) reduces when we distribute the input array across more DPUs. However, the downside is that the inter-DPU communication cost increases, as we explain below.

Second, DPU performance scaling (i.e., the "DPU" portion of each bar in Figure 14) is sublinear for SpMV, BFS, and NW. For SpMV and BFS, there is load imbalance across DPUs due to the irregular nature of graphs and sparse matrices. For NW, we observe small speedups when we double the number of DPUs ($1.60\times$ from 256 to 512 DPUs, and $1.25\times$ from 512 to 1,024 DPUs), and almost no speedup (only 8%) from 1,024 to 2,048 DPUs. As explained above, NW does not use all DPUs in all iterations, but only the number that is needed for the diagonal that is processed in each iteration. As a result, doubling the number of DPUs does not reduce the execution time spent on the DPU at the same rate.

KEY OBSERVATION 14

Load balancing across DRAM Processing Units (DPUs) ensures linear reduction of the execution time spent on the DPUs for a given problem size, when all available DPUs are used (as observed in strong scaling experiments).

Third, inter-DPU synchronization (as depicted by the "Inter-DPU" portion of each bar in Figure 14) imposes a small overhead (if any) for 12 of the benchmarks (VA, GEMV, SpMV, SEL, UNI, BS, TS, HST-S, HST-L, RED, and TRNS). VA, GEMV, SpMV, BS, TS, and TRNS do not require inter-DPU synchronization. For SEL, UNI, HST-S, HST-L, and RED, the inter-DPU synchronization involves only DPU-CPU transfers, since it is only used to merge final results at the end of execution. The inter-DPU synchronization overhead increases with the number of DPUs, since the amount of partial results to merge increases. However, the inter-DPU synchronization cost is small, and a larger number of DPUs results in larger overall performance.

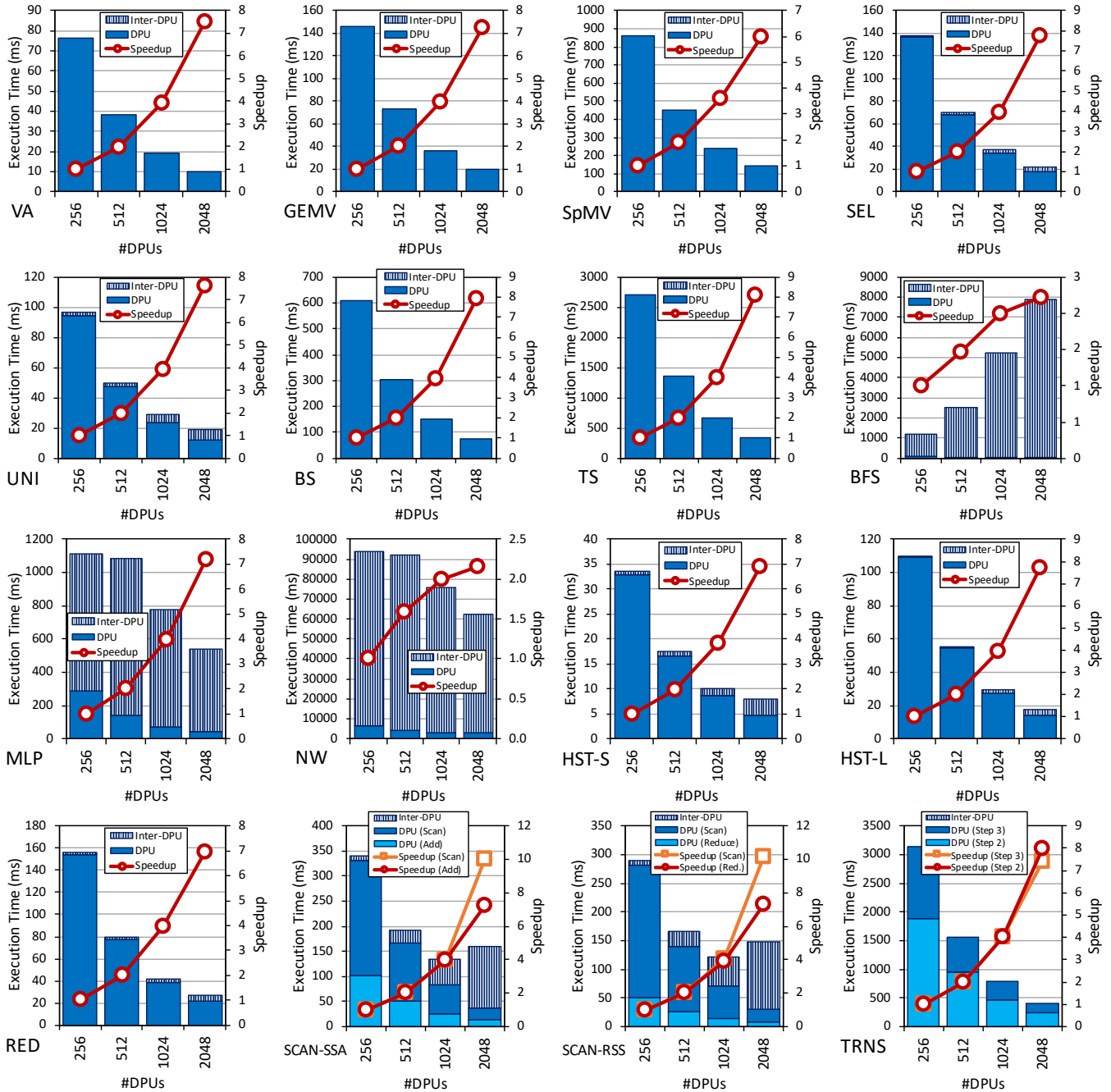


Figure 14: Execution time (ms) of 16 benchmarks on 4, 8, 16, and 32 ranks (256, 512, 1,024, and 2,048 DPUs, using strong scaling³) (left y-axis), and speedup (considering only the portion of execution time spent on the DPU) provided by more DPUs normalized to the performance of 4 ranks (256 DPUs) (right y-axis). Inside a DPU, we use the best-performing number of tasklets from Figure 12.

KEY OBSERVATION 15

The overhead of merging partial results from DRAM Processing Units in the host CPU is tolerable across all PRIM benchmarks that need it.

Fourth, the inter-DPU synchronization imposes significant overhead when it requires more complex patterns (involving both CPU-DPU and DPU-CPU transfers). We observe this for five benchmarks (BFS, MLP, NW, SCAN-SSA, and SCAN-RSS). For NW and MLP, we observe that inter-DPU synchronization times are significantly

higher than DPU times. If we compare these results to the results in Figure 13, we conclude that these benchmarks' overall performance is greatly burdened by inter-DPU synchronization when using more than one rank. SCAN-SSA and SCAN-RSS perform a more complex intermediate step in the CPU: (1) the CPU gathers partial results from the first kernel (Scan in SCAN-SSA, Reduce in SCAN-RSS) from the DPUs (via DPU-CPU transfers), (2) the CPU performs a scan operation, and (3) the CPU returns a value to be used by the second kernel (Add in SCAN-SSA, Scan in SCAN-RSS) to each DPU (via CPU-DPU transfers). The significant increase in "Inter-DPU" from 1,024 to 2,048 DPUs is due to the dual-socket system configuration (Section 2.1), since the CPU in one socket obtains lower memory bandwidth from remote MRAM banks (i.e., in the other socket). For BFS, the trend is even worse. We observe that the huge increase in the inter-DPU synchronization time makes 256 DPUs the best choice for executing BFS. Our observations for BFS, SCAN-SSA, and SCAN-RSS are *against* the general programming recommendation of using as many working DPUs as possible (Section 2.3.2). These three benchmarks show that the best-performing number of DPUs is limited by the inter-DPU synchronization overhead.

KEY OBSERVATION 16

Complex synchronization across DRAM Processing Units (i.e., inter-DPU synchronization involving two-way communication with the host CPU) imposes significant overhead, which limits scalability to more DPUs. This is more noticeable when DPUs involved in the synchronization span multiple ranks.

5.1.2 Weak Scaling Results. Figure 15 shows the weak scaling results inside a single rank for 1, 4, 16, and 64 DPUs. In each DPU, we run the number of tasklets that produces the best performance in Section 5.1.1. The size of the dataset per DPU is the size shown in Table 3. The time is broken down into execution time on the DPU ("DPU"), inter-DPU synchronization time ("Inter-DPU"), and CPU-DPU and DPU-CPU transfer times ("CPU-DPU", "DPU-CPU"), similarly to the strong scaling results presented in Figures 12 to 14 in Section 5.1.1.

We make the following five observations from Figure 15.

First, we observe perfect (i.e., linear) weak scaling on the DPU for 14 benchmarks: the execution time on the DPU (i.e., the "DPU" portion of each bar in Figure 15) remains constant for VA, GEMV, SpMV, SEL, UNI, BS, TS, MLP, HST-S, HST-L, RED, SCAN-SSA, SCAN-RSS, and TRNS, as we increase the number of DPUs (and the dataset size). Since there is no direct communication between DPUs in these kernels, the even distribution of workload (i.e., load balance) among DPUs leads to performance scaling. We observe a similar trend of perfect weak scaling for BFS even though there is some load imbalance across DPUs in BFS.

KEY OBSERVATION 17

Equally-sized problems assigned to different DRAM Processing Units (DPUs) and little/no inter-DPU synchronization lead to linear weak scaling of the execution time spent on the DPUs (i.e., constant execution time when we increase the number of DPUs and the dataset size accordingly).

Second, NW does *not* scale linearly (i.e., the execution time spent on the DPU is *not* constant) because the size of the problem does *not* increase linearly with the number of DPUs. We increase the lengths of the input sequences to NW *linearly* with the number of DPUs (see Table 3, weak scaling dataset). Thus, the size of the 2D score matrix increases *quadratically* with the number of DPUs. As a result, the execution times on the DPUs increase when we increase the number of DPUs. However, the longest diagonal of the 2D score matrix increases linearly with the number of DPUs. The processing time of this diagonal shows linear weak scaling as we increase the number of DPUs. We show these experimental results in the Appendix (Section 9.2.1).

Third, among the benchmarks that require inter-DPU synchronization (SEL, UNI, BFS, MLP, NW, HST-S, HST-L, RED, SCAN-SSA, and SCAN-RSS, the synchronization overhead (i.e., the "Inter-DPU" portion of each bar in Figure 15) is negligible for SEL, UNI, HST-S, HST-L, RED, SCAN-SSA, and SCAN-RSS. For MLP and NW, the inter-DPU synchronization time takes a significant fraction of the overall execution time, and it increases with the number of DPUs because the total problem size (and thus, the size of weight matrices in MLP and the number of iterations in NW) increases, as indicated above. In BFS, the inter-DPU synchronization time increases linearly, as we explain in Section 5.1.1 (Figures 13 and 14) for strong scaling experiments. As a result, BFS obtains the best tradeoff between overall execution time (including portions of the execution time spent on the DPUs, i.e., "DPU", and inter-DPU synchronization, i.e., "Inter-DPU") and number of DPUs at 16 DPUs (i.e., the ratio of overall execution time, the "DPU" portions + the "Inter-DPU" portions, over number of DPUs is lower for 16 DPUs).

Fourth, CPU-DPU and DPU-CPU transfer times increase slowly with the number of DPUs for the 13 benchmarks that use parallel transfers between main memory and MRAM banks (VA, GEMV, SEL (only CPU-DPU), UNI (only CPU-DPU), BS, TS, MLP, HST-S, HST-L, RED, SCAN-SSA, SCAN-RSS, and TRNS). As observed from Figure 10, the sustained CPU-DPU and DPU-CPU bandwidths increase sublinearly with the number of DPUs. On average, the increase in sustained CPU-DPU/DPU-CPU bandwidth for these 13 benchmarks from 1 DPU to 64 DPUs is $20.95\times/23.16\times$. NW uses parallel CPU-DPU and DPU-CPU transfers, but the CPU-DPU transfer and DPU-CPU transfer times increase with the number of DPUs because the amount of transferred data increases (i.e., the total problem size increases, as described above in the second observation from Figure 15).

Fifth, CPU-DPU transfer and DPU-CPU transfer times increase linearly with the number of DPUs for the benchmarks that cannot use parallel transfers. SEL and UNI employ serial DPU-CPU transfers, as we discuss above. This makes the DPU-CPU transfer times in these two benchmarks increase dramatically with the number of DPUs, dominating the entire execution time. In SpMV and BFS, where we cannot use parallel transfers due to the irregular nature of datasets, CPU-DPU transfer and DPU-CPU transfer times also increase significantly. In full-blown real-world applications, where SEL, UNI, SpMV, or BFS may be just one of the multiple or many kernels executed by the application, the CPU-DPU transfer and DPU-CPU transfer times can be amortized and their overhead alleviated.

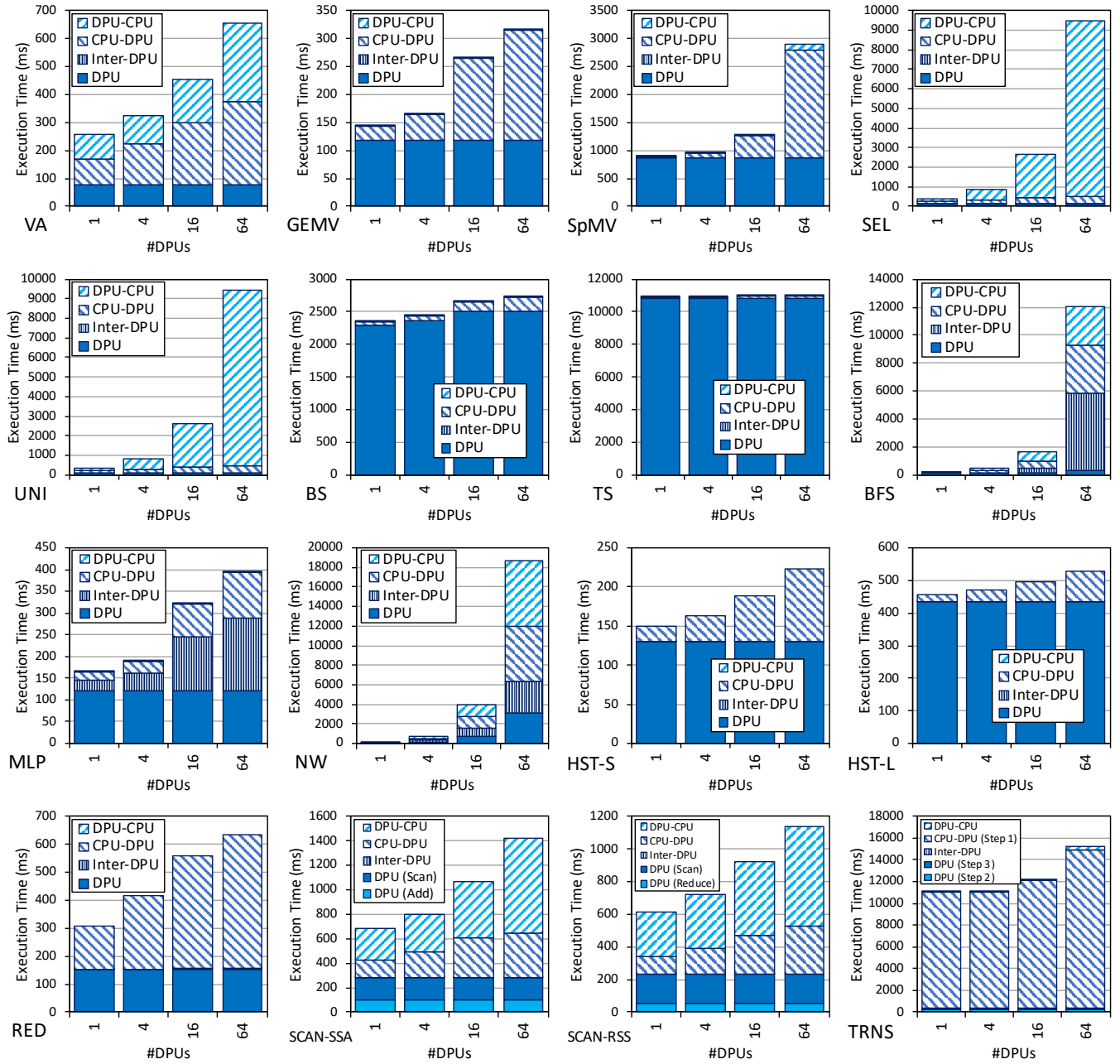


Figure 15: Execution time (ms) of 16 benchmarks on one rank (1, 4, 16, and 64 DPUs, using weak scaling⁴). Inside a DPU, we use the best-performing number of tasklets from Figure 12.

KEY OBSERVATION 18

Sustained bandwidth of parallel CPU-DPU/DPU-CPU transfers inside a rank of DRAM Processing Units (DPUs) increases sublinearly with the number of DPUs.

5.2 Comparison to CPU and GPU

We compare the UPMEM PIM architecture to a modern CPU and a modern GPU in terms of performance and energy consumption. Our goal is to quantify the potential of the UPMEM PIM architecture as a general-purpose accelerator. We use state-of-the-art CPU and GPU versions of PrIM benchmarks for comparison to our PIM implementations. The sources of the CPU and GPU versions of the benchmarks are listed in the Appendix (Table 5).

We compare the UPMEM-based PIM systems with 640 and 2,556 DPUs (Table 1) to an Intel Xeon E3-1225 v6 CPU [241] and an NVIDIA Titan V GPU [277] based on the Volta architecture [278] for all our benchmarks. Table 4 summarizes key characteristics of the CPU, the GPU, and the two UPMEM-based PIM systems.

For our UPMEM-based PIM system performance measurements, we include the time spent in the DPU and the time spent for inter-DPU synchronization on the UPMEM-based PIM systems. For our CPU and GPU performance measurements, we include only the kernel times (i.e., we do not include data transfers between the host CPU and the GPU in the GPU versions). For energy measurements, we use Intel RAPL [279] on the CPU and NVIDIA SMI [280] on the GPU. In the UPMEM PIM systems, we obtain the energy consumed by the DIMMs connected to the memory controllers, which can be done in x86 sockets [281]. The measurements include only the energy of the PIM chips.

5.2.1 Performance Comparison. Figure 16 shows the speedup of the UPMEM-based PIM systems with 640 and 2,556 DPUs and the Titan V GPU over the Intel Xeon CPU.

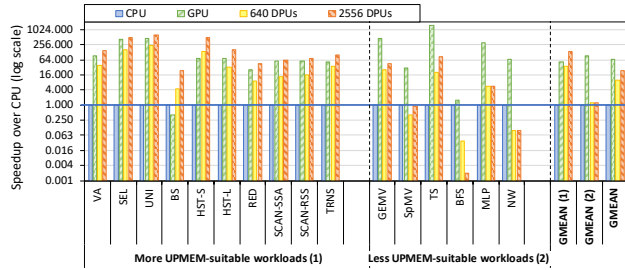


Figure 16: Performance comparison between the UPMEM-based PIM systems with 640 and 2,556 DPUs, a Titan V GPU, and an Intel Xeon E3-1240 CPU. Results are normalized to the CPU performance (y-axis is log scale). There are two groups of benchmarks: (1) benchmarks that are more suitable to the UPMEM PIM architecture, and (2) benchmarks that are less suitable to the UPMEM PIM architecture.

We make four key observations from Figure 16.

First, the 2,556-DPU system and the 640-DPU system are on average 23.2 $\times$ and 10.1 $\times$ faster than the CPU. The highest speedup is for UNI: the 2,556-DPU system is 629.5 $\times$ and the 640-DPU system is 234.4 $\times$ faster than the CPU. Even benchmarks that make heavy use of integer multiplication (GEMV, TS, and MLP) are much faster on the UPMEM-based PIM systems (5.8-86.6 $\times$ faster on the 2,556-DPU system, and 5.6-25.2 $\times$ faster on the 640-DPU system). This observation reflects the large performance improvements that workloads running on a conventional system with a CPU can experience if we expand the system with DIMMs of PIM-enabled memory (see Figure 1).

Second, the UPMEM-based PIM systems outperform the CPU for all of the benchmarks except SpMV, BFS, and NW. SpMV has three characteristics that make it less suitable for UPMEM-based PIM systems: (1) it operates on floating point data, (2) it uses multiplication heavily, and (3) it suffers from load imbalance due to the irregular nature of sparse matrices. Regarding the first two characteristics, we know from our analyses in Sections 3.1.2 and 3.3 that floating

point multiplication is very costly because of the lack of native support. Regarding the third characteristic, we know from our strong scaling evaluation in Section 5.1.1 that load imbalance across DPUs causes sublinear scaling for SpMV. BFS performs much worse than CPU on the UPMEM-based PIM systems because of the large overhead of inter-DPU synchronization via the host CPU, as we discuss in Section 5.1. Since the inter-DPU synchronization overhead of BFS increases linearly with the number of DPUs, the 2,556-DPU system is significantly slower than the 640-DPU system.¹⁵ Note that the goal of these experiments is *not* to show the performance of the best-performing number of DPUs for a given workload, but the performance of the full-blown systems with all 2,556 DPUs and 640 DPUs active for each workload. NW is one order of magnitude slower on both UPMEM-based PIM systems than on the CPU due to the inter-DPU synchronization overhead. The inter-DPU synchronization overhead of NW is not as dependent on the number of DPUs. As a result, the 2,556-DPU system has the same performance as the 640-DPU system for this benchmark.

Third, the 2,556-DPU system is faster than the GPU for 10 benchmarks: VA, SEL, UNI, BS, HST-S, HST-L, RED, SCAN-SSA, SCAN-RSS, and TRNS. These 10 benchmarks are more suitable to the UPMEM PIM architecture due to three key characteristics: (1) streaming memory accesses, (2) no or little inter-DPU communication, and (3) no or little use of integer multiplication, integer division, or floating point operations. The speedups of the 2,556-DPU system over the GPU for these benchmarks range between 6% (for SCAN-SSA) and 57.5 $\times$ (for BS), with an average of 2.54 $\times$. It is especially interesting that the 2,556-DPU system outperforms the Titan V for some of these benchmarks, which are traditionally considered GPU-friendly and are subject of GPU optimization studies, libraries, and reference implementations, such as VA [282], SEL and UNI [250, 283], HST-S and HST-L [260, 272, 284], RED [262, 263], SCAN-SSA [264, 265, 283], SCAN-RSS [251, 264, 266, 285], and TRNS [269, 270, 286]. In summary, the UPMEM PIM architecture outperforms the modern GPU for workloads that exhibit the three key characteristics that make them potentially suitable for execution on the UPMEM-based PIM system.

Fourth, the 640-DPU system is generally slower than the GPU, but for the 10 benchmarks where the 2,556-DPU system is faster than the GPU (VA, SEL, UNI, BS, HST-S, HST-L, RED, SCAN-SSA, SCAN-RSS, and TRNS) the average performance of the 640-DPU system is within 65% the performance of the GPU. Among these benchmarks, the 640-DPU system is faster than the GPU for two benchmarks: HST-S and BS. The GPU version of histogram [260, 287] (the same one for HST-S and HST-L) uses atomic operations that burden the performance heavily [272]. As a result, the 640-DPU system is 1.89 $\times$ faster than the GPU for HST-S. For BS, the GPU version suffers from many random memory accesses, which greatly reduce the achievable memory bandwidth. The 640-DPU system is 11.0 $\times$ faster than the GPU for BS.

¹⁵BFS can obtain better performance by running it using much fewer DPUs. The reason is that BFS performance does not scale with many DPUs, as shown in Sections 5.1.1 and 5.1.2 (Figures 13-15). However, we do not run BFS using much fewer DPUs as we study the full-blown system performance utilizing *all* DPUs in this experiment.

Table 4: Evaluated CPU, GPU, and UPMEM-based PIM Systems.

System	Process Node	Processor Cores			Memory		TDP
		Total Cores	Frequency	Peak Performance	Capacity	Total Bandwidth	
Intel Xeon E3-1225 v6 CPU [241]	14 nm	4 (8 threads)	3.3 GHz	26.4 GFLOPS*	32 GB	37.5 GB/s	73 W
NVIDIA Titan V GPU [277]	14 nm	80 (5,120 SIMD lanes)	1.2 GHz	12,288.0 GFLOPS	12 GB	652.8 GB/s	250 W
2,556-DPU PIM System	2x nm	2,556 ⁹	350 MHz	894.6 GOPS	159.75 GB	1.7 TB/s	383 W [†]
640-DPU PIM System	2x nm	640	267 MHz	170.9 GOPS	40 GB	333.75 GB/s	96 W [†]

*Estimated GFLOPS = 3.3 GHz × 4 cores × 2 instructions per cycle.

[†]Estimated TDP = $\frac{\text{Total DPUs}}{\text{DPUs/chip}} \times 1.2 \text{ W/chip}$ [199].

KEY OBSERVATION 19

The UPMEM-based PIM system can outperform a modern GPU on workloads with three key characteristics:

1. Streaming memory accesses
2. No or little inter-DPU synchronization
3. No or little use of integer multiplication, integer division, or floating point operations

These three key characteristics make a **workload potentially suitable to the UPMEM PIM architecture.**

5.2.2 Energy Comparison. Figure 17 shows the energy savings of the UPMEM-based PIM system with 640 DPUs and the Titan V GPU over the Intel Xeon CPU. At the time of writing, the 2,556-DPU system is not enabled to perform energy measurements, but we will aim to include them in an extended version of our work.

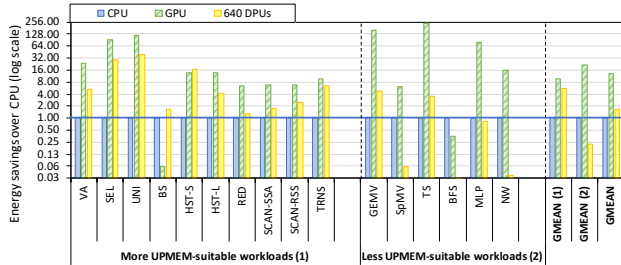


Figure 17: Energy comparison between the UPMEM-based PIM system with 640 DPUs, a Titan V GPU, and an Intel Xeon E3-1240 CPU. Results are normalized to the CPU performance (y-axis is log scale). There are two groups of benchmarks: (1) benchmarks that are more suitable to the UPMEM PIM architecture, and (2) benchmarks that are less suitable to the UPMEM PIM architecture.

We make three key observations from Figure 17.

First, the 640-DPU system consumes, on average, 1.64× less energy than the CPU for all 16 benchmarks. For 12 benchmarks (VA, GEMV, SEL, UNI, BS, TS, HST-S, HST-L, RED, SCAN-SSA, SCAN-RSS, and TRNS), the 640-DPU system provides an energy savings of 5.23× over the CPU. The maximum energy savings is 39.14× for UNI. Our experiments show that the 640-DPU system, featuring PIM-enabled memory with a capacity of 40 GB, provides outstanding energy savings over a modern Intel Xeon CPU (with

memory capacity of 32 GB) for 12 out of 16 benchmarks. This energy savings comes from the lower execution times of these 12 benchmarks on the 640-DPU system (Figure 16). We expect that the energy savings of the 2,556-DPU system, with ~6× more DPUs, 160 GB of PIM-enabled memory, and higher frequency (350 vs. 267 MHz), over the CPU will be even higher due to higher performance (thus, lower static energy) and less data movement.

Second, the 640-DPU system is only less energy efficient than the CPU for SpMV, BFS, and NW, which is in line with our observations about performance (Section 5.2.1).

Third, compared to the GPU, the 640-DPU system consumes less energy for BS and HST-S, since these are the two benchmarks for which the 640-DPU system outperforms the GPU (see Section 5.2.1). For the 2,556-DPU system, we expect energy results to follow the performance results in Section 5.2.1. The 10 benchmarks (VA, SEL, UNI, BS, HST-S, HST-L, RED, SCAN-SSA, SCAN-RSS, and TRNS) that run faster on the 2,556-DPU system than on the GPU will also likely consume less energy. This is because the major cause of performance improvement and energy reduction is the same: the reduction in data movement between memory and processors that the UPMEM-based PIM systems provide.

KEY OBSERVATION 20

The UPMEM-based PIM system provides large energy savings over a modern CPU due to higher performance (thus, lower static energy) and less data movement between memory and processors.

The UPMEM-based PIM system provides energy savings over a modern CPU/GPU on workloads where it outperforms the CPU/GPU. This is because the source of both performance improvement and energy savings is the same: **the significant reduction in data movement between the memory and the processor cores**, which the UPMEM-based PIM system can provide for PIM-suitable workloads.

5.2.3 Discussion. These observations are useful for programmers to anticipate how much performance and energy savings they can get from the UPMEM hardware compared to traditional CPU and GPU systems for different types of workloads.

One limitation of this comparison is the difficulty of establishing a common control factor across all three types of systems (CPU, GPU, and UPMEM-based PIM system) to ensure a fair comparison. To this end, the 640-DPU PIM system has comparable memory capacity to the CPU (40 GB vs. 32 GB). However, the 2,556-DPU

system has much higher memory capacity (~160 GB). On the other hand, the 640-DPU UPMEM-based PIM system and the GPU have comparable cost (the 640-DPU system being a little cheaper). Other hardware characteristics, such as fabrication technology, process node, number of cores, or frequency (Table 5), are very different across the four systems that we evaluate in Section 5.2.

We note that the UPMEM hardware is still maturing and is expected to run at a higher frequency in the near future (400-450 MHz instead of 350 or 267 MHz) and potentially be manufactured with a smaller technology node [231]. Hence, the results we report in this comparison may underestimate the full potential of the UPMEM-based PIM architecture. CPU and GPU systems have been heavily optimized for decades in terms of architecture, software, and manufacturing. We believe the architecture, software, and manufacturing of PIM systems will continue to improve (see our suggestions for future improvement in Section 6).

6 KEY TAKEAWAYS

In this section, we reiterate several key empirical observations in the form of four key takeaways we provide throughout this paper. We also provide implications on workload suitability and good programming practices for the UPMEM PIM architecture, and suggestions for hardware and architecture designers of future PIM systems.

Key Takeaway #1. The UPMEM PIM architecture is fundamentally compute bound. Section 3.2 shows that workloads with more complex operations than integer addition fully utilize the instruction pipeline before they can potentially saturate the memory bandwidth. Section 3.3 shows that even workloads with as simple operations as integer addition saturate the compute throughput with an operational intensity as low as 0.25 operations/byte (1 addition per integer accessed). This key takeaway shows that **the most suitable workloads for the UPMEM PIM architecture are memory-bound workloads**. From a programmer’s perspective, the architecture requires a shift in how we think about computation and data access, since the relative cost of computation vs. data access in the PIM system is very different from that in the dominant processor-centric architectures of today.

KEY TAKEAWAY 1

The UPMEM PIM architecture is fundamentally compute bound. As a result, the most suitable workloads are memory-bound.

Key Takeaway #2. The workloads most well-suited for the UPMEM PIM architecture are those with simple or no arithmetic operations. This is because DPUs include native support for *only* integer addition/subtraction and bitwise operations. More complex integer (e.g., multiplication, division) and floating point operations are implemented using software library routines. Section 3.1.2 shows that the arithmetic throughput of more complex integer operations and floating point operations are an order of magnitude lower than that of simple addition and subtraction. Section 5.2 shows that benchmarks with little amount of computation and no use of multiplication, division, or floating point operations

(10 out of 16 benchmarks) run faster (2.54× on average) on a 2,556-DPU system than on a modern NVIDIA Titan V GPU. These observations show that **the workloads most well-suited for the UPMEM PIM architecture are those with no arithmetic operations or simple operations (e.g., bitwise operations and integer addition/subtraction)**. Based on this key takeaway, we recommend devising much more efficient software library routines or, more importantly, specialized and fast in-memory hardware for complex operations in future PIM architecture generations to improve the general-purpose performance of PIM systems.

KEY TAKEAWAY 2

The most well-suited workloads for the UPMEM PIM architecture use no arithmetic operations or use only simple operations (e.g., bitwise operations and integer addition/subtraction).

Key Takeaway #3. The workloads most well-suited for the UPMEM PIM architecture are those with little global communication, because there is no direct communication channel among DPUs. As a result, there is a huge disparity in performance scalability of benchmarks that do *not* require inter-DPU communication and benchmarks that do (especially if parallel transfers across MRAM banks cannot be used), as Section 5.1 shows. This key takeaway shows that **the workloads most well-suited for the UPMEM PIM architecture are those with little or no inter-DPU communication**. Based on this takeaway, we recommend that the hardware architecture and the software stack be enhanced with support for inter-DPU communication (e.g., by leveraging new in-DRAM data copy techniques [27, 28, 33, 38, 39, 188, 190] and providing better connectivity inside DRAM [33, 38]).

KEY TAKEAWAY 3

The most well-suited workloads for the UPMEM PIM architecture require little or no communication across DRAM Processing Units (inter-DPU communication).

Summary. We find that the workloads most suitable for the UPMEM PIM architecture in its current form are (1) memory-bound workloads with (2) simple or no arithmetic operations and (3) little or no inter-DPU communication.

Key Takeaway #4. We observe that the existing UPMEM-based PIM systems greatly improve energy efficiency and performance over modern CPU and GPU systems across many workloads we examine. Section 5.2 shows that the 2,556-DPU and the 640-DPU systems are 23.2× and 10.1× faster, respectively, than a modern Intel Xeon CPU, averaged across the entire set of 16 PrIM benchmarks. The 640-DPU system is 1.64× more energy efficient than the CPU, averaged across the entire set of 16 PrIM benchmarks, and 5.23× more energy efficient for 12 of the PrIM benchmarks.

The 2,556-DPU system is faster (on average by 2.54×) than the modern GPU in 10 out of 16 PrIM benchmarks, which have three key characteristics that define a workload’s PIM suitability: (1)

streaming memory accesses, (2) little or no inter-DPU communication, and (3) little or no use of multiplication, division, or floating point operations.¹⁶

We expect that the 2,556-DPU system will provide even higher performance and energy benefits, and that future PIM systems will be even better (especially after implementing our recommendations for future PIM hardware). If the architecture is improved based on our recommendations under Key Takeaways 1-3, we believe the future PIM system will be even more attractive, leading to much higher performance and energy benefits versus modern CPUs and GPUs over potentially all workloads.

KEY TAKEAWAY 4

- UPMEM-based PIM systems **outperform modern CPUs in terms of performance and energy efficiency on most of PrIM benchmarks.**
- UPMEM-based PIM systems **outperform modern GPUs on a majority of PrIM benchmarks**, and the outlook is even more positive for future PIM systems.
- UPMEM-based PIM systems are **more energy-efficient than modern CPUs and GPUs on workloads that they provide performance improvements over the CPUs and the GPUs.**

7 RELATED WORK

To our knowledge, this paper provides the first comprehensive characterization and analysis of the first publicly-available real-world PIM architecture along with the first open-source benchmark suite for a real-world PIM architecture.

We briefly review related work on PIM architectures.

There are two main approaches to PIM [1–4]: (1) *processing-using-memory (PUM)* and (2) *processing-near-memory (PNM)*. No prior work on PUM or PNM provides results from real commercial systems or a benchmark suite to evaluate PIM architectures.

Processing using memory (PUM) exploits the existing memory architecture and the operational principles of the memory cells and circuitry to perform operations within each memory chip at low cost. Prior works propose PUM mechanisms using SRAM [21–24], DRAM [25–40, 99, 100, 119, 188–191, 288], PCM [41], MRAM [42–44], or RRAM/memristive [45–55, 70, 103, 114, 118, 192, 193, 289] memories. PUM mechanisms enable different types of operations such as data copy and initialization [21, 27, 28, 33, 38, 39, 119, 190], bulk bitwise operations (e.g., a functionally-complete set of Boolean logic operations) [21, 25, 26, 32, 34, 35, 41–44, 145, 188, 191], and simple arithmetic operations (e.g., addition, multiplication, implication) [21–24, 29, 35, 36, 45–55, 99]. A recent work, called SIM-DRAM [37], designs a framework for implementing and executing arbitrary operations in a bit-serial SIMD fashion inside DRAM arrays, building on the Ambit substrate [25, 26].

¹⁶Note that these three characteristics are not exactly the same three characteristics highlighted by key takeaways #1 to #3, but more specific. The difference is that the 2,556-DPU system outperforms the GPU for memory-bound workloads with streaming memory accesses. These workloads do not need to have *only* streaming memory accesses, since BS, HST-S, HST-L, and TRNS, for which the 2,556-DPU system outperforms the GPU, have also random accesses. Since all PrIM workloads (see Table 2) contain some streaming memory accesses, we cannot say that the 2,556-DPU system outperforms the GPU for workloads with *only* strided and/or random accesses.

Processing near memory (PNM) integrates processing elements (e.g., functional units, accelerators, simple processing cores, reconfigurable logic) near or inside the memory (e.g., [8, 56–69, 71–96, 98, 101, 104–110, 113, 120–130, 290, 291]). Many of these PNM works place PIM logic inside the logic layer of 3D-stacked memories [8, 56, 58–67, 75, 78–80, 82, 84–87, 89–95, 101, 106, 108–111, 113, 128–130, 290, 291], at the memory controller [76, 77], on the DDRX DIMMs [68, 121, 292, 293], or in the same package as the CPU connected via silicon interposers [57, 126, 127, 186].

Another body of recent works study and propose solutions to system integration challenges in PIM-enabled systems, such as **memory coherence** [64–66], **virtual memory** [185, 294], **synchronization** [56], or **PIM suitability of workloads** [128, 184].

Several works explore the acceleration opportunities offered by the UPMEM PIM architecture for bioinformatics [295, 296], skyline computation [297], compression [298], or sparse linear algebra [299]. Readers can refer to these works for in-depth analysis of specific applications on the UPMEM PIM architecture. Our work is the first one that performs a comprehensive architecture characterization of the UPMEM PIM architecture and studies the PIM suitability of a large number of workloads. We are also the first to openly and freely provide the first benchmark suite for real PIM systems.

A recent work [122, 123] presents a real-world PIM system with programmable near-bank computation units, called **FIM-DRAM, based on HBM technology** [168, 169]. The **FIMDRAM architecture**, designed specifically for machine learning applications, implements a SIMD pipeline with simple multiply-and-accumulate units [300, 301]. More recently presented, **Accelerator-in-Memory** [120] is a GDDR6-based PIM architecture with specialized units for multiply-and-accumulate and activation functions for deep learning applications. **AxDIMM** [121] is a DIMM-based solution which places an FPGA fabric in the buffer chip of the DIMM. It has been tested for recommendation inference. Compared to the more general-purpose UPMEM PIM architecture, these architectures focus on a specific domain of applications (i.e., machine learning), and thus it may lack flexibility to support a wider range of applications. A comprehensive characterization and analysis of these architectures, along the lines of our work, can greatly help researchers, programmers, and architects to understand their potential.

8 SUMMARY & CONCLUSION

We present the first comprehensive characterization and analysis of a real commercial PIM architecture. Through this analysis, we develop a rigorous, thorough understanding of the UPMEM PIM architecture, the first publicly-available PIM architecture, and its suitability to various types of workloads.

First, we conduct a characterization of the UPMEM-based PIM system using microbenchmarks to assess various architecture limits such as compute throughput and memory bandwidth, yielding new insights. Second, we present PrIM, a benchmark suite of 16 memory-bound workloads from different application domains (e.g., dense/sparse linear algebra, databases, data analytics, graph processing, neural networks, bioinformatics, image processing).

Our extensive evaluation of PrIM benchmarks conducted on two real systems with UPMEM memory modules provides new insights about suitability of different workloads to the PIM system, programming recommendations for software designers, and suggestions and hints for hardware and architecture designers of future PIM systems. We compare the performance and energy consumption of the UPMEM-based PIM systems for PrIM benchmarks to those of a modern CPU and a modern GPU, and identify key workload characteristics that can successfully leverage the key strengths of a real PIM system over conventional processor-centric architectures. We note that we compare the first ever commercial PIM system to CPU and GPU systems that have been heavily optimized for decades in terms of architecture, software, and manufacturing. As the architecture, software, and manufacturing of PIM systems continue to improve, it will be possible to do more fair comparisons to CPU and GPU systems, which reveal even higher benefits for PIM systems in the future.

We believe and hope that our work will provide valuable insights to programmers, users and architects of this PIM architecture as well as of future PIM systems, and will represent an enabling milestone in the development of memory-centric computing systems.

ACKNOWLEDGMENTS

We thank UPMEM's Fabrice Devaux, Rémy Cimadomo, Romaric Jodin, and Vincent Palatin for their valuable support. We acknowledge the support of SAFARI Research Group's industrial partners, especially ASML, Facebook, Google, Huawei, Intel, Microsoft, VMware, and the Semiconductor Research Corporation. Izzat El Hajj acknowledges the support of the University Research Board of the American University of Beirut (URB-AUB-103951-25960).

This article is a greatly extended version of [302], a summary of our work presented at the 2021 12th International Green and Sustainable Computing Conference (IGSC). Talk videos for this work are available on YouTube, including a 3-minute talk video (https://youtu.be/SrFD_u46EDA), a 20-minute talk video (<https://youtu.be/Pp9jSU2b9oM>), a 1-hour talk video (https://youtu.be/6Ws3h_CQO_Q), and a long talk video (<https://youtu.be/D8Hjy2iU9l4>).

REFERENCES

- [1] O. Mutlu *et al.*, "Processing Data Where It Makes Sense: Enabling In-Memory Computation," *MicPro*, 2019.
- [2] O. Mutlu *et al.*, "A Modern Primer on Processing in Memory," *Emerging Computing: From Devices to Systems - Looking Beyond Moore and Von Neumann*, 2021, <https://arxiv.org/pdf/2012.03112.pdf>.
- [3] S. Ghose *et al.*, "Processing-in-Memory: A Workload-Driven Perspective," *IBM JRD*, 2019.
- [4] S. Ghose *et al.*, "A Workload and Programming Ease Driven Perspective of Processing-in-Memory," arXiv:1907.12947 [cs:AR], 2019.
- [5] S. Ghose *et al.*, "Enabling the Adoption of Processing-in-Memory: Challenges, Mechanisms, Future Research Directions," arXiv:1802.00320 [cs:AR], 2018.
- [6] O. Mutlu, "Processing Data Where It Makes Sense: Enabling In-Memory Computation," <https://bit.ly/3IVpj0j>, 2017, keynote talk at MST.
- [7] O. Mutlu *et al.*, "Enabling Practical Processing in and near Memory for Data-Intensive Computing," in *DAC*, 2019.
- [8] A. Boroumand *et al.*, "Google Workloads for Consumer Devices: Mitigating Data Movement Bottlenecks," in *ASPLOS*, 2018.
- [9] D. Pandiyan and C.-J. Wu, "Quantifying the Energy Cost of Data Movement for Emerging Smart Phone Workloads on Mobile Platforms," in *IISWC*, 2014.
- [10] G. Kestor *et al.*, "Quantifying the Energy Cost of Data Movement in Scientific Applications," in *IISWC*, 2013.
- [11] W. H. Kautz, "Cellular Logic-in-Memory Arrays," *IEEE TC*, 1969.
- [12] H. S. Stone, "A Logic-in-Memory Computer," *IEEE TC*, 1970.
- [13] D. E. Shaw *et al.*, "The NON-VON Database Machine: A Brief Overview," *IEEE Database Eng. Bull.*, 1981.
- [14] P. M. Kogge, "EXECUBE - A New Architecture for Scaleable MPPs," in *ICPP*, 1994.
- [15] M. Gokhale *et al.*, "Processing in Memory: The Terasys Massively Parallel PIM Array," *IEEE Computer*, 1995.
- [16] D. Patterson *et al.*, "A Case for Intelligent RAM," *IEEE Micro*, 1997.
- [17] M. Oskin *et al.*, "Active Pages: A Computation Model for Intelligent Memory," in *ISCA*, 1998.
- [18] Y. Kang *et al.*, "FlexRAM: Toward an Advanced Intelligent Memory System," in *ICCD*, 1999.
- [19] K. Mai *et al.*, "Smart Memories: A Modular Reconfigurable Architecture," in *ISCA*, 2000.
- [20] J. Draper *et al.*, "The Architecture of the DIVA Processing-in-Memory Chip," in *SC*, 2002.
- [21] S. Aga *et al.*, "Compute Caches," in *HPCA*, 2017.
- [22] C. Eckert *et al.*, "Neural Cache: Bit-serial In-cache Acceleration of Deep Neural Networks," in *ISCA*, 2018.
- [23] D. Fujiki *et al.*, "Duality Cache for Data Parallel Acceleration," in *ISCA*, 2019.
- [24] M. Kang *et al.*, "An Energy-Efficient VLSI Architecture for Pattern Recognition via Deep Embedding of Computation in SRAM," in *ICASSP*, 2014.
- [25] V. Seshadri *et al.*, "Ambit: In-Memory Accelerator for Bulk Bitwise Operations Using Commodity DRAM Technology," in *MICRO*, 2017.
- [26] V. Seshadri *et al.*, "Buddy-RAM: Improving the Performance and Efficiency of Bulk Bitwise Operations Using DRAM," arXiv:1611.09988 [cs:AR], 2016.
- [27] V. Seshadri *et al.*, "RowClone: Fast and Energy-Efficient In-DRAM Bulk Data Copy and Initialization," in *MICRO*, 2013.
- [28] V. Seshadri *et al.*, "RowClone: Accelerating Data Movement and Initialization Using DRAM," arXiv:1805.03502 [cs:AR], 2018.
- [29] S. Angizi and D. Fan, "Graphide: A Graph Processing Accelerator Leveraging In-dram-computing," in *GLSVLSI*, 2019.
- [30] J. Kim *et al.*, "The DRAM Latency PUF: Quickly Evaluating Physical Unclonable Functions by Exploiting the Latency-Reliability Tradeoff in Modern DRAM Devices," in *HPCA*, 2018.
- [31] J. Kim *et al.*, "D-RaNGe: Using Commodity DRAM Devices to Generate True Random Numbers with Low Latency and High Throughput," in *HPCA*, 2019.
- [32] F. Gao *et al.*, "ComputeDRAM: In-Memory Compute Using Off-the-Shelf DRAMs," in *MICRO*, 2019.
- [33] K. K. Chang *et al.*, "Low-Cost Inter-Linked Subarrays (LISA): Enabling Fast Inter-Subarray Data Movement in DRAM," in *HPCA*, 2016.
- [34] X. Xin *et al.*, "ELP2IM: Efficient and Low Power Bitwise Operation Processing in DRAM," in *HPCA*, 2020.
- [35] S. Li *et al.*, "DRISA: A DRAM-Based Reconfigurable In-Situ Accelerator," in *MICRO*, 2017.
- [36] Q. Deng *et al.*, "DrAcc: a DRAM Based Accelerator for Accurate CNN Inference," in *DAC*, 2018.
- [37] N. Hajinazar *et al.*, "SIMDRAM: A Framework for Bit-Serial SIMD Processing Using DRAM," in *ASPLOS*, 2021.
- [38] S. H. S. Rezaei *et al.*, "NoM: Network-on-Memory for Inter-Bank Data Transfer in Highly-Banked Memories," *CAL*, 2020.
- [39] Y. Wang *et al.*, "FIGARO: Improving System Performance via Fine-Grained In-DRAM Data Relocation and Caching," in *MICRO*, 2020.
- [40] M. F. Ali *et al.*, "In-Memory Low-Cost Bit-Serial Addition Using Commodity DRAM Technology," in *TCAS-I*, 2019.
- [41] S. Li *et al.*, "Pinatubo: A Processing-in-Memory Architecture for Bulk Bitwise Operations in Emerging Non-Volatile Memories," in *DAC*, 2016.
- [42] S. Angizi *et al.*, "PIMA-Logic: A Novel Processing-in-Memory Architecture for Highly Flexible and Energy-efficient Logic Computation," in *DAC*, 2018.
- [43] S. Angizi *et al.*, "CMP-PIM: An Energy-efficient Comparator-based Processing-in-Memory Neural Network Accelerator," in *DAC*, 2018.
- [44] S. Angizi *et al.*, "AlignS: A Processing-in-Memory Accelerator for DNA Short Read Alignment Leveraging SOT-MRAM," in *DAC*, 2019.
- [45] Y. Levy *et al.*, "Logic Operations in Memory Using a Memristive Akers Array," *Microelectronics Journal*, 2014.
- [46] S. Kvatsinsky *et al.*, "MAGIC—Memristor-Aided Logic," *IEEE TCAS II: Express Briefs*, 2014.
- [47] A. Shafiee *et al.*, "ISAAC: A Convolutional Neural Network Accelerator with In-situ Analog Arithmetic in Crossbars," in *ISCA*, 2016.
- [48] S. Kvatsinsky *et al.*, "Memristor-Based IMPLY Logic Design Procedure," in *ICCD*, 2011.
- [49] S. Kvatsinsky *et al.*, "Memristor-Based Material Implication (IMPLY) Logic: Design Principles and Methodologies," *TVLSI*, 2014.
- [50] P.-E. Gaillardon *et al.*, "The Programmable Logic-in-Memory (PLiM) Computer," in *DATE*, 2016.
- [51] D. Bhattacharjee *et al.*, "ReVAMP: ReRAM based VLIW Architecture for In-memory Computing," in *DATE*, 2017.
- [52] S. Hamdioui *et al.*, "Memristor Based Computation-in-Memory Architecture for Data-intensive Applications," in *DATE*, 2015.

- [53] L. Xie *et al.*, "Fast Boolean Logic Papped on Memristor Crossbar," in *ICCD*, 2015.
- [54] S. Hamdioui *et al.*, "Memristor for Computing: Myth or Reality?" in *DATE*, 2017.
- [55] J. Yu *et al.*, "Memristive Devices for Computation-in-Memory," in *DATE*, 2018.
- [56] C. Giannoula *et al.*, "SynCron: Efficient Synchronization Support for Near-Data-Processing Architectures," in *HPCA*, 2021.
- [57] I. Fernandez *et al.*, "NATSA: A Near-Data Processing Accelerator for Time Series Analysis," in *ICCD*, 2020.
- [58] M. Alser *et al.*, "Accelerating Genome Analysis: A Primer on an Ongoing Journey," *IEEE Micro*, 2020.
- [59] D. S. Cali *et al.*, "GenASM: A High-Performance, Low-Power Approximate String Matching Acceleration Framework for Genome Sequence Analysis," in *MICRO*, 2020.
- [60] J. S. Kim *et al.*, "GRIM-Filter: Fast Seed Filtering in Read Mapping Using Emerging Memory Technologies," arXiv:1708.04329 [q-bio.GN], 2017.
- [61] J. S. Kim *et al.*, "GRIM-Filter: Fast Seed Location Filtering in DNA Read Mapping Using Processing-in-Memory Technologies," *BMC Genomics*, 2018.
- [62] J. Ahn *et al.*, "PIM-Enabled Instructions: A Low-Overhead, Locality-Aware Processing-in-Memory Architecture," in *ISCA*, 2015.
- [63] J. Ahn *et al.*, "A Scalable Processing-in-Memory Accelerator for Parallel Graph Processing," in *ISCA*, 2015.
- [64] A. Boroumand *et al.*, "CoNDA: Efficient Cache Coherence Support for near-Data Accelerators," in *ISCA*, 2019.
- [65] A. Boroumand *et al.*, "LazyPIM: An Efficient Cache Coherence Mechanism for Processing-in-Memory," *CAL*, 2016.
- [66] A. Boroumand *et al.*, "LazyPIM: Efficient Support for Cache Coherence in Processing-in-Memory Architectures," arXiv:1706.03162 [cs.AR], 2017.
- [67] G. Singh *et al.*, "NAPEL: Near-memory Computing Application Performance Prediction via Ensemble Learning," in *DAC*, 2019.
- [68] H. Asghari-Moghaddam *et al.*, "Chameleon: Versatile and Practical Near-DRAM Acceleration Architecture for Large Memory Systems," in *MICRO*, 2016.
- [69] O. O. Babarinsa and S. Idreos, "JAFAR: Near-Data Processing for Databases," in *SIGMOD*, 2015.
- [70] P. Chi *et al.*, "PRIME: A Novel Processing-In-Memory Architecture for Neural Network Computation In ReRAM-Based Main Memory," in *ISCA*, 2016.
- [71] A. Farmahini-Farahani *et al.*, "NDA: Near-DRAM acceleration architecture leveraging commodity DRAM devices and standard memory modules," in *HPCA*, 2015.
- [72] M. Gao *et al.*, "Practical Near-Data Processing for In-Memory Analytics Frameworks," in *PACT*, 2015.
- [73] M. Gao and C. Kozyrakis, "HRL: Efficient and Flexible Reconfigurable Logic for Near-Data Processing," in *HPCA*, 2016.
- [74] B. Gu *et al.*, "Biscuit: A Framework for Near-Data Processing of Big Data Workloads," in *ISCA*, 2016.
- [75] Q. Guo *et al.*, "3D-Stacked Memory-Side Acceleration: Accelerator and System Design," in *WoNDP*, 2014.
- [76] M. Hashemi *et al.*, "Accelerating Dependent Cache Misses with an Enhanced Memory Controller," in *ISCA*, 2016.
- [77] M. Hashemi *et al.*, "Continuous Runahead: Transparent Hardware Acceleration for Memory Intensive Workloads," in *MICRO*, 2016.
- [78] K. Hsieh *et al.*, "Transparent Offloading and Mapping (TOM): Enabling Programmer-Transparent Near-Data Processing in GPU Systems," in *ISCA*, 2016.
- [79] D. Kim *et al.*, "Neurocube: A Programmable Digital Neuromorphic Architecture with High-Density 3D Memory," in *ISCA*, 2016.
- [80] G. Kim *et al.*, "Toward Standardized Near-Data Processing with Unrestricted Data Placement for GPUs," in *SC*, 2017.
- [81] J. H. Lee *et al.*, "BSSync: Processing Near Memory for Machine Learning Workloads with Bounded Staleness Consistency Models," in *PACT*, 2015.
- [82] Z. Liu *et al.*, "Concurrent Data Structures for Near-Memory Computing," in *SPAA*, 2017.
- [83] A. Morad *et al.*, "GP-SIMD Processing-in-Memory," *ACM TACO*, 2015.
- [84] L. Nai *et al.*, "GraphPIM: Enabling Instruction-Level PIM Offloading in Graph Computing Frameworks," in *HPCA*, 2017.
- [85] A. Pattnaik *et al.*, "Scheduling Techniques for GPU Architectures with Processing-in-Memory Capabilities," in *PACT*, 2016.
- [86] S. H. Pugsley *et al.*, "NDC: Analyzing the Impact of 3D-Stacked Memory+Logic Devices on MapReduce Workloads," in *ISPASS*, 2014.
- [87] D. P. Zhang *et al.*, "TOP-PIM: Throughput-Oriented Programmable Processing in Memory," in *HPDC*, 2014.
- [88] Q. Zhu *et al.*, "Accelerating Sparse Matrix-Matrix Multiplication with 3D-Stacked Logic-in-Memory Hardware," in *HPEC*, 2013.
- [89] B. Akin *et al.*, "Data Reorganization in Memory Using 3D-Stacked DRAM," in *ISCA*, 2015.
- [90] M. Gao *et al.*, "Tetris: Scalable and Efficient Neural Network Acceleration with 3D Memory," in *ASPLOS*, 2017.
- [91] M. Drumond *et al.*, "The Mondrian Data Engine," in *ISCA*, 2017.
- [92] G. Dai *et al.*, "GraphH: A Processing-in-Memory Architecture for Large-scale Graph Processing," *IEEE TCAD*, 2018.
- [93] M. Zhang *et al.*, "GraphP: Reducing Communication for PIM-based Graph Processing with Efficient Data Partition," in *HPCA*, 2018.
- [94] Y. Huang *et al.*, "A Heterogeneous PIM Hardware-Software Co-Design for Energy-Efficient Graph Processing," in *IPDPS*, 2020.
- [95] Y. Zhuo *et al.*, "GraphQ: Scalable PIM-based Graph Processing," in *MICRO*, 2019.
- [96] P. C. Santos *et al.*, "Operand Size Reconfiguration for Big Data Processing in Memory," in *DATE*, 2017.
- [97] W.-M. Hwu *et al.*, "Rebooting the Data Access Hierarchy of Computing Systems," in *ICRC*, 2017.
- [98] M. Besta *et al.*, "SISA: Set-Centric Instruction Set Architecture for Graph Mining on Processing-in-Memory Systems," in *MICRO*, 2021.
- [99] J. D. Ferreira *et al.*, "pLUTo: In-DRAM Lookup Tables to Enable Massively Parallel General-Purpose Computation," arXiv:2104.07699 [cs.AR], 2021.
- [100] A. Olgun *et al.*, "QUAC-TRNG: High-Throughput True Random Number Generation Using Quadruple Row Activation in Commodity DRAMs," in *ISCA*, 2021.
- [101] S. Lloyd and M. Gokhale, "In-memory Data Rearrangement for Irregular, Data-intensive Computing," *Computer*, 2015.
- [102] D. G. Elliott *et al.*, "Computational RAM: Implementing Processors in Memory," *IEEE Design & Test of Computers*, 1999.
- [103] L. Zheng *et al.*, "RRAM-based TCAMs for pattern search," in *ISCAS*, 2016.
- [104] J. Landgraf *et al.*, "Combining Emulation and Simulation to Evaluate a Near Memory Key/Value Lookup Accelerator," 2021.
- [105] A. Rodrigues *et al.*, "Towards a Scatter-Gather Architecture: Hardware and Software Issues," in *MEMSYS*, 2019.
- [106] S. Lloyd and M. Gokhale, "Design Space Exploration of Near Memory Accelerators," in *MEMSYS*, 2018.
- [107] S. Lloyd and M. Gokhale, "Near Memory Key/Value Lookup Acceleration," in *MEMSYS*, 2017.
- [108] M. Gokhale *et al.*, "Near Memory Data Structure Rearrangement," in *MEMSYS*, 2015.
- [109] R. Nair *et al.*, "Active Memory Cube: A Processing-in-Memory Architecture for Exascale Systems," *IBM JRD*, 2015.
- [110] A. C. Jacob *et al.*, "Compiling for the Active Memory Cube," Tech. rep. RC25644 (WAT1612-008). IBM Research Division, Tech. Rep., 2016.
- [111] Z. Sura *et al.*, "Data Access Optimization in a Processing-in-Memory System," in *CF*, 2015.
- [112] R. Nair, "Evolution of Memory Architecture," *Proceedings of the IEEE*, 2015.
- [113] R. Balasubramanian *et al.*, "Near-Data Processing: Insights from a MICRO-46 Workshop," *IEEE Micro*, 2014.
- [114] Y. Xi *et al.*, "In-Memory Learning With Analog Resistive Switching Memory: A Review and Perspective," *Proceedings of the IEEE*, 2020.
- [115] Onur Mutlu and Juan Gómez-Luna, "Exploring the Processing-in-Memory Paradigm for Future Computing Systems (Fall 2021)," https://safari.ethz.ch/projects_and_seminars/fall2021/doku.php?id=processing_in_memory.
- [116] Onur Mutlu, "Computer Architecture (Fall 2021)," <https://safari.ethz.ch/architecture/fall2021/doku.php?id=start>.
- [117] Onur Mutlu and Mohammed Alser and Juan Gómez-Luna, "Seminar in Computer Architecture (Fall 2021)," https://safari.ethz.ch/architecture_seminar/spring2022/doku.php?id=start.
- [118] L. Yavits *et al.*, "GIRAF: General Purpose In-Storage Resistive Associative Framework," *IEEE TPDS*, 2021.
- [119] A. Olgun *et al.*, "PiDRAM: A Holistic End-to-end FPGA-based Framework for Processing-in-DRAM," arXiv preprint arXiv:2111.00082, 2021.
- [120] S. Lee *et al.*, "A 1nm 1.25V 8Gb, 16Gb/s/pin GDDR6-based Accelerator-in-Memory supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep-Learning Applications," in *ISSCC*, 2022.
- [121] L. Ke *et al.*, "Near-Memory Processing in Action: Accelerating Personalized Recommendation with AxDIMM," *IEEE Micro*, 2021.
- [122] Y.-C. Kwon *et al.*, "25.4 A 20nm 6GB Function-In-Memory DRAM, Based on HBM2 with a 1.2 TFLOPS Programmable Computing Unit Using Bank-Level Parallelism, for Machine Learning Applications," in *ISSCC*, 2021.
- [123] S. Lee *et al.*, "Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product," in *ISCA*, 2021.
- [124] B. Asgari *et al.*, "FAFNIR: Accelerating Sparse Gathering by Using Efficient Near-Memory Intelligent Reduction," in *HPCA*, 2021.
- [125] J. M. Herruzo *et al.*, "Enabling Fast and Energy-Efficient FM-Index Exact Matching Using Processing-Near-Memory," *The Journal of Supercomputing*, 2021.
- [126] G. Singh *et al.*, "Fpga-based Near-memory Acceleration of Modern Data-intensive Applications," *IEEE Micro*, 2021.
- [127] G. Singh *et al.*, "Accelerating Weather Prediction using Near-Memory Reconfigurable Fabric," *ACM TRET*, 2021.
- [128] G. F. Oliveira *et al.*, "DAMOV: A New Methodology and Benchmark Suite for Evaluating Data Movement Bottlenecks," *IEEE Access*, 2021.
- [129] A. Boroumand *et al.*, "Google Neural Network Models for Edge Devices: Analyzing and Mitigating Machine Learning Inference Bottlenecks," in *PACT*, 2021.
- [130] A. Boroumand *et al.*, "Google Neural Network Models for Edge Devices: Analyzing and Mitigating Machine Learning Inference Bottlenecks," arXiv preprint

arXiv:2109.14320, 2021.

- [131] A. Denzler *et al.*, "Casper: Accelerating stencil computation using near-cache processing," *arXiv preprint arXiv:2112.14216*, 2021.
- [132] U. Kang *et al.*, "Co-Architecting Controllers and DRAM to Enhance DRAM Process Scaling," in *The Memory Forum*, 2014.
- [133] J. Liu *et al.*, "An Experimental Study of Data Retention Behavior in Modern DRAM Devices: Implications for Retention Time Profiling Mechanisms," in *ISCA*, 2013.
- [134] O. Mutlu, "Memory Scaling: A Systems Architecture Perspective," *IMW*, 2013.
- [135] Y. Kim *et al.*, "Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors," in *ISCA*, 2014.
- [136] O. Mutlu, "The RowHammer Problem and Other Issues We May Face as Memory Becomes Denser," in *DATE*, 2017.
- [137] S. Ghose *et al.*, "What Your DRAM Power Models Are Not Telling You: Lessons from a Detailed Experimental Study," in *SIGMETRICS*, 2018.
- [138] O. Mutlu and L. Subramanian, "Research Problems and Opportunities in Memory Systems," *SUPERFRI*, 2014.
- [139] J. S. Kim *et al.*, "Revisiting RowHammer: An Experimental Analysis of Modern DRAM Devices and Mitigation Techniques," in *ISCA*, 2020.
- [140] O. Mutlu and J. S. Kim, "RowHammer: A Retrospective," *IEEE TCAD*, 2019.
- [141] P. Frigo *et al.*, "TRRespass: Exploiting the Many Sides of Target Row Refresh," in *S&P*, 2020.
- [142] J. Kim *et al.*, "Solar-DRAM: Reducing DRAM Access Latency by Exploiting the Variation in Local Bitlines," in *ICCD*, 2018.
- [143] J. Liu *et al.*, "RAIDR: Retention-Aware Intelligent DRAM Refresh," in *ISCA*, 2012.
- [144] O. Mutlu, "Main Memory Scaling: Challenges and Solution Directions," in *More than Moore Technologies for Next Generation Computer Design*. Springer, 2015.
- [145] J. A. Mandelman *et al.*, "Challenges and Future Directions for the Scaling of Dynamic Random-Access Memory (DRAM)," *IBM JRD*, 2002.
- [146] B. C. Lee *et al.*, "Architecting Phase Change Memory as a Scalable DRAM Alternative," in *ISCA*, 2009.
- [147] L. Cocjocar *et al.*, "Are We Susceptible to Rowhammer? An End-to-End Methodology for Cloud Providers," in *S&P*, 2020.
- [148] A. G. Yağlikçi *et al.*, "BlockHammer: Preventing RowHammer at Low Cost by Blacklisting Rapidly-Accessed DRAM Rows," in *HPCA*, 2021.
- [149] M. Patel *et al.*, "The Reach Profiler (REAPER): Enabling the Mitigation of DRAM Retention Failures via Profiling at Aggressive Conditions," in *ISCA*, 2017.
- [150] S. Khan *et al.*, "The Efficacy of Error Mitigation Techniques for DRAM Retention Failures: A Comparative Experimental Study," in *SIGMETRICS*, 2014.
- [151] S. Khan *et al.*, "PARBOR: An Efficient System-Level Technique to Detect Data Dependent Failures in DRAM," in *DSN*, 2016.
- [152] S. Khan *et al.*, "Detecting and Mitigating Data-Dependent DRAM Failures by Exploiting Current Memory Content," in *MICRO*, 2017.
- [153] D. Lee *et al.*, "Adaptive-Latency DRAM: Optimizing DRAM Timing for the Common-Case," in *HPCA*, 2015.
- [154] D. Lee *et al.*, "Design-Induced Latency Variation in Modern DRAM Chips: Characterization, Analysis, and Latency Reduction Mechanisms," in *SIGMETRICS*, 2017.
- [155] K. K. Chang *et al.*, "Understanding Reduced-Voltage Operation in Modern DRAM Devices: Experimental Characterization, Analysis, and Mechanisms," in *SIGMETRICS*, 2017.
- [156] K. K. Chang *et al.*, "Understanding Latency Variation in Modern DRAM Chips: Experimental Characterization, Analysis, and Optimization," in *SIGMETRICS*, 2016.
- [157] K. K. Chang *et al.*, "Improving DRAM Performance by Parallelizing Refreshes with Accesses," in *HPCA*, 2014.
- [158] J. Meza *et al.*, "Revisiting Memory Errors in Large-Scale Production Data Centers: Analysis and Modeling of New Trends from the Field," in *DSN*, 2015.
- [159] H. David *et al.*, "Memory Power Management via Dynamic Voltage/Frequency Scaling," in *ICAC*, 2011.
- [160] Q. Deng *et al.*, "Memscale: Active Low-power Modes for Main Memory," in *ASPLOS*, 2011.
- [161] S. Hong, "Memory Technology Trend and Future Challenges," in *IEDM*, 2010.
- [162] S. Kanev *et al.*, "Profiling a Warehouse-Scale Computer," in *ISCA*, 2015.
- [163] M. K. Qureshi *et al.*, "AVATAR: A Variable-Retention-Time (VRT) Aware Refresh for DRAM Systems," in *DSN*, 2015.
- [164] L. Orosa *et al.*, "A Deeper Look into RowHammer's Sensitivities: Experimental Analysis of Real DRAM Chips and Implications on Future Attacks and Defenses," in *MICRO*, 2021.
- [165] H. Hassan *et al.*, "Uncovering In-DRAM RowHammer Protection Mechanisms: A New Methodology, Custom RowHammer Patterns, and Implications," in *MICRO*, 2021.
- [166] M. Patel *et al.*, "Harp: Practically and effectively identifying uncorrectable errors in memory chips that use on-die error-correcting codes," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 623–640.
- [167] Hybrid Memory Cube Consortium, "HMC Specification 2.0," 2014.
- [168] JEDEC, "High Bandwidth Memory (HBM) DRAM," Standard No. JESD235, 2013.
- [169] D. Lee *et al.*, "Simultaneous Multi-Layer Access: Improving 3D-Stacked Memory Bandwidth at Low Cost," *TACO*, 2016.
- [170] S. Ghose *et al.*, "Demystifying Complex Workload-DRAM Interactions: An Experimental Study," in *SIGMETRICS*, 2019.
- [171] Y. Kim *et al.*, "Ramulator: A Fast and Extensible DRAM Simulator," *CAL*, 2015.
- [172] M. Gokhale *et al.*, "Hybrid Memory Cube Performance Characterization on Data-Centric Workloads," in *IA³*, 2015.
- [173] E. Kültürsay *et al.*, "Evaluating STT-RAM as an Energy-Efficient Main Memory Alternative," in *ISPASS*, 2013.
- [174] D. B. Strukov *et al.*, "The Missing Memristor Found," *Nature*, 2008.
- [175] H.-S. P. Wong *et al.*, "Metal-Oxide RRAM," *Proc. IEEE*, 2012.
- [176] B. C. Lee *et al.*, "Phase Change Memory Architecture and the Quest for Scalability," *CACM*, 2010.
- [177] M. K. Qureshi *et al.*, "Scalable High Performance Main Memory System Using Phase-Change Memory Technology," in *ISCA*, 2009.
- [178] P. Zhou *et al.*, "A Durable and Energy Efficient Main Memory Using Phase Change Memory Technology," in *ISCA*, 2009.
- [179] B. C. Lee *et al.*, "Phase-Change Technology and the Future of Main Memory," *IEEE Micro*, 2010.
- [180] H.-S. P. Wong *et al.*, "Phase Change Memory," *Proc. IEEE*, 2010.
- [181] H. Yoon *et al.*, "Efficient Data Mapping and Buffering Techniques for Multilevel Cell Phase-Change Memories," *ACM TACO*, 2014.
- [182] H. Yoon *et al.*, "Row Buffer Locality Aware Caching Policies for Hybrid Memories," in *ICCD*, 2012.
- [183] P. Girard *et al.*, "A Survey of Test and Reliability Solutions for Magnetic Random Access Memories," *Proceedings of the IEEE*, 2020.
- [184] G. F. Oliveira *et al.*, "DAMOV: A New Methodology and Benchmark Suite for Evaluating Data Movement Bottlenecks," *arXiv:2105.03725 [cs.AR]*, 2021.
- [185] K. Hsieh *et al.*, "Accelerating Pointer Chasing in 3D-Stacked Memory: Challenges, Mechanisms, Evaluation," in *ICCD*, 2016.
- [186] G. Singh *et al.*, "NERO: A Near High-Bandwidth Memory Stencil Accelerator for Weather Prediction Modeling," in *FPL*, 2020.
- [187] R. Hadidi *et al.*, "CAIRO: A Compiler-assisted Technique for Enabling Instruction-level Offloading of Processing-in-Memory," *ACM TACO*, vol. 14, 2017.
- [188] V. Seshadri and O. Mutlu, "In-DRAM Bulk Bitwise Execution Engine," *arXiv:1905.09822 [cs.AR]*, 2020.
- [189] V. Seshadri and O. Mutlu, "The Processing Using Memory Paradigm: In-DRAM Bulk Copy, Initialization, Bitwise AND and OR," *arXiv:1610.09603 [cs.AR]*, 2016.
- [190] V. Seshadri and O. Mutlu, "Simple Operations in Memory to Reduce Data Movement," in *Advances in Computers, Volume 106*, 2017.
- [191] V. Seshadri *et al.*, "Fast Bulk Bitwise AND and OR in DRAM," *CAL*, 2015.
- [192] A. Ankit *et al.*, "PUMA: A Programmable Ultra-Efficient Memristor-Based Accelerator for Machine Learning Inference," in *ASPLOS*, 2019.
- [193] A. Ankit *et al.*, "PANTHER: A Programmable Architecture for Neural Network Training Harnessing Energy-efficient ReRAM," *IEEE TC*, 2020.
- [194] J. Ambrosi *et al.*, "Hardware-software Co-design for an Analog-digital Accelerator for Machine Learning," in *ICRC*, 2018.
- [195] P. Bruel *et al.*, "Generalize or Die: Operating Systems Support for Memristor-based Accelerators," in *ICRC*, 2017.
- [196] S. Huang *et al.*, "Mixed Precision Quantization for ReRAM-based DNN Inference Accelerators," in *ASP-DAC*, 2021.
- [197] Y. Kim *et al.*, "A Case for Exploiting Subarray-Level Parallelism (SALP) in DRAM," in *ISCA*, 2012.
- [198] UPMEM, "Introduction to UPMEM PIM. Processing-in-memory (PIM) on DRAM Accelerator (White Paper)," 2018.
- [199] F. Devaux, "The True Processing In Memory Accelerator," in *Hot Chips*, 2019.
- [200] D. Weber *et al.*, "Current and Future Challenges of DRAM Metallization," in *IITC*, 2005.
- [201] Y. Peng *et al.*, "Design, Packaging, and Architectural Policy Co-optimization for DC Power Integrity in 3D DRAM," in *DAC*, 2015.
- [202] M. Yuffe *et al.*, "A Fully Integrated Multi-CPU, GPU and Memory Controller 32nm processor," in *ISSCC*, 2011.
- [203] R. Christy *et al.*, "8.3 A 3GHz ARM Neoverse N1 CPU in 7nm FinFET for Infrastructure Applications," in *ISSCC*, 2020.
- [204] T. Singh *et al.*, "3.2 Zen: A Next-generation High-performance x86 Core," in *ISSCC*, 2017.
- [205] O. Mutlu, "Lecture 18c: Fine-Grained Multithreading," <https://safari.ethz.ch/digitaltechnik/spring2020/lib/exe/fetch.php?media=onur-digitaldesign-2020-lecture18c-fgmt-beforelecture.pptx>, video available at <http://www.youtube.com/watch?v=bu5dxKTvQVs>, 2020, Digital Design and Computer Architecture. Spring 2020.
- [206] J. L. Hennessy and D. A. Patterson, *Computer Architecture - A Quantitative Approach, 5th Edition, Chapter 3: Instruction-level Parallelism and Its Exploitation*. Morgan Kaufmann, 2012.
- [207] B. J. Smith, "A Pipelined, Shared Resource MIMD Computer," in *ICPP*, 1978.

- [208] B. J. Smith, "Architecture and Applications of the HEP Multiprocessor Computer System," in *SPIE, Real-Time signal processing IV*, 1981.
- [209] J. E. Thornton, *CDC 6600: Design of a Computer*, 1970.
- [210] O. Mutlu, "Lecture 19: SIMD Processors," <https://safari.ethz.ch/digitaltechnik/spring2020/lib/exe/fetch.php?media=onur-digitaldesign-2020-lecture19-simd-beforelecture.pptx>, video available at <http://www.youtube.com/watch?v=2XZ3ik6xSzM>, 2020, Digital Design and Computer Architecture. Spring 2020.
- [211] J. L. Hennessy and D. A. Patterson, *Computer Architecture - A Quantitative Approach, 5th Edition, Chapter 4: Data-level Parallelism in Vector, SIMD, and GPU Architectures*. Morgan Kaufmann, 2012.
- [212] M. J. Flynn, "Very High-speed Computing Systems," *Proc. IEEE*, 1966.
- [213] UPMEM, "UPMEM User Manual. Version 2021.1.0," 2021.
- [214] SAFARI Research Group, "PRIM Benchmark Suite," <https://github.com/CMU-SAFARI/prim-benchmarks>.
- [215] S. Williams *et al.*, "Roofline: An Insightful Visual Performance Model for Multi-core Architectures," *CACM*, 2009.
- [216] G. Hager and G. Wellein, *Introduction to High Performance Computing for Scientists and Engineers, Chapter 5: Basics of Parallelization*. CRC Press, 2010.
- [217] G. M. Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale," in *AFIPS*, 1967.
- [218] J. L. Gustafson, "Reevaluating Amdahl's Law," *CACM*, 1988.
- [219] A. Saini, "Design of the Intel Pentium Processor," in *ICCD*, 1993.
- [220] D. Jaggard, "ARM Architecture and Systems," *IEEE Annals of the History of Computing*, 1997.
- [221] A. S. Waterman, "Design of the RISC-V Instruction Set Architecture," Ph.D. dissertation, UC Berkeley, 2016.
- [222] Y. Kim and O. Mutlu, *Computing Handbook: Computer Science and Software Engineering, 3rd Edition, Chapter 1: Memory Systems*. CRC Press, 2014.
- [223] O. Mutlu, "Lecture 2b: Data Retention and Memory Refresh," <https://bit.ly/3KhVgAq>, video available at <http://www.youtube.com/watch?v=v702wUnaWGE>, 2020, Computer Architecture. Fall 2020.
- [224] O. Mutlu, "Lecture 3b: Memory Systems: Challenges and Opportunities," <https://bit.ly/3sQIFyU>, video available at <http://www.youtube.com/watch?v=Q2FbUxD7GHs>, 2020, Computer Architecture. Fall 2020.
- [225] O. Mutlu, "Lecture 4a: Memory Systems: Solution Directions," <https://safari.ethz.ch/architecture/fall2020/lib/exe/fetch.php?media=onur-comparch-fall2020-lecture4a-memory-solutions-afterlecture.pptx>, video available at <http://www.youtube.com/watch?v=PANTCVTYe8M>, 2020, Computer Architecture. Fall 2020.
- [226] JEDEC, "JESD79-4 DDR4 SDRAM standard," 2012.
- [227] UPMEM, "UPMEM Website," <https://www.upmem.com>, 2020.
- [228] V. Seshadri *et al.*, "Gather-Scatter DRAM: In-DRAM Address Translation to Improve the Spatial Locality of Non-Unit Strided Accesses," in *MICRO*, 2015.
- [229] Intel, "Intel Xeon Silver 4215 Processor," <https://ark.intel.com/content/www/us/en/ark/products/193389/intel-xeon-silver-4215-processor-11m-cache-2-50-ghz.html>, 2019.
- [230] Intel, "Intel Xeon Silver 4110 Processor," <https://ark.intel.com/content/www/us/en/ark/products/123547/intel-xeon-silver-4110-processor-11m-cache-2-10-ghz.html>, 2017.
- [231] R. Jodin and R. Cimadomo, "UPMEM. Personal Communication," October 2020.
- [232] O. Mutlu, "Lecture 20: Graphics Processing Units," <https://safari.ethz.ch/digitaltechnik/spring2020/lib/exe/fetch.php?media=onur-digitaldesign-2020-lecture20-gpu-beforelecture.pptx>, video available at <http://www.youtube.com/watch?v=dg0VN-XCGKQ>, 2020, Digital Design and Computer Architecture. Spring 2020.
- [233] T. Rauber and G. Rünger, *Parallel Programming, 2nd Edition, Chapter 3: Parallel Programming Models*. Springer, 2013.
- [234] UPMEM, "UPMEM Software Development Kit (SDK)," <https://sdk.upmem.com>, 2021.
- [235] C. Lattner and V. Adve, "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation," in *CGO*, 2004.
- [236] UPMEM, "Compiler Explorer," <https://dpu.dev>, 2020.
- [237] J. D. McCalpin, "Memory Bandwidth and Machine Balance in Current High Performance Computers," *IEEE TCCA newsletter*, 1995.
- [238] LLVM, "Compiler-RT, LLVM project," <https://github.com/llvm/llvm-project/tree/main/compiler-rt/lib/builtins>, 2021.
- [239] P. R. Luszczek *et al.*, "The HPC Challenge (HPCC) Benchmark Suite," in *SC*, 2006.
- [240] Intel, "Intel Advanced Vector Extensions Programming Reference," 2011.
- [241] Intel, "Intel Xeon Processor E3-1225 v6," <https://ark.intel.com/content/www/us/en/ark/products/97476/intel-xeon-processor-e3-1225-v6-8m-cache-3-30-ghz.html>, 2017.
- [242] Intel, "Intel Advisor," 2020.
- [243] L. S. Blackford *et al.*, "An Updated Set of Basic Linear Algebra Subprograms (BLAS)," *ACM TOMS*, 2002.
- [244] Y. Saad, *Iterative Methods for Sparse Linear Systems, 2nd Edition, Chapter 1: Parallel Implementations*. SIAM, 2003.
- [245] Y. Saad, *Iterative Methods for Sparse Linear Systems, 2nd Edition, Chapter 3: Sparse Matrices*. SIAM, 2003.
- [246] W. Liu and B. Vinter, "CSR5: An Efficient Storage Format for Cross-platform Sparse Matrix-vector Multiplication," in *ICS*, 2015.
- [247] K. Kanellopoulos *et al.*, "SMASH: Co-designing Software Compression and Hardware-Accelerated Indexing for Efficient Sparse Matrix Operations," in *MI-CRO*, 2019.
- [248] S. Ceri and G. Gottlob, "Translating SQL into Relational Algebra: Optimization, Semantics, and Equivalence of SQL Queries," *IEEE TSE*, 1985.
- [249] G. E. Blelloch, "Scans as Primitive Parallel Operations," *IEEE TC*, 1989.
- [250] J. Gómez-Luna *et al.*, "In-Place Data Sliding Algorithms for Many-Core Architectures," in *ICPP*, 2015.
- [251] S. Yan *et al.*, "StreamScan: Fast Scan Algorithms for GPUs without Global Barrier Synchronization," in *PPoPP*, 2013.
- [252] D. E. Knuth, "Optimum Binary Search Trees," *Acta informatica*, 1971.
- [253] C.-C. M. Yeh *et al.*, "Matrix profile I: All Pairs Similarity Joins for Time Series: a Unifying View that Includes Motifs, Discords and Shapelets," in *ICDM*, 2016.
- [254] Y. Zhu *et al.*, "Matrix Profile XI: SCRIMP++: Time Series Motif Discovery at Interactive Speeds," in *ICDM*, 2018.
- [255] A. Bundy and L. Wallen, "Breadth-first Search," in *Catalogue of Artificial Intelligence Tools*. Springer, 1984.
- [256] D. B. Kirk *et al.*, *Programming Massively Parallel Processors, 3rd Edition, Chapter 12 - Parallel Patterns: Graph Search*. Morgan Kaufmann, 2017.
- [257] L. Luo *et al.*, "An Effective GPU Implementation of Breadth-first Search," in *DAC*, 2010.
- [258] G. E. Hinton, "Learning Translation Invariant Recognition in a Massively Parallel Networks," in *PARLE*, 1987.
- [259] S. Needleman and C. Wunsch, "A General Method Applicable to the Search for Similarities in the Amino Acid Sequence of Two Proteins," *Journal of Molecular Biology*, 1970.
- [260] J. Gómez-Luna *et al.*, "An Optimized Approach to Histogram Computation on GPU," *MVAP*, 2013.
- [261] R. Rabenseifner, "Optimization of Collective Reduction Operations," in *ICCS*, 2004.
- [262] M. Harris, "Optimizing Parallel Reduction in CUDA," *Nvidia Developer Technology*, 2007.
- [263] S. G. De Gonzalo *et al.*, "Automatic Generation of Warp-level Primitives and Atomic Instructions for Fast and Portable Parallel Reduction on GPUs," in *CGO*, 2019.
- [264] D. B. Kirk *et al.*, *Programming Massively Parallel Processors, 3rd Edition, Chapter 8 - Parallel Patterns: Prefix Sum: An Introduction to Work Efficiency in Parallel Algorithms*. Morgan Kaufmann, 2017.
- [265] S. Sengupta *et al.*, "Efficient Parallel Scan Algorithms for GPUs," *NVIDIA Technical Report NVR-2008-003*, 2008.
- [266] Y. Dotsenko *et al.*, "Fast Scan Algorithms on Graphics Processors," in *ICS*, 2008.
- [267] A. Cayley, "II. A Memoir on the Theory of Matrices," *Philosophical Transactions of the Royal Society of London*, 1858.
- [268] T. W. Hungerford, *Abstract Algebra: An Introduction, 3rd Edition*. Cengage Learning, 2012.
- [269] I.-J. Sung *et al.*, "In-place Transposition of Rectangular Matrices on Accelerators," in *PPoPP*, 2014.
- [270] J. Gomez-Luna *et al.*, "In-place Matrix Transposition on GPUs," *IEEE TPDS*, 2016.
- [271] I.-J. Sung *et al.*, "DL: A Data Layout Transformation System for Heterogeneous Computing," in *Innovative Parallel Computing*, 2012.
- [272] J. Gómez-Luna *et al.*, "Performance Modeling of Atomic Additions on GPU Scratchpad Memory," *IEEE TPDS*, 2013.
- [273] R. F. Boisvert *et al.*, "Matrix Market: A Web Resource for Test Matrix Collections," in *Quality of Numerical Software*, 1996.
- [274] E. Cho *et al.*, "Friendship and Mobility: User Movement in Location-Based Social Networks," in *KDD*, 2011.
- [275] D. Chakrabarti *et al.*, "R-MAT: A Recursive Model for Graph Mining," in *SDM*, 2004.
- [276] J. H. van Hateren and A. van der Schaaf, "Independent Component Filters of Natural Images Compared with Simple Cells in Primary Visual Cortex," *Proceedings of the Royal Society of London. Series B: Biological Sciences*, 1998.
- [277] NVIDIA, "NVIDIA Titan V," <https://www.nvidia.com/en-us/titan/titan-v/>, 2017.
- [278] NVIDIA, "NVIDIA Tesla V100 GPU Architecture. White Paper," <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>, 2017.
- [279] Intel Open Source, "RAPL Power Meter," <https://01.org/rapl-power-meter>, 2014.
- [280] NVIDIA, "NVIDIA System Management Interface Program," <http://developer.download.nvidia.com/compute/DCGM/docs/nvidia-smi-367.38.pdf>, 2016.
- [281] Intel, "Intel 64 and IA-32 Architectures Software Developer's Manual," *Volume 3B: System Programming Guide*, 2011.
- [282] NVIDIA, "CUDA Samples v. 11.2," 2021.
- [283] N. Bell and J. Hoberock, "Thrust: A Productivity-Oriented Library for CUDA," in *GPU Computing Gems, Jade Edition*, 2012.
- [284] G.-J. van den Braak *et al.*, "Simulation and Architecture Improvements of Atomic Operations on GPU Scratchpad Memory," in *ICCD*, 2013.

- [285] D. Merrill and NVIDIA-Labs, "CUDA Unbound (CUB) Library," *NVIDIA-Labs*, 2015.
- [286] B. Catanzaro *et al.*, "A Decomposition for In-place Matrix Transposition," in *PPoPP*, 2014.
- [287] J. Gómez-Luna *et al.*, "Chai: Collaborative Heterogeneous Applications for Integrated-architectures," in *ISPASS*, 2017.
- [288] V. Seshadri, "Simple DRAM and Virtual Memory Abstractions to Enable Highly Efficient Memory Systems," Ph.D. dissertation, Carnegie Mellon University, 2016.
- [289] L. Song *et al.*, "GraphR: Accelerating Graph Processing using ReRAM," in *HPCA*, 2018.
- [290] A. Boroumand *et al.*, "Polynesia: Enabling Effective Hybrid Transactional/Analytical Databases with Specialized Hardware/Software Co-Design," *arXiv:2103.00798 [cs.AR]*, 2021.
- [291] A. Boroumand *et al.*, "Mitigating Edge Machine Learning Inference Bottlenecks: An Empirical Study on Accelerating Google Edge Models," *arXiv:2103.00768 [cs.AR]*, 2021.
- [292] M. A. Alves *et al.*, "Opportunities and Challenges of Performing Vector Operations inside the DRAM," in *MEMSYS*, 2015.
- [293] W. Huangfu *et al.*, "MEDAL: Scalable DIMM Based Near Data Processing Accelerator for DNA Seeding Algorithm," in *MICRO*, 2019.
- [294] N. Hajinazar *et al.*, "The Virtual Block Interface: A Flexible Alternative to the Conventional Virtual Memory Framework," in *ISCA*, 2020.
- [295] D. Lavenier *et al.*, "DNA Mapping using Processor-in-Memory Architecture," in *BIBM*, 2016.
- [296] D. Lavenier *et al.*, "Variant Calling Parallelization on Processor-in-Memory Architecture," in *BIBM*, 2020.
- [297] V. Zois *et al.*, "Massively Parallel Skyline Computation for Processing-in-Memory Architectures," in *PACT*, 2018.
- [298] J. Nider *et al.*, "Processing in Storage Class Memory," in *HotStorage*, 2020.
- [299] C. Giannoula *et al.*, "SparseP: Towards Efficient Sparse Matrix Vector Multiplication on Real Processing-In-Memory Systems," *arXiv preprint arXiv:2201.05072*, 2022.
- [300] H. Shin *et al.*, "McDRAM: Low latency and energy-efficient matrix computations in DRAM," *IEEE TCADICS*, 2018.
- [301] S. Cho *et al.*, "McDRAM v2: In-Dynamic Random Access Memory Systolic Array Accelerator to Address the Large Model Problem in Deep Neural Networks on the Edge," *IEEE Access*, 2020.
- [302] J. Gómez-Luna *et al.*, "Benchmarking Memory-Centric Computing Systems: Analysis of Real Processing-in-Memory Hardware," in *IGSC*, 2021.
- [303] S. Che *et al.*, "Rodinia: A Benchmark Suite for Heterogeneous Computing," in *IISWC*, 2009.

9 APPENDIX

This appendix presents some additional results for one of our microbenchmarks (Section 9.1) and four of the PrIM benchmarks (Section 9.2). Section 9.3 shows the sources of the CPU and GPU versions of PrIM benchmarks.

9.1 Arithmetic Throughput versus Number of Tasklets

Figure 18 presents arithmetic throughput results for different numbers of tasklets at different operational intensities. This figure shows a different view of the experimental results presented in Figure 9, with the goal of showing the variation in arithmetic throughput for different operational intensities.

We make two key observations from Figure 18.

First, for any data type and operation, the highest possible throughput is achieved at 11 tasklets, i.e., the number of tasklets to fully utilize the pipeline. However, the operational intensity at which the highest throughput value is reached depends on the actual data type and operation. For example, the highest throughput of 32-bit integer addition is achieved at $\frac{1}{4}$ OP/B, i.e., 1 addition per 32-bit element. For floating point multiplication, the highest throughput is achieved at $\frac{1}{128}$ OP/B, i.e., 1 multiplication every 32 32-bit elements.

Second, for lower operational intensities, the number of tasklets necessary to reach the saturation throughput is less than 11. This happens in the memory-bound regions, where the MRAM access latency dominates the overall latency. This observation is in line with our observations for COPY and ADD benchmarks in Section 3.2.2.

9.2 Extended results for Needleman-Wunsch, image histogram, reduction, and scan

This section presents some additional results for four of the PrIM benchmarks. First, we present an extended evaluation of NW (Section 9.2.1). Second, we compare HST-S and HST-L for different histogram sizes (Section 9.2.2). Third, we show an evaluation of RED with three different mechanisms to perform local intra-DPU reduction (Section 9.2.3). Fourth, we compare SCAN-SSA and SCAN-RSS for different array sizes (Section 9.2.4).

9.2.1 Needleman-Wunsch. We present additional results for the weak scaling experiment of NW. In this experiment, we increase the length of the sequences to align proportionally to the number of DPUs. Thus, the size of the 2D score matrix increases quadratically with the number of DPUs. Figure 19 shows weak scaling results of (a) the complete execution of NW (including all iterations) and (b) the execution of only the longest diagonal.

We make two observations from Figure 19. First, the execution times on the DPUs for the complete execution (Figure 19a) increase with the number of DPUs, since the size of the problem (the 2D score matrix) increases quadratically. We make the same observation in Section 5.1.2. Second, the execution times on the DPUs for the longest diagonal (Figure 19b) remain flat as the number of DPUs increases. The reason is that the length of the longest diagonal increases linearly with the length of the sequences and the number of DPUs. As a result, we observe linear weak scaling for the longest diagonal.

These results show (1) that a larger number of active DPUs is more beneficial for NW in the computation of the longest diagonals of the 2D score matrix, and (2) why we do not observe linear scaling for the complete NW.

9.2.2 Image Histogram. We present results for different histogram sizes for our two versions of histogram (HST-S, HST-L). Figure 20 shows the execution time results for histogram sizes between 64 and 4096. The input is the one specified in Table 3, which is an image of 12-bit depth (thus, maximum histogram size is 4096).

The results show that HST-S is $1.6 - 2.5\times$ faster than HST-L for histograms between 64 and 1024 bins. The performance of HST-S gets worse when increasing the histogram size because the number of tasklets that it is possible to run on a DPU reduces. For example, for 512 bins, only 8 tasklets can be launched because of the limited amount of WRAM (each tasklet has its own local histogram). For 4046 bins, HST-S can only launch 2 tasklets. After 2048 bins, HST-L performs faster, as its execution time is independent of the histogram size.

9.2.3 Reduction. We compare three versions of RED that we introduce in Section 4.12. Recall that RED has two steps. In the first step, each tasklet accumulates the values of an assigned chunk of the input array. In the second step, RED performs the final reduction of the local sums of all tasklets. The difference between the three

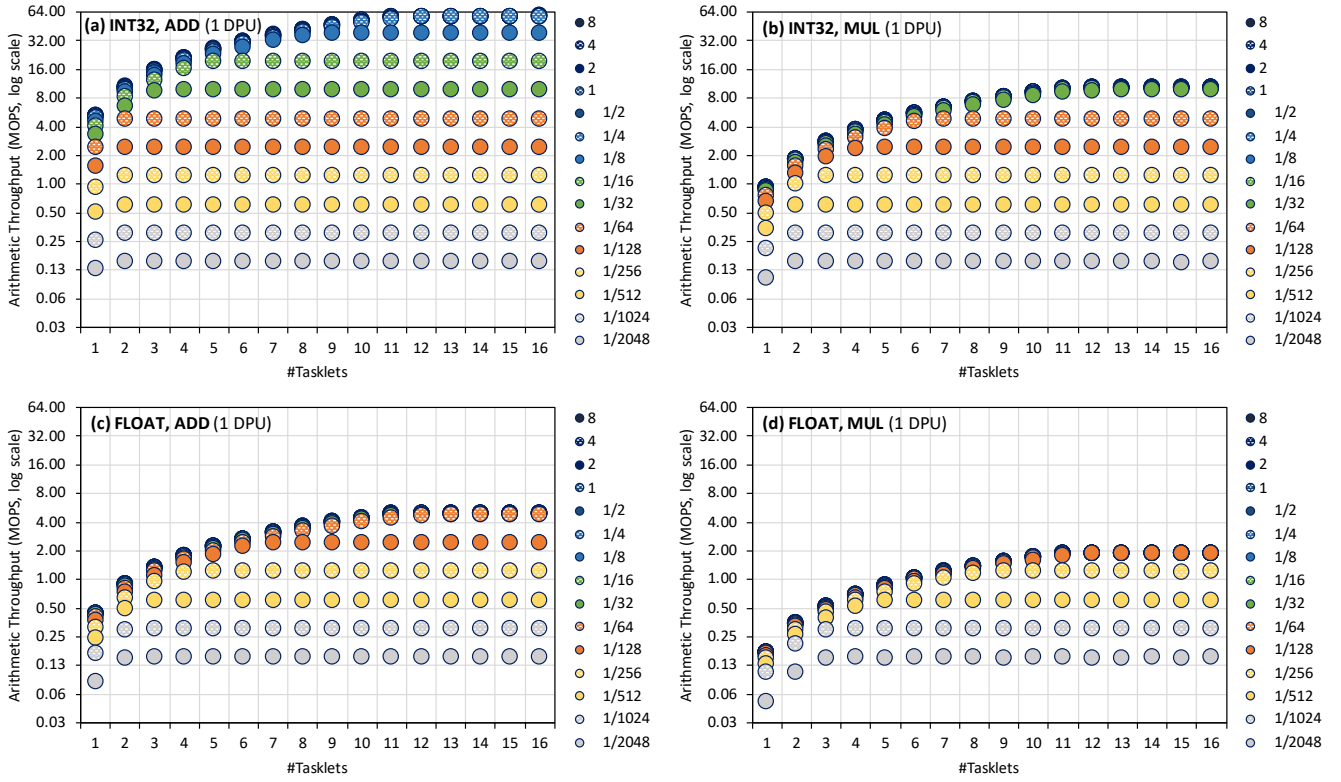


Figure 18: Arithmetic throughput versus number of tasklets for different operational intensities of (a) 32-bit integer addition, (b) 32-bit integer multiplication, (c) 32-bit floating point addition, and (d) 32-bit floating point multiplication. The legend shows the operational intensity values (in OP/B). The y-axis is log scale.

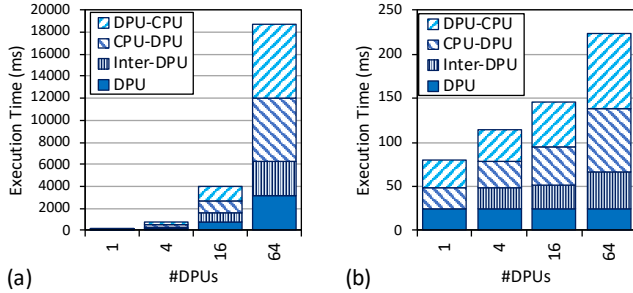


Figure 19: Weak scaling evaluation of NW: (a) complete execution of NW, (b) execution of the longest diagonal.

versions is in how the second step is implemented. The first version uses a single tasklet to perform a sequential reduction in the second step (SINGLE in Figures 21 to 23). The other two versions implement a parallel tree-based reduction in the second step (see Section 4.12). The only difference between the other two versions is the synchronization primitive used for synchronization at the end of each tree level: (1) barriers for all tasklets (BARRIER in Figures 21 to 23), or (2) handshakes between pairs of tasklets (HANDS in Figures 21 to 23). Figure 21 shows the number of execution cycles

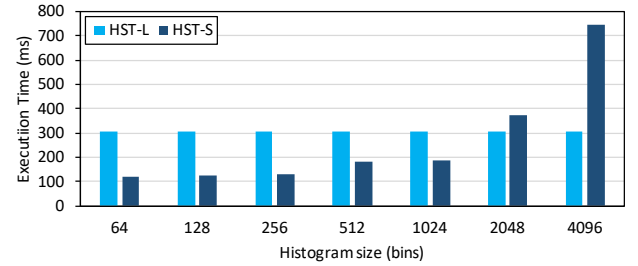


Figure 20: Execution times (ms) of two versions of histogram (HST-L, HST-S) on 1 DPU.

needed to perform sequential (SINGLE) or the parallel tree-based (BARRIER, HANDS) reduction for 2 to 16 tasklets on one DPU.

We observe that the most efficient of the three versions is the sequential reduction (SINGLE). However, it is only a few cycles faster (6% faster with 16 tasklets) that the tree-based version with handshakes (HANDS). We also observe the high cost of barriers when the number of tasklets increases. These results indicate that synchronization primitives impose high overhead in the current implementation of the UPMEM PIM architecture. Nevertheless, the relative weight of the final reduction is negligible when the input

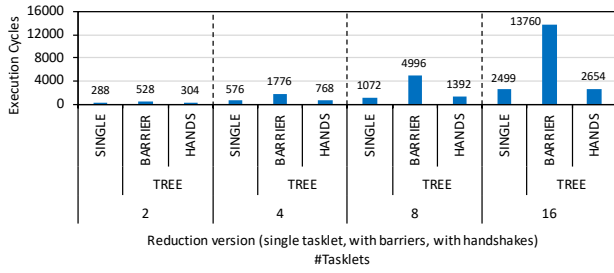


Figure 21: Effect of sequential reduction (SINGLE) vs. parallel tree-based reductions (BARRIER, HANDS), in the second step of the RED benchmark.

array is large. Figure 22 shows the execution cycles of the three versions for an input array of 2K 64-bit elements with 2-16 tasklets on one DPU. The difference between the three versions is very small, but we still observe that SINGLE is slightly faster (i.e., 2% over HANDS, and 47% over BARRIER).

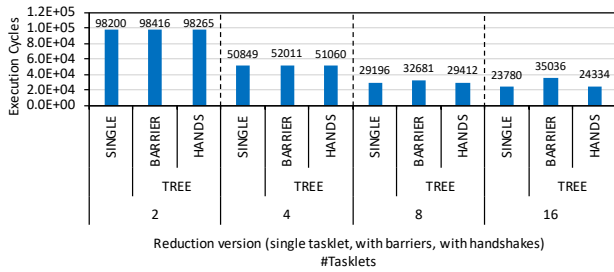


Figure 22: Execution cycles of three versions of reduction of 2K 64-bit elements on 1 DPU.

For an array of 2M 64-bit elements (Figure 23), the difference in performance of the three versions is completely negligible, since most of the execution cycles are spent in the first step of RED.

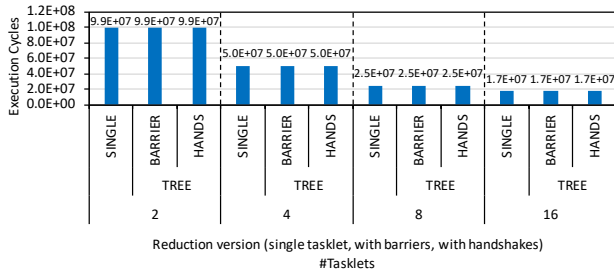


Figure 23: Execution cycles of three versions of reduction of 2M 64-bit elements on 1 DPU.

9.2.4 Prefix Sum (Scan). We compare the execution time of our two versions of scan, SCAN-SSA and SCAN-RSS, for different array sizes (2048, 4096, 8192, 16384, 65536 elements) on the DPU. Figure 24 shows the execution time results. For both versions, the figure shows the breakdown of DPU kernel times ("DPU Scan" + "DPU

Add" in SCAN-SSA, and "DPU Reduction" + "DPU Scan" in SCAN-RSS) and the intermediate scan in the host CPU ("Inter-DPU").

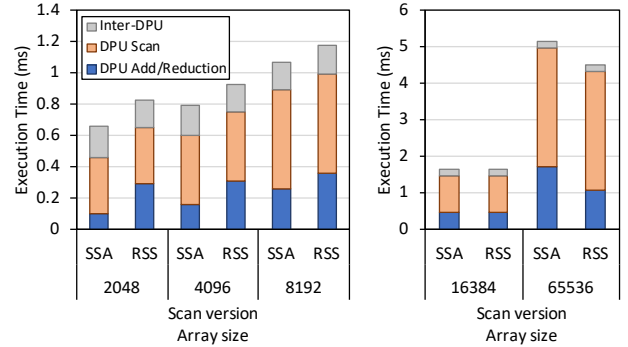


Figure 24: Two versions of scan (SCAN-SSA, SCAN-RSS) on 1 DPU.

The main observation from these results is that SCAN-SSA runs faster for small arrays (2048-8192). Scan kernel time and Inter-DPU time are very similar in both SCAN-SSA and SCAN-RSS, but the Add kernel is faster than the Reduction kernel for small sizes. The reason is that the Reduction kernel is burdened by the overhead of intra-DPU synchronization (barrier) and the final reduction, where only a single tasklet works. This overhead becomes negligible for larger arrays. As a result, SCAN-RSS is faster for large arrays (more than 16384 elements).

9.3 CPU and GPU versions of the benchmarks

Table 5 shows the sources of the CPU and GPU versions of PrIM benchmarks, which we use for comparison purposes in Section 5.2. We provide these CPU and GPU versions as part of our PrIM benchmark suite [214].

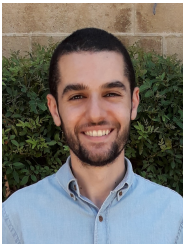
Table 5: CPU and GPU versions of PrIM benchmarks.

Benchmark	CPU version	GPU version
VA	OpenMP (custom)	CUDA SDK [282]
GEMV	OpenMP (custom)	CUDA (custom)
SpMV	OpenMP (custom)	CUDA (custom)
SEL	DS algorithms [250]	DS algorithms [250]
UNI	DS algorithms [250]	DS algorithms [250]
BS	OpenMP (custom)	CUDA (custom)
TS	[254]	[254]
BFS	OpenMP (custom)	CUDA [256]
NW	Rodinia [303]	Rodinia [303]
MLP	OpenMP (custom)	CUDA (custom)
HST	Chai [287]	Chai [287]
RED	Thrust [283]	Thrust [283]
SCAN	Thrust [283]	Thrust [283]
TRNS	[270]	[270]



Juan Gómez-Luna is a senior researcher and lecturer at SAFARI Research Group @ ETH Zürich. He received the BS and MS degrees in Telecommunication Engineering from the University of Sevilla, Spain, in 2001, and the PhD degree in Computer Science from the University of Córdoba, Spain, in 2012.

Between 2005 and 2017, he was a faculty member of the University of Córdoba. His research interests focus on processing-in-memory, memory systems, heterogeneous computing, and hardware and software acceleration of medical imaging and bioinformatics. He is the lead author of PRIM (<https://github.com/CMU-SAFARI/prim-benchmarks>), the first publicly-available benchmark suite for a real-world processing-in-memory architecture, and Chai (<https://github.com/chai-benchmarks/chai>), a benchmark suite for heterogeneous systems with CPU/GPU/FPGA.



Izzat El Hajj is an Assistant Professor in the Department of Computer Science at the American University of Beirut. He received his M.S. and Ph.D. in Electrical and Computer Engineering in 2018 from the University of Illinois at Urbana-Champaign, where he received the Dan Vivoli Endowed Fellowship. Prior to that, he received his B.E. in Electrical and Computer Engineering in 2011 from the American University of Beirut, where he

received the Distinguished Graduate Award. His research interests are in application acceleration and programming support for emerging accelerators and memories, with a particular interest in GPUs and processing-in-memory.



Ivan Fernandez received his B.S. degree in computer engineering and his M.S. degree in mechatronics engineering from University of Malaga in 2017 and 2018, respectively. He is currently working toward the Ph.D. degree at the University of Malaga. His current research interests include processing in memory, near-data processing, stacked memory architectures, high-performance computing, transprecision computing, and time

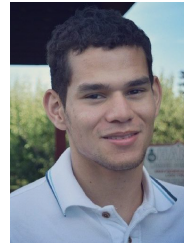
series analysis.



Christina Giannoula Christina Giannoula is a Ph.D. student at the School of Electrical and Computer Engineering of the National Technical University of Athens. She received a Diploma in Electrical and Computer Engineering from NTUA in 2016, graduating in the top 2% of her class. Her research interests lie in the intersection of computer architecture

and high-performance computing. Specifically, her research focuses

on the hardware/software co-design of emerging applications, including graph processing, pointer-chasing data structures, machine learning workloads, and sparse linear algebra, with modern computing paradigms, such as large-scale multicore systems and near-data processing architectures. She is a member of ACM, ACM-W, and of the Technical Chamber of Greece.



Geraldo F. Oliveira received a B.S. degree in computer science from the Federal University of Viçosa, Viçosa, Brazil, in 2015, and an M.S. degree in computer science from the Federal University of Rio Grande do Sul, Porto Alegre, Brazil, in 2017. Since 2018, he has been working toward a Ph.D. degree with Onur Mutlu at ETH Zürich, Zürich, Switzerland. His current research interests include system support for processing-in-memory and

processing-using-memory architectures, data-centric accelerators for emerging applications, approximate computing, and emerging memory systems for consumer devices. He has several publications on these topics.



Onur Mutlu is a Professor of Computer Science at ETH Zurich. He is also a faculty member at Carnegie Mellon University, where he previously held the Strecker Early Career Professorship. His current broader research interests are in computer architecture, systems, hardware security, and bioinformatics. A variety of techniques he, along with his group and collaborators, has invented over the years have influenced industry

and have been employed in commercial microprocessors and memory/storage systems. He obtained his PhD and MS in ECE from the University of Texas at Austin and BS degrees in Computer Engineering and Psychology from the University of Michigan, Ann Arbor. He started the Computer Architecture Group at Microsoft Research (2006-2009), and held various product and research positions at Intel Corporation, Advanced Micro Devices, VMware, and Google. He received the IEEE High Performance Computer Architecture Test of Time Award, the IEEE Computer Society Edward J. McCluskey Technical Achievement Award, ACM SIGARCH Maurice Wilkes Award, the inaugural IEEE Computer Society Young Computer Architect Award, the inaugural Intel Early Career Faculty Award, US National Science Foundation CAREER Award, Carnegie Mellon University Ladd Research Award, faculty partnership awards from various companies, and a healthy number of best paper or "Top Pick" paper recognitions at various computer systems, architecture, and security venues. He is an ACM Fellow, IEEE Fellow for, and an elected member of the Academy of Europe (Academia Europaea). His computer architecture and digital logic design course lectures and materials are freely available on YouTube (<https://www.youtube.com/OnurMutluLectures>), and his research group makes a wide variety of software and hardware artifacts

freely available online (<https://safari.ethz.ch/>). For more information, please see his webpage at <https://people.inf.ethz.ch/omutlu/>.